



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería Informática

SmartEye Flow

Analizador de imágenes con flujos
encadenados de inteligencia artificial.



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 31 de mayo
de 2026

Tutores: José Luis Garrido Labrador y José
Miguel Ramírez Sanz

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	18
Apéndice B Especificación de Requisitos	29
B.1. Introducción	29
B.2. Objetivos generales	30
B.3. Catálogo de requisitos	31
B.4. Especificación de requisitos	37
Apéndice C Especificación de diseño	47
C.1. Introducción	47
C.2. Diseño de datos	47
C.3. Diseño arquitectónico	54
C.4. Diseño procedimental	58
C.5. Diseño de interfaces	73
Apéndice D Documentación técnica de programación	83
D.1. Introducción	83
D.2. Estructura de directorios	84

D.3. Manual del programador	85
D.4. Compilación, instalación y ejecución del proyecto	94
D.5. Pruebas del sistema	103
Apéndice E Documentación de usuario	109
E.1. Introducción	109
E.2. Requisitos de usuarios	109
E.3. Instalación	110
E.4. Manual del usuario	111
Apéndice F Anexo de sostenibilización curricular	113
F.1. Introducción	113
F.2. Innovación, infraestructura y mantenibilidad	113
F.3. Uso responsable de recursos y datos	114
F.4. Relación con el trabajo decente y la formación profesional . .	114
F.5. Conclusión	115
Bibliografía	117

Índice de figuras

A.1. Informe de trabajo completado del Sprint 1.	3
A.2. Informe de trabajo completado del Sprint 2.	4
A.3. Informe de trabajo completado del Sprint 3.	5
A.4. Informe de trabajo completado del Sprint 4.	6
A.5. Informe de trabajo completado del Sprint 5.	7
A.6. Informe de trabajo completado del Sprint 6.	8
A.7. Informe de trabajo completado del Sprint 7.	9
A.8. Informe de trabajo completado del Sprint 8.	10
A.9. Informe de trabajo completado del Sprint 9.	11
A.10. Informe de trabajo completado del Sprint 10.	12
A.11. Informe de trabajo completado del Sprint 11.	13
A.12. Informe de trabajo completado del Sprint 12.	14
A.13. Informe de trabajo completado del Sprint 13.	15
A.14. Informe de trabajo completado del Sprint 14.	16
A.15. Informe de trabajo completado del Sprint 15.	18
 B.1. Diagrama de casos de uso del sistema	 37
 C.1. Diagrama entidad-relación del sistema	 48
C.2. Diagrama del modelo relacional del sistema	48
C.3. Arquitectura MVC adaptada a Flask	55
C.4. Vista de módulos y dependencias	56
C.5. Diagrama de la capa de servicios	57
C.6. CU-1 Registrarse	59
C.7. CU-2 Iniciar sesión	59
C.8. CU-3 Gestionar perfil	60
C.9. CU-4 Solicitar baja de cuenta	60
C.10. CU-5 Consultar tienda de servicios	61

C.11.CU-6 Suscribirse a un plan	62
C.12.CU-7 Alquilar un modelo de IA	62
C.13.CU-8 Consultar compras y servicios activos	63
C.14.CU-9 Consultar guía de compra	63
C.15.CU-10 Consultar pipelines disponibles	64
C.16.CU-11 Cargar y modificar configuración del pipeline	65
C.17.CU-12 Ejecutar pipeline sobre una imagen	66
C.18.CU-13 Visualizar y descargar resultados	67
C.19.CU-14 Consultar historial de ejecuciones	68
C.20.CU-15 Acceder al panel de administración	69
C.21.CU-16 Gestionar usuarios desde administración	70
C.22.CU-17 Gestionar pipelines desde administración	71
C.23.CU-18 Consultar ejecuciones globales	72
C.24.CU-19 Ejecutar tareas de mantenimiento	72
C.25.Diagrama de navegabilidad	75
C.26.Relación entre interfaces y backend	76
C.27.Flujo de ejecución desde la interfaz	76
C.28.Página inicial funcional	77
C.29.Inicio de sesión inicial	78
C.30.Registro inicial	78
C.31.Panel inicial del usuario	78
C.32.Tienda inicial	79
C.33.Guía inicial de pipelines	79
C.34.Suscripciones y alquileres iniciales	80
C.35.Perfil inicial	80
C.36.Historial inicial	80
C.37.Resultados iniciales	81
C.38.Gestión inicial de usuarios	81
C.39.Actividad global inicial	82
C.40.Administración inicial de pipelines	82
D.1. Extensión de modelos y modos	90
D.2. Informe de cobertura del backend	105
D.3. Informe de SonarCloud	108

Índice de tablas

A.1. Sprint 1 – Resumen.	2
A.2. Sprint 2 – Resumen.	4
A.3. Sprint 3 – Resumen.	5
A.4. Sprint 4 – Resumen.	6
A.5. Sprint 5 – Resumen.	7
A.6. Sprint 6 – Resumen.	8
A.7. Sprint 7 – Resumen.	9
A.8. Sprint 8 – Resumen.	10
A.9. Sprint 9 – Resumen.	11
A.10.Sprint 10 – Resumen.	12
A.11.Sprint 11 – Resumen.	13
A.12.Sprint 12 – Resumen.	14
A.13.Sprint 13 – Resumen.	15
A.14.Sprint 14 – Resumen.	16
A.15.Sprint 15 – Resumen.	17
A.16.Costes de personal.	20
A.17.Costes de hardware.	20
A.18.Costes de software.	21
A.19.Costes varios.	21
A.20.Costes totales del proyecto.	21
A.21.Modelo de ingresos previsto	22
A.22.Estimación de recuperación de la inversión.	22
A.23.Dependencias del proyecto y licencias.	23
A.24.Modelos y pesos utilizados.	24
A.25.Compatibilidad de licencias.	25
A.26.Resumen de la licencia AGPL-3.0.	25
A.27.Resumen de la licencia CC-BY-4.0.	26
A.28.Resumen de licencias del proyecto.	27

B.1. CU-1 Registrarse.	38
B.2. CU-2 Iniciar sesión.	39
B.3. CU-3 Gestionar perfil.	39
B.4. CU-4 Solicitar baja de cuenta.	39
B.5. CU-5 Consultar tienda de servicios.	40
B.6. CU-6 Suscribirse a un plan.	40
B.7. CU-7 Alquilar un modelo de IA.	41
B.8. CU-8 Consultar compras y servicios activos.	41
B.9. CU-9 Consultar guía de compra.	42
B.10.CU-10 Consultar pipelines disponibles.	42
B.11.CU-11 Cargar y modificar configuración del pipeline.	43
B.12.CU-12 Ejecutar pipeline sobre una imagen.	43
B.13.CU-13 Visualizar y descargar resultados.	44
B.14.CU-14 Consultar historial de ejecuciones.	44
B.15.CU-15 Acceder al panel de administración.	45
B.16.CU-16 Gestionar usuarios desde administración.	45
B.17.CU-17 Gestionar pipelines desde administración.	46
B.18.CU-18 Consultar ejecuciones globales.	46
B.19.CU-19 Ejecutar tareas de mantenimiento.	46
C.1. Atributos de la tabla USUARIO.	49
C.2. Atributos de la tabla ROL.	49
C.3. Atributos de la tabla USUARIO_ROL.	50
C.4. Atributos de la tabla TIPO_PLAN.	50
C.5. Atributos de la tabla SUSCRIPCION_PLAN.	50
C.6. Atributos de la tabla IA_MODELO.	51
C.7. Atributos de la tabla IA_MODO.	51
C.8. Atributos de la tabla ALQUILA.	51
C.9. Atributos de la tabla PIPELINE.	52
C.10.Atributos de la tabla EJECUCION.	52
C.11.Atributos de la tabla TEMPORAL_ARCHIVO.	52
C.12.Atributos de la tabla PIPELINE_ETAPA.	53
C.13.Interfaces principales de la aplicación.	74
D.1. Directorios principales del proyecto	84
D.2. Launchers existentes	88
D.3. Reglas de sincronización	93
D.4. Resumen de extensión del sistema	94
D.5. Requisitos de ejecución	94
D.6. Problemas frecuentes en desarrollo	103
D.7. Configuración del análisis con SonarCloud	106

<i>Índice de tablas</i>	VII
-------------------------	-----

D.8. Exclusiones del análisis estático	107
--	-----

E.1. Problemas frecuentes de acceso	111
---	-----

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este apéndice recoge la planificación temporal y el estudio de viabilidad del proyecto. Aunque se ha tomado Scrum como referencia para organizar el trabajo de forma iterativa, su aplicación se ha adaptado a las características de un Trabajo Fin de Grado desarrollado por una sola persona. Por tanto, no se ha aplicado como una metodología ágil completa de equipo, sino como una forma de estructurar tareas, revisar avances y planificar incrementos sucesivos.

La planificación se ha dividido en sprints de corta duración, cada uno con un objetivo, un criterio de aceptación y un conjunto de tareas estimadas. Para el seguimiento del trabajo se han empleado puntos de historia, usando como equivalencia práctica que 8 puntos representan una jornada completa de trabajo. Esta estimación ha permitido comparar el esfuerzo previsto con el avance real y ajustar los siguientes sprints conforme evolucionaba el proyecto.

A.2. Planificación temporal

A continuación se describen los sprints realizados durante el desarrollo. En cada uno se incluye una tabla resumen de tareas, la evidencia gráfica del trabajo completado y una breve explicación del avance conseguido.

Visión general del Sprint 1

- **Fechas:** 27/10/2025 – 06/11/2025.

- **Objetivo del sprint:** Investigación y pruebas iniciales de modelos de IA para procesamiento de imágenes.

- **Criterio de aceptación global:** disponer de dos modelos instalados y probados con evidencias, junto con una comparativa preliminar documentada.

Tal y como se muestra en la Tabla A.1 y en la Figura A.1, este sprint se centró en la preparación inicial del proyecto, el aprendizaje básico de $\text{\LaTeX}$ como herramienta de edición de documentos técnicos y académicos, y la evaluación inicial de YOLO y MediaPipe como alternativas para visión por computador.

Tarea	Pts
Organización inicial del proyecto	8
Aprendizaje básico de $\text{\LaTeX}$	8
Estructura base del documento TFG	8
Investigar modelos IA de imágenes	8
Instalar y probar 1ª IA	8
Instalar y probar 2ª IA	8
Comparativa entre IAs seleccionadas	8
Cierre del sprint	8

Tabla A.1: Sprint 1 – Resumen.



Figura A.1: Informe de trabajo completado del Sprint 1.

Durante este sprint se configuró el repositorio y la estructura inicial de documentación, código, experimentos y resultados. Además, se probaron YOLO y MediaPipe sobre imágenes propias para valorar su facilidad de integración, rendimiento y adecuación al sistema planteado. Esta fase permitió validar la viabilidad técnica inicial de emplear modelos de visión por computador dentro del proyecto.

Visión general del Sprint 2

- **Fechas:** 09/11/2025 – 20/11/2025.
- **Objetivo del sprint:** Construir un servidor funcional en Flask capaz de recibir imágenes, ejecutar detecciones simuladas y ofrecer una interfaz mínima de usuario.
- **Criterio de aceptación global:** disponer de un servidor operativo, endpoints funcionales, selección de modelos simulada y una interfaz básica para validar el flujo de subida y procesado.

Como se recoge en la Tabla A.2 y en la Figura A.2, este sprint introdujo la primera versión funcional del servidor y de la interfaz web.

Tarea	Pts
Definir alcance del sprint y crear historias	8
Elegir framework servidor Flask	8
Elección de librería para simulación de detecciones	6
Configurar entorno de desarrollo y estructura base	6
Probar funcionamiento del servidor	8
Desarrollar endpoint para subir imágenes	10
Desarrollar endpoint para detección simulada	12
Crear interfaz mínima de aplicación	8
Validación final y documentación del sprint	6

Tabla A.2: Sprint 2 – Resumen.

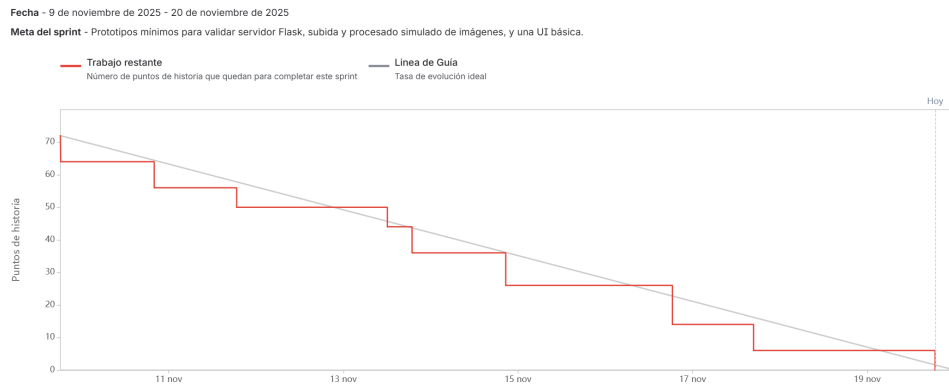


Figura A.2: Informe de trabajo completado del Sprint 2.

Se seleccionó Flask por su ligereza y facilidad para construir endpoints REST. Antes de integrar los modelos reales, se implementó una detección simulada para validar el flujo completo de subida de imagen, procesamiento y visualización de resultados. Esta decisión permitió separar la validación de la arquitectura web de la complejidad propia de los modelos de IA.

Visión general del Sprint 3

- **Fechas:** 14/02/2026 – 19/02/2026.
- **Objetivo del sprint:** Definir un borrador de requisitos funcionales, requisitos no funcionales y casos de uso.

- **Criterio de aceptación global:** disponer de una primera especificación suficiente para cubrir el funcionamiento previsto de la aplicación.

Como se observa en la Tabla A.3 y en la Figura A.3, este sprint se dedicó al análisis inicial del sistema.

Tarea	Pts
Definición del alcance del sprint	2
Plantear alcance de requisitos y casos de uso	6
Definir requisitos funcionales	8
Definir requisitos no funcionales	8
Definir casos de uso	16

Tabla A.3: Sprint 3 – Resumen.

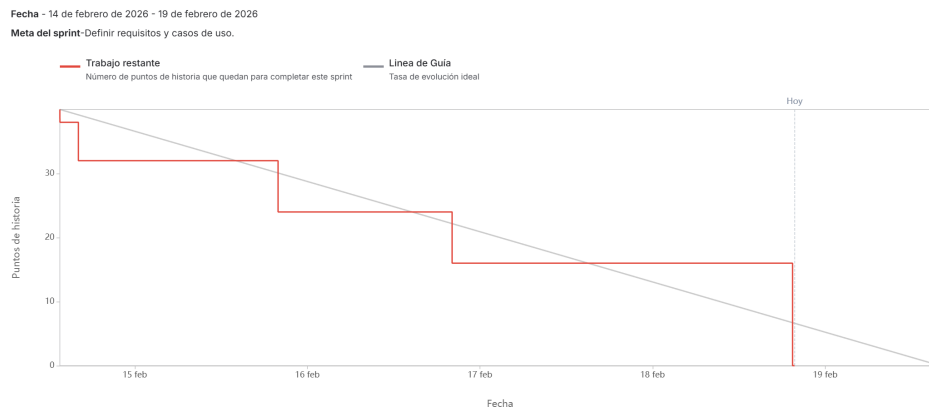


Figura A.3: Informe de trabajo completado del Sprint 3.

El trabajo realizado permitió concretar las funcionalidades esperadas del sistema, diferenciando requisitos funcionales, requisitos no funcionales y casos de uso. Esta especificación sirvió como base para las siguientes decisiones de diseño, especialmente las relacionadas con usuarios, permisos, pipelines y ejecución de modelos.

Visión general del Sprint 4

- **Fechas:** 19/02/2026 – 25/02/2026.

- **Objetivo del sprint:** Completar la documentación del MVP y diseñar el modelo E-R de la aplicación.
- **Criterio de aceptación global:** disponer de la especificación del MVP y de un modelo E-R coherente con las principales reglas de negocio.

Tal y como se muestra en la Tabla A.4 y en la Figura A.4, el sprint se centró en transformar el análisis funcional en una primera propuesta de modelo de datos.

Tarea	Pts
Corregir y subir documento “RF_RNF_CU_aplicación_completa”	2
Documentar los RF/RNF/CU para el producto mínimo viable	6
Modelo E-R: identificación de entidades y alcance	8
Modelo E-R: atributos y claves	8
Modelo E-R: relaciones y cardinalidades	8
Modelo E-R: normalización, integridad y restricciones de negocio	8

Tabla A.4: Sprint 4 – Resumen.

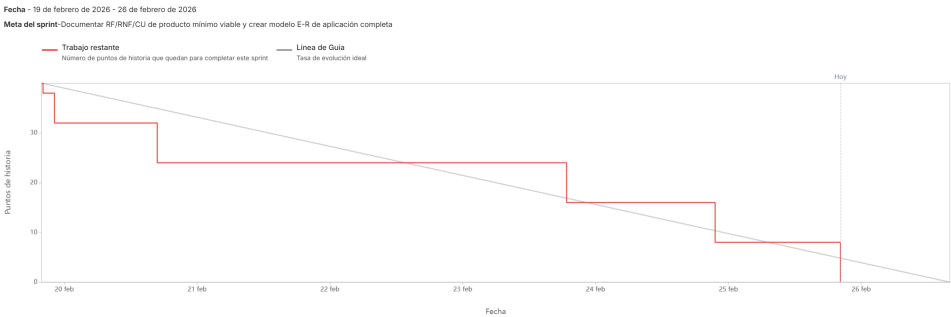


Figura A.4: Informe de trabajo completado del Sprint 4.

Se revisó la especificación de requisitos y se seleccionaron los elementos necesarios para el producto mínimo viable. A partir de ello se diseñaron entidades, atributos, claves, cardinalidades y restricciones, prestando especial atención a la coherencia entre usuarios, planes, modelos de IA, modos de ejecución y pipelines.

Visión general del Sprint 5

- **Fechas:** 03/03/2026 – 05/03/2026.
- **Objetivo del sprint:** Revisar y corregir los diagramas E-R y relacional.
- **Criterio de aceptación global:** incorporar las correcciones detectadas en la revisión y comprobar la adecuación del modelo a los casos de uso.

Como se refleja en la Tabla A.5 y en la Figura A.5, este sprint se dedicó a mejorar la consistencia del modelo de datos.

Tarea	Pts
Modificar modelo relacional	8
Actualizar modelo E-R	8
Comprobar la validez de ambos modelos frente a los casos de uso	8

Tabla A.5: Sprint 5 – Resumen.

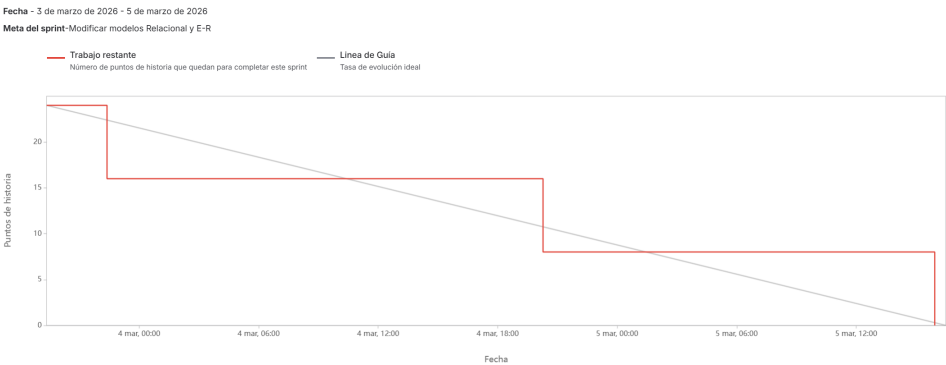


Figura A.5: Informe de trabajo completado del Sprint 5.

Se ajustaron los diagramas para mejorar la normalización, la trazabilidad con los casos de uso y la capacidad de crecimiento de la aplicación. La revisión confirmó que el modelo debía soportar tanto la funcionalidad básica como futuras ampliaciones relacionadas con planes, alquileres y pipelines.

Visión general del Sprint 6

- **Fechas:** 07/03/2026 – 12/03/2026.
- **Objetivo del sprint:** Definir y ajustar el modelo de configuración y ejecución de pipelines.
- **Criterio de aceptación global:** reflejar en el modelo de datos las decisiones sobre configuración, creación de pipelines y registro de ejecuciones.

La Tabla A.6 y la Figura A.6 muestran el trabajo realizado para concretar una de las partes más importantes del sistema: los pipelines de procesamiento.

Tarea	Pts
Modificación del modelo E-R	4
Analizar si la configuración será modificable por el usuario	6
Definir si los pipelines los crea el administrador o los usuarios	2
Analizar si las ejecuciones pueden repetirse	2
Actualizar el modelo según las decisiones tomadas	10
Revisar coherencia entre requisitos, casos de uso y modelo	8

Tabla A.6: Sprint 6 – Resumen.

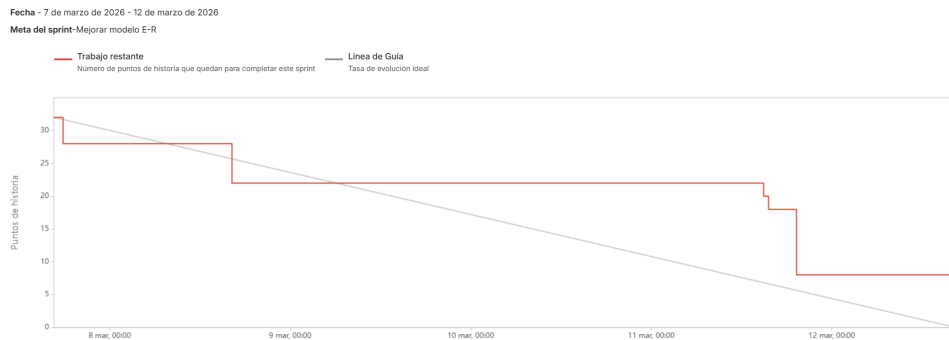


Figura A.6: Informe de trabajo completado del Sprint 6.

Durante este sprint se tomaron decisiones clave sobre la configuración de los pipelines: los usuarios podrían partir de configuraciones predeterminadas y modificarlas en la ejecución, mientras que la administración sería

responsable de crear y mantener los pipelines disponibles. Estas decisiones se trasladaron al modelo de datos para mantener la coherencia entre configuración, etapas y ejecuciones.

Visión general del Sprint 7

- **Fechas:** 16/03/2026 – 19/03/2026.
- **Objetivo del sprint:** Preparar el entorno de desarrollo definitivo en Debian e iniciar la implementación real de la base de datos.
- **Criterio de aceptación global:** disponer de un entorno funcional en Debian con MariaDB operativo y comenzar la implementación física de la base de datos.

La Tabla A.7 y la Figura A.7 resumen la transición desde el diseño lógico hacia un entorno de desarrollo más cercano al despliegue real.

Tarea	Pts
Crear Debian en dual boot con Windows	8
Preparar Debian con configuración mínima	4
Instalar MariaDB	4
Realizar pruebas con MariaDB	8
Empezar base de datos del TFG	8

Tabla A.7: Sprint 7 – Resumen.

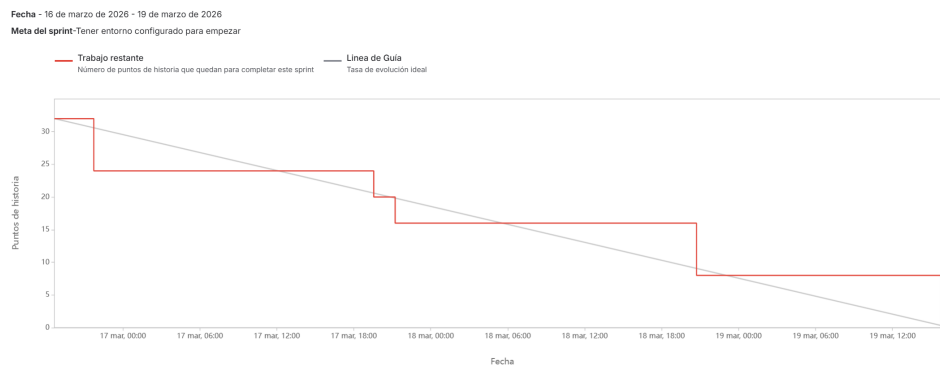


Figura A.7: Informe de trabajo completado del Sprint 7.

Se instaló Debian en arranque dual con Windows y se configuró un entorno de trabajo más estable y alineado con servidores Linux. También se instaló MariaDB, se realizaron pruebas de conexión y se comenzó a trasladar el diseño relacional a tablas reales.

Visión general del Sprint 8

- **Fechas:** 20/03/2026 – 26/03/2026.
- **Objetivo del sprint:** Ajustar el modelo de datos e implementar completamente la base de datos del sistema.
- **Criterio de aceptación global:** disponer de una base de datos implementada en MariaDB y coherente con el modelo relacional actualizado.

Como se observa en la Tabla A.8 y en la Figura A.8, este sprint consolidó la parte de persistencia del proyecto.

Tarea	Pts
Actualizar modelo relacional	4
Implementar base de datos	12
Documentar base de datos	10
Documentar aspectos relevantes de la memoria	10
Añadir referencias a imágenes y tablas en anexos	4

Tabla A.8: Sprint 8 – Resumen.

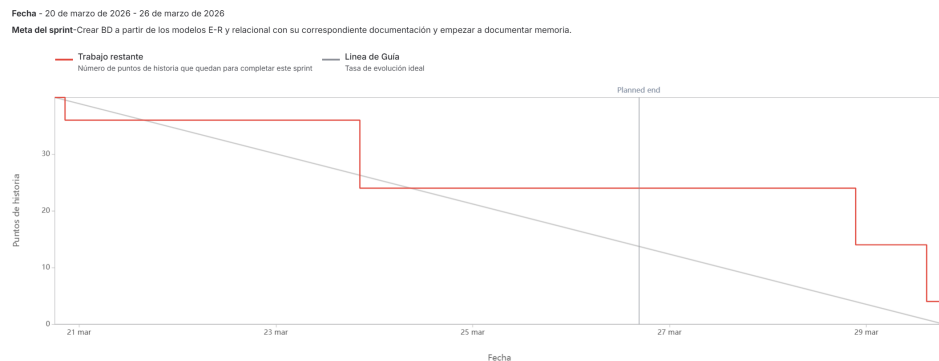


Figura A.8: Informe de trabajo completado del Sprint 8.

Se completó la implementación de la base de datos y se actualizaron los diagramas y la documentación asociada. Además, se avanzó en la memoria y se incorporaron referencias cruzadas a figuras y tablas para mejorar la trazabilidad del documento.

Visión general del Sprint 9

- **Fechas:** 06/04/2026 – 09/04/2026.
- **Objetivo del sprint:** Definir la estructura base del sistema bajo una arquitectura MVC adaptada e integrar los primeros componentes funcionales.
- **Criterio de aceptación global:** disponer de una arquitectura funcional con modelos de IA, autenticación básica e interfaz mínima.

La Tabla A.9 y la Figura A.9 recogen el inicio de la arquitectura definitiva de la aplicación.

Tarea	Pts
Crear primera estructura modelo-vista-controlador	12
Integrar los modelos de IA en la lógica del sistema	8
Poner en funcionamiento usuarios en el modelo	6
Crear interfaz mínima	6

Tabla A.9: Sprint 9 – Resumen.

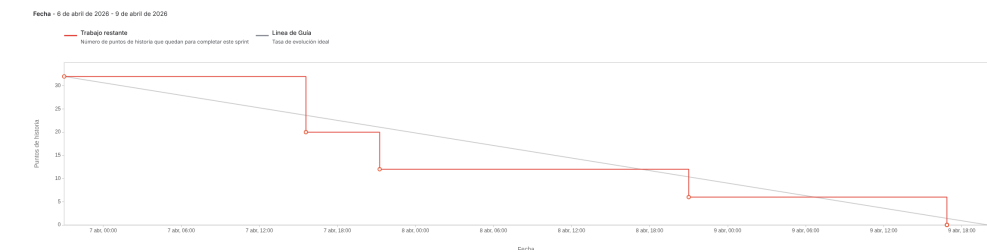


Figura A.9: Informe de trabajo completado del Sprint 9.

Se definió una estructura basada en controladores, modelos, servicios y vistas, adaptando el patrón MVC a Flask. También se integraron los primeros componentes funcionales relacionados con modelos de IA, usuarios e interfaz, preparando la aplicación para las siguientes funcionalidades.

Visión general del Sprint 10

- **Fechas:** 13/04/2026 – 16/04/2026.
- **Objetivo del sprint:** Mejorar la documentación y avanzar en el sistema de autenticación desde la interfaz.
- **Criterio de aceptación global:** disponer de una documentación más refinada y de un inicio de sesión funcional con diferenciación básica de roles.

La Tabla A.10 y la Figura A.10 muestran un sprint centrado en documentación y autenticación.

Tarea	Pts
Corregir problemas de documentación	3
Mejorar documentación de técnicas y herramientas	5
Reducir la extensión de aspectos relevantes	8
Empezar con inicio de sesión de usuarios	16

Tabla A.10: Sprint 10 – Resumen.



Figura A.10: Informe de trabajo completado del Sprint 10.

Se corrigieron problemas de formato y redacción en la documentación, especialmente en los apartados de técnicas, herramientas y aspectos relevantes. En paralelo, se inició la integración del inicio de sesión desde la interfaz gráfica, separando las vistas destinadas a usuarios y administradores.

Visión general del Sprint 11

- **Fechas:** 24/04/2026 – 29/04/2026.
- **Objetivo del sprint:** Mejorar la documentación de sprints anteriores e incorporar persistencia de usuarios, ejecuciones y archivos.
- **Criterio de aceptación global:** disponer de una base funcional capaz de registrar usuarios, guardar ejecuciones y gestionar archivos temporales.

Como se recoge en la Tabla A.11 y en la Figura A.11, este sprint amplió la aplicación con persistencia real y gestión de resultados.

Tarea	Pts
Mejorar documentación de sprints anteriores	5
Actualizar documentación de base de datos	5
Hacer registro de usuarios	8
Crear script semilla para base de datos	6
Implementar guardado de ejecuciones en base de datos	8
Implementar gestión de archivos y limpieza	8
Refactorización del orquestador para soporte multi-imagen	8

Tabla A.11: Sprint 11 – Resumen.

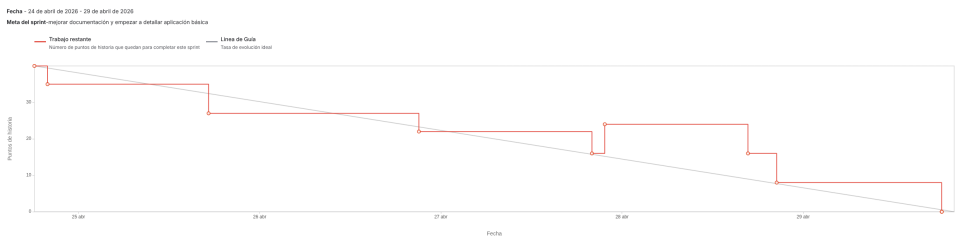


Figura A.11: Informe de trabajo completado del Sprint 11.

Se implementó el registro de usuarios, el script de datos iniciales, el almacenamiento de ejecuciones y la gestión de archivos temporales. Durante este trabajo se detectó que algunos pipelines podían generar varias imágenes de salida, especialmente en recortes o análisis por sujetos. Por ello se añadió la refactorización del orquestador para soportar resultados multi-imagen desde una fase temprana del diseño.

Visión general del Sprint 12

- **Fechas:** 04/05/2026 – 07/05/2026.
- **Objetivo del sprint:** Incorporar funcionalidades de usuario, gestión comercial e historial de ejecuciones.
- **Criterio de aceptación global:** permitir al usuario modificar sus datos, alquilar modelos, cambiar de plan y consultar ejecuciones realizadas.

La Tabla A.12 y la Figura A.12 muestran el avance hacia una aplicación más completa desde el punto de vista funcional.

Tarea	Pts
Crear opción para modificar datos de usuario	8
Crear opción para alquilar modelos	8
Crear opción para ver historial de ejecuciones	8
Crear opción para cambiar de plan	8

Tabla A.12: Sprint 12 – Resumen.

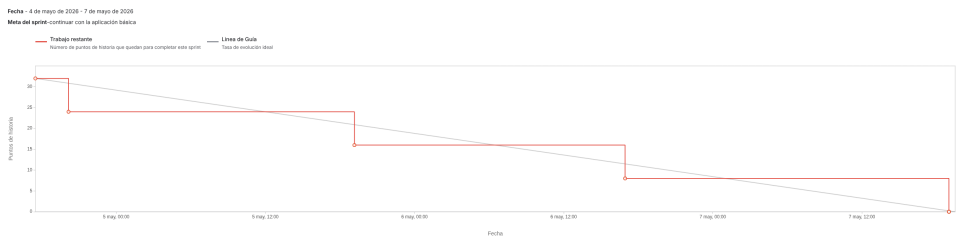


Figura A.12: Informe de trabajo completado del Sprint 12.

Se incorporaron funcionalidades vinculadas al perfil del usuario, contratación simulada de modelos, cambio de plan e historial de ejecuciones. Estas tareas permitieron conectar la lógica de negocio definida en los requisitos con una experiencia de usuario más completa.

Visión general del Sprint 13

- **Fechas:** 10/05/2026 – 14/05/2026.

- **Objetivo del sprint:** Corregir detalles de la aplicación básica, mejorar la estructura del código y comenzar la creación de pruebas.
- **Criterio de aceptación global:** disponer de una aplicación más estable, con mejor organización interna y una primera batería de tests.

Como se muestra en la Tabla A.13 y en la Figura A.13, este sprint se orientó a estabilizar el sistema.

Tarea	Pts
Mejoras de comentarios y estructura de código	8
Corregir detalles de la aplicación básica	16
Creación de tests	16

Tabla A.13: Sprint 13 – Resumen.

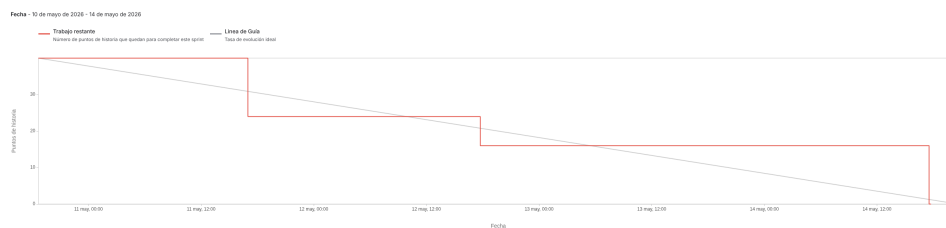


Figura A.13: Informe de trabajo completado del Sprint 13.

Se revisó la estructura del código, se corrigieron detalles detectados durante el uso de la aplicación y se comenzó la creación de pruebas automatizadas. Estas pruebas permitieron validar partes relevantes del backend y detectar fallos derivados de los cambios introducidos en funcionalidades anteriores.

Visión general del Sprint 14

- **Fechas:** 16/05/2026 – 21/05/2026.
- **Objetivo del sprint:** Incorporar la gestión administrativa de pipelines, ampliar las pruebas asociadas y realizar una revisión general de memoria y anexos.

- **Criterio de aceptación global:** permitir que un administrador pueda crear, editar y eliminar pipelines desde la aplicación, validar la funcionalidad mediante pruebas y dejar revisada la documentación principal del proyecto.

Como se muestra en la Tabla A.14 y en la Figura A.14, este sprint combinó trabajo de desarrollo, pruebas y revisión documental.

Tarea	Pts
Crear funcionalidad para que el administrador pueda crear, editar y eliminar pipelines	16
Revisar y modificar memoria a nivel general	12
Crear tests y corregir código asociado a la nueva funcionalidad administrativa	8
Revisar y modificar anexos a nivel general	12

Tabla A.14: Sprint 14 – Resumen.

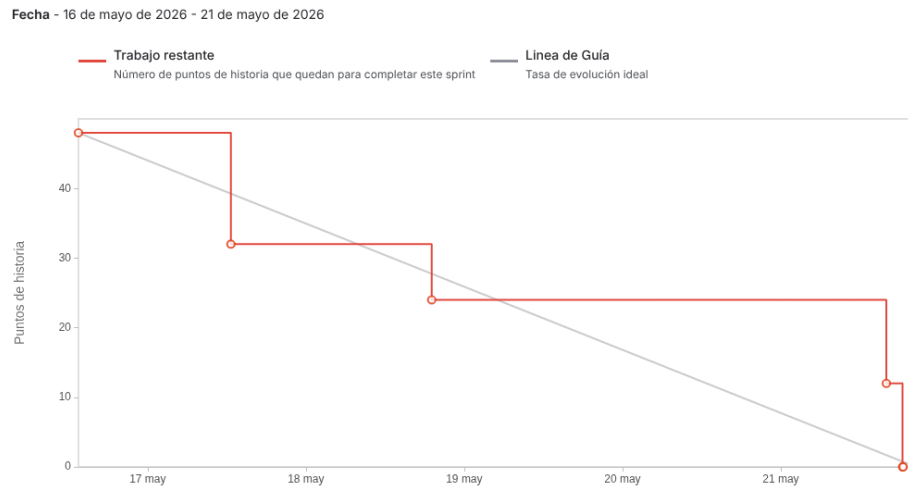


Figura A.14: Informe de trabajo completado del Sprint 14.

Durante este sprint se desarrolló una de las funcionalidades más relevantes para la escalabilidad del sistema: la administración de pipelines desde la propia aplicación. Esta mejora permite que un usuario con rol de administrador pueda definir, modificar o eliminar flujos de procesamiento

sin necesidad de cambiar el código fuente. Además, se añadieron pruebas para validar esta lógica y se corrigieron problemas detectados durante su integración. En paralelo, se revisaron la memoria y los anexos para alinear la documentación con el estado real de la aplicación.

Visión general del Sprint 15

- **Fechas:** 23/05/2026 – 28/05/2026.
- **Objetivo del sprint:** Continuar con la documentación de anexos y aplicar un análisis de calidad mediante SonarQube.
- **Criterio de aceptación global:** completar los principales apartados pendientes de anexos, incorporar evidencias de calidad del código y corregir los problemas detectados durante el análisis estático.

La Tabla A.15 y la Figura A.15 recogen el trabajo realizado en la fase final de documentación y revisión técnica del proyecto.

Tarea	Pts
Documentar anexos: estudio de viabilidad	16
Documentar anexos: diseño procedimental	8
Documentar anexos: manual del programador	10
Documentar anexos: diseño de interfaces	6
Documentar anexos: manual de usuario	8
Documentar anexos: diseño arquitectónico	8
Aplicar análisis mediante SonarQube y corregir los problemas detectados	8

Tabla A.15: Sprint 15 – Resumen.



Figura A.15: Informe de trabajo completado del Sprint 15.

Este sprint se centró principalmente en completar y revisar los anexos del proyecto: estudio de viabilidad, diseño arquitectónico, diseño procedimental, diseño de interfaces, manual del programador y manual de usuario. Durante esta revisión se detectó también la necesidad de reforzar la calidad del código antes de cerrar la documentación técnica. Por ello, se añadió una tarea específica para ejecutar SonarQube, analizar los problemas señalados y aplicar las correcciones necesarias. Esta actividad permitió dejar constancia documental del proceso de revisión estática y mejorar la robustez del código antes de la entrega.

A.3. Estudio de viabilidad

El estudio de viabilidad analiza el proyecto desde dos perspectivas: la económica, centrada en estimar el coste aproximado de desarrollo y la posible recuperación de la inversión; y la legal, orientada a revisar las licencias de software, modelos utilizados, documentación y tratamiento de datos personales.

Viabilidad económica

Aunque el proyecto se ha desarrollado en el contexto de un Trabajo Fin de Grado, se ha estimado el coste que tendría realizar una aplicación de

estas características en un entorno profesional. Para ello se han considerado costes de personal, hardware amortizado, software y gastos generales.

Costes

Costes de personal Para estimar el coste de personal se ha tomado como referencia el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública. Se ha considerado el perfil de programación, grupo D, nivel 1, por ajustarse a las tareas realizadas: desarrollo backend, diseño de base de datos, integración de modelos de IA, autenticación, control de acceso, pruebas y documentación técnica [10].

El salario anual bruto de referencia utilizado es de 20.436,01 €. Considerando una jornada anual de 1.800 horas, el coste bruto por hora es:

$$\frac{20,436,01 \text{ €}}{1,800 \text{ h}} = 11,35 \text{ €/h}$$

Para una dedicación aproximada de 300 horas, el salario bruto proporcional sería:

$$11,35 \text{ €/h} \times 300 \text{ h} = 3,406,00 \text{ €}$$

A este importe se añaden las cotizaciones empresariales. Se han considerado contingencias comunes, contingencias profesionales, desempleo, Fondo de Garantía Salarial, formación profesional y Mecanismo de Equidad Inter-generacional, tomando como referencia la información oficial de la Seguridad Social y la tarifa de primas aplicable al CNAE 62 [9, 12]. El desglose se recoge en la Tabla A.16.

Costes de hardware Para el desarrollo se ha utilizado un ordenador portátil, dos pantallas externas, teclado y ratón. Al tratarse de recursos reutilizables, se ha calculado su amortización proporcional considerando una vida útil de 4 años y un uso asociado al proyecto de 9 meses:

$$\text{Coste amortizado} = \text{Coste total} \times \frac{9 \text{ meses}}{48 \text{ meses}}$$

El desglose se muestra en la Tabla A.17.

Concepto	Porcentaje	Coste
Salario bruto proporcional a 300 horas	–	3.406,00 €
Contingencias comunes	23,60 %	803,82 €
Contingencias profesionales	1,50 %	51,09 €
Desempleo	5,50 %	187,33 €
Fondo de Garantía Salarial	0,20 %	6,81 €
Formación profesional	0,60 %	20,44 €
Mecanismo de Equidad Intergeneracional	0,75 %	25,55 €
Total cotizaciones empresariales	32,15 %	1.095,03 €
Coste total de personal		4.501,03 €

Tabla A.16: Costes de personal.

Concepto	Coste total	Coste amortizado
Ordenador portátil	1.000,00 €	187,50 €
Pantalla externa principal	300,00 €	56,25 €
Pantalla externa secundaria	200,00 €	37,50 €
Teclado	60,00 €	11,25 €
Ratón	70,00 €	13,13 €
Total	1.630,00 €	305,63 €

Tabla A.17: Costes de hardware.

Costes de software La mayor parte de herramientas empleadas son gratuitas o de código abierto, como Python, Flask, MariaDB, Visual Studio Code, GitHub, Draw.io, DBeaver, L^AT_EX y las principales librerías del proyecto. No obstante, se han utilizado suscripciones de apoyo para depuración, revisión técnica y documentación. La estimación se muestra en la Tabla A.18.

Costes varios También se han considerado internet y electricidad. Para internet se ha estimado una tarifa mensual de 30 € durante 9 meses. Para electricidad, se ha usado un consumo medio de 180 W durante 300 horas y un coste de 0,20 €/kWh:

$$0,18 \text{ kW} \times 300 \text{ h} \times 0,20 \text{ €/kWh} = 10,80 \text{ €}$$

El resumen aparece en la Tabla A.19.

Concepto	Coste mensual	Coste total
ChatGPT Plus	21,99 €	197,91 €
Google AI Pro / Gemini	21,99 €	197,91 €
Herramientas libres y gratuitas	0,00 €	0,00 €
Total		395,82 €

Tabla A.18: Costes de software.

Concepto	Coste
Internet	270,00 €
Electricidad	10,80 €
Total	280,80 €

Tabla A.19: Costes varios.

Concepto	Coste
Personal	4.501,03 €
Hardware amortizado	305,63 €
Software	395,82 €
Varios	280,80 €
Total	5.483,28 €

Tabla A.20: Costes totales del proyecto.

No se ha incluido alojamiento en producción, ya que la entrega se plantea como una aplicación reproducible en entorno local o de demostración. En una explotación real habría que añadir servidor, dominio, almacenamiento, copias de seguridad y mantenimiento.

Costes totales

El coste total estimado se obtiene sumando personal, hardware amortizado, software y gastos varios. La Tabla A.20 resume el resultado.

Por tanto, el coste económico aproximado del proyecto asciende a 5.483,28 €. Esta cifra representa una estimación profesional del esfuerzo invertido, no un desembolso real completo durante el TFG.

Concepto	Precio
Alquiler mensual de un modelo IA	9,99 €/mes
Plan Pro	19,99 €/mes

Tabla A.21: Modelo de ingresos previsto

Vía de ingresos	Precio mensual	Mensualidades necesarias
Plan Pro	19,99 €	275
Alquiler individual de modelo IA	9,99 €	549

Tabla A.22: Estimación de recuperación de la inversión.

Modelo de ingresos y recuperación de la inversión

La aplicación se plantea con un modelo de acceso por servicios: alquiler mensual de modelos de IA y plan Pro. La Tabla A.21 recoge los precios previstos.

A partir del coste total estimado, la recuperación de la inversión sería:

$$\frac{5,483,28 \text{ €}}{19,99 \text{ €/mes}} = 274,30 \text{ mensualidades}$$

$$\frac{5,483,28 \text{ €}}{9,99 \text{ €/mes}} = 548,88 \text{ mensualidades}$$

La Tabla A.22 resume las mensualidades necesarias en cada vía de ingresos.

La viabilidad económica dependería de alcanzar una base suficiente de usuarios recurrentes. El diseño basado en modelos, modos y pipelines favorece esta posibilidad, ya que permite ampliar el catálogo sin rehacer la aplicación desde cero.

Viabilidad legal

La viabilidad legal se ha analizado teniendo en cuenta las dependencias de software, los modelos preentrenados, la licencia del código fuente, la

documentación y el tratamiento de datos personales. La mayoría de dependencias emplean licencias permisivas, aunque la presencia de **ultralytics** condiciona la licencia final del código.

Software

La Tabla A.23 recoge las dependencias más relevantes del proyecto, su licencia y la fuente utilizada para consultarla.

Dependencia	Versión	Uso en el proyecto	Licencia / fuente
Flask	3.0.3	Framework web principal.	BSD-3-Clause [22]
Flask-SQLAlchemy	3.1.1	Integración de SQLAlchemy con Flask.	BSD License [23]
SQLAlchemy	2.0.31	ORM y ejecución de sentencias auxiliares.	MIT [2]
Flask-JWT-Extended	4.6.0	Autenticación mediante JWT.	MIT [5]
python-dotenv	1.0.1	Carga de variables de entorno.	BSD-3-Clause [20]
Werkzeug	3.0.3	Utilidades WSGI y funciones de seguridad.	BSD-3-Clause [24]
PyMySQL	1.1.1	Conexión con MariaDB/MySQL.	MIT [8]
cryptography	42.0.8	Soporte criptográfico.	Apache-2.0 / BSD-3-Clause [1]
numpy	1.26.4	Cálculo numérico para procesamiento de imágenes.	BSD-3-Clause [13]
opencv-python-headless	4.10.0.84	Procesamiento de imágenes sin dependencias gráficas.	MIT / Apache-2.0 / LGPL-2.1 [21]
Pillow	10.4.0	Apertura y manipulación de imágenes.	HPND [7]
mediapipe	0.10.14	Detección de manos y estimación de pose.	Apache-2.0 [3]
torch	2.3.1	Soporte de aprendizaje profundo.	BSD-3-Clause [26]
torchvision	0.18.1	Tareas de visión por computador.	BSD-3-Clause [27]
ultralytics	8.2.50	Integración de YOLOv8.	AGPL-3.0 / empresarial [28, 29]
gunicorn	22.0.0	Servidor WSGI para despliegue.	MIT [6]
pytest	8.2.2	Ejecución de pruebas automatizadas.	MIT [25]

Tabla A.23: Dependencias del proyecto y licencias.

Como puede observarse, la mayoría de dependencias utilizan licencias permisivas como MIT, BSD o Apache-2.0. Sin embargo, **ultralytics** introduce una restricción importante, al distribuirse bajo AGPL-3.0 salvo uso de licencia empresarial. Por ello, mientras el proyecto mantenga esta dependencia, la opción más conservadora es licenciar el código fuente bajo AGPL-3.0.

Modelos y pesos utilizados

Además del software, el proyecto utiliza modelos preentrenados. Estos recursos deben analizarse por separado, ya que forman parte del funciona-

Recurso	Uso en el proyecto	Licencia / condiciones
yolov8n.pt	Pesos de YOLOv8 utilizados mediante <code>ultralytics</code> .	AGPL-3.0 / licencia empresarial
hand_landmarker.task	Modelo de MediaPipe Tasks para landmarks de manos.	Apache-2.0
pose_landmarker_full.task	Modelo de MediaPipe Tasks para estimación de pose.	Apache-2.0
Configuraciones JSON propias	Ficheros <code>mp_manos.json</code> , <code>mp_pose.json</code> , <code>yolo_deteccion.json</code> y <code>yolo_recortes.json</code> .	Propias del proyecto

Tabla A.24: Modelos y pesos utilizados.

miento de la aplicación aunque no sean código fuente propio. La Tabla A.24 resume los principales modelos y configuraciones.

Los modelos de MediaPipe no condicionan la licencia final del proyecto, al estar asociados al ecosistema MediaPipe y a condiciones compatibles con Apache-2.0 [14, 15, 19]. En cambio, YOLOv8 utilizado mediante `ultralytics` debe considerarse junto con las obligaciones de AGPL-3.0 [28, 29].

Compatibilidad de licencias

Las licencias MIT, BSD, HPND y Apache-2.0 son permisivas y compatibles con un proyecto distribuido bajo una licencia copyleft fuerte. La licencia que condiciona la decisión final es AGPL-3.0, por la dependencia con `ultralytics`. La Tabla A.25 resume esta compatibilidad.

Si en el futuro se quisiera publicar el proyecto bajo una licencia más permisiva o cerrada, habría que sustituir `ultralytics` por una alternativa compatible o adquirir una licencia empresarial.

Licencia del código fuente

Teniendo en cuenta las dependencias utilizadas, la licencia más adecuada para el código fuente es GNU Affero General Public License v3.0 (AGPL-3.0). Esta licencia permite uso, modificación y distribución del software, pero exige mantener la misma licencia en obras derivadas. Además, está pensada para

Licencia	Uso en el proyecto	Compatibilidad
MIT	Presente en dependencias como <code>SQLAlchemy</code> , <code>PyMySQL</code> , <code>gunicorn</code> y <code>pytest</code> .	Compatible
BSD / BSD-3-Clause	Presente en <code>Flask</code> , <code>Werkzeug</code> , <code>NumPy</code> , <code>PyTorch</code> y otras dependencias.	Compatible
Apache-2.0	Presente en <code>MediaPipe</code> y en componentes del ecosistema <code>OpenCV</code> .	Compatible
HPND	Presente en <code>Pillow</code> .	Compatible
LGPL-2.1	Presente en componentes binarios asociados a <code>FFmpeg</code> dentro de <code>OpenCV</code> .	Compatible con uso como librería
AGPL-3.0	Presente en <code>Ultralytics/YOLOv8</code> .	Licencia dominante

Tabla A.25: Compatibilidad de licencias.

Derechos	Condiciones	Limitaciones
Uso comercial	Mantener avisos de copyright y licencia	Sin garantía
Distribución	Liberar el código fuente bajo la misma licencia	Limitación de responsabilidad
Modificación	Indicar los cambios realizados	No concede uso de marcas
Uso como servicio web	Hacer disponible el código fuente de la versión ofrecida por red	Copyleft fuerte
Uso de patentes	Mantener las condiciones de la licencia	Obligaciones sobre obras derivadas

Tabla A.26: Resumen de la licencia AGPL-3.0.

aplicaciones ofrecidas a través de red, por lo que encaja con una aplicación web [18, 17].

Esta licencia no impide monetizar el servicio mediante alquileres de modelos o planes de suscripción, pero sí evita una explotación cerrada del código mientras se mantenga la dependencia con `ultralytics`.

Derechos	Condiciones	Limitaciones
Uso comercial	Reconocimiento de autoría	Sin garantía
Distribución	Indicar cambios realizados si los hubiera	No concede derechos sobre marcas
Modificación	Mantener referencia a la licencia	No concede derechos sobre patentes
Uso privado	No aplicar restricciones adicionales incompatibles	Limitación de responsabilidad

Tabla A.27: Resumen de la licencia CC-BY-4.0.

Documentación

Para la memoria, anexos, diagramas y material escrito se propone utilizar Creative Commons Attribution 4.0 International (**CC-BY-4.0**), más adecuada para documentos que AGPL-3.0. Esta licencia permite copiar, distribuir y modificar el material, incluso con fines comerciales, siempre que se reconozca la autoría original [4].

Protección de datos personales

La aplicación gestiona cuentas de usuario, correos electrónicos, nombres de usuario, contraseñas cifradas mediante hash, suscripciones, alquileres de modelos y registros de ejecuciones. Por ello, debe aproximarse a los principios del Reglamento General de Protección de Datos y de la Ley Orgánica 3/2018, especialmente en minimización, confidencialidad y limitación del plazo de conservación [16, 11].

Desde el diseño se han incorporado varias medidas: las contraseñas no se almacenan en texto plano, las imágenes y resultados se gestionan como archivos temporales, el acceso a resultados se realiza mediante tokens de descarga y las ejecuciones quedan asociadas al usuario autenticado. Además, se contempla la baja de cuenta, la revocación de servicios activos y la posterior anonimización de datos personales cuando proceda.

En una explotación real sería necesario completar esta parte con una política de privacidad, condiciones de uso, identificación del responsable del tratamiento, base jurídica, plazos de conservación y procedimiento formal para ejercer derechos.

Recurso	Licencia propuesta
Código fuente de la aplicación	AGPL-3.0
Código fuente de pruebas	AGPL-3.0
Configuraciones propias del proyecto	AGPL-3.0, al formar parte del funcionamiento de la aplicación
Modelos de MediaPipe utilizados localmente	Apache-2.0, según condiciones del ecosistema MediaPipe
Modelo YOLOv8 utilizado mediante Ultralytics	AGPL-3.0 / licencia empresarial
Documentación del proyecto	CC-BY-4.0
Diagramas e imágenes propias de la documentación	CC-BY-4.0
Imágenes subidas por usuarios	Propiedad del usuario que las aporta
Resultados generados por la aplicación	Según condiciones de uso del servicio

Tabla A.28: Resumen de licencias del proyecto.

Resumen de licencias del proyecto

La Tabla [A.28](#) recoge la propuesta final de licenciamiento.

En conclusión, la viabilidad legal del proyecto es positiva siempre que se respeten las condiciones de las dependencias empleadas. La presencia de **ultralytics** obliga a adoptar una postura conservadora y licenciar el código bajo **AGPL-3.0**, salvo que se sustituya dicha dependencia o se obtenga una licencia empresarial. Para la documentación y recursos propios no software se propone **CC-BY-4.0**.

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge la especificación de requisitos y los casos de uso principales del sistema desarrollado. La aplicación consiste en una plataforma web de procesamiento de imágenes mediante modelos de inteligencia artificial, organizados en forma de *pipelines* configurables.

El sistema contempla distintos perfiles de acceso: usuarios no autenticados, usuarios registrados con plan Básico, usuarios con plan Pro y usuarios administradores. Los usuarios registrados pueden consultar los servicios disponibles, contratar planes o modelos individuales, seleccionar un pipeline autorizado, modificar su configuración, subir una imagen, ejecutar el procesamiento, visualizar los resultados y acceder temporalmente a los archivos generados.

Desde el punto de vista administrativo, la aplicación permite consultar usuarios, revisar ejecuciones globales y gestionar los pipelines disponibles sin modificar directamente el código fuente. Esta última funcionalidad resulta especialmente importante para la escalabilidad del sistema, ya que permite definir nuevos flujos de procesamiento combinando modelos y modos ya registrados.

La especificación se divide en tres partes: objetivos generales, catálogo de requisitos funcionales y no funcionales, y descripción detallada de los casos de uso.

B.2. Objetivos generales

Los objetivos generales del sistema son los siguientes:

- Permitir el acceso de usuarios autenticados a una plataforma web de procesamiento de imágenes mediante modelos de inteligencia artificial.
- Diferenciar entre usuarios no autenticados, usuarios registrados y usuarios con rol de administrador.
- Aplicar control de acceso en el servidor para impedir que un usuario ejecute pipelines que requieran modelos no contratados o no disponibles para su plan.
- Ofrecer un flujo completo de uso: registro o inicio de sesión, consulta de servicios, selección de pipeline, configuración de la ejecución, subida de imagen, procesamiento, visualización de resultados y descarga temporal de archivos.
- Permitir la contratación simulada de planes y modelos individuales, diferenciando entre acceso completo mediante plan Pro y acceso concreto mediante alquiler de modelos.
- Registrar las ejecuciones realizadas, manteniendo trazabilidad básica sobre usuario, pipeline ejecutado, estado de la operación, duración, errores y configuración aplicada.
- Gestionar los archivos generados como recursos temporales, evitando su conservación indefinida y controlando su acceso mediante tokens de descarga.
- Facilitar la administración del sistema mediante un panel específico para revisar usuarios, estados, ejecuciones y pipelines.
- Diseñar una base funcional ampliable, de forma que puedan añadirse nuevos modelos, modos o pipelines con cambios reducidos sobre la estructura general de la aplicación.

B.3. Catálogo de requisitos

Requisitos funcionales

RF-1 Gestión de cuentas de usuario

La aplicación debe permitir la creación, acceso, modificación y baja lógica de cuentas de usuario.

- RF-1.1 Registrar un nuevo usuario.
- RF-1.2 Iniciar sesión mediante credenciales válidas.
- RF-1.3 Cerrar sesión desde la interfaz.
- RF-1.4 Consultar los datos del perfil.
- RF-1.5 Modificar los datos básicos de la cuenta.
- RF-1.6 Cambiar la contraseña verificando previamente la contraseña actual.
- RF-1.7 Solicitar la baja de la cuenta.

RF-2 Autenticación, roles y permisos

La aplicación debe proteger las operaciones principales mediante autenticación y aplicar permisos según el rol y el estado del usuario.

- RF-2.1 Generar un token de acceso tras un inicio de sesión correcto.
- RF-2.2 Proteger las rutas privadas para impedir accesos no autenticados.
- RF-2.3 Distinguir entre usuario normal y usuario administrador.
- RF-2.4 Impedir el acceso a cuentas suspendidas o dadas de baja.
- RF-2.5 Restringir las acciones administrativas a usuarios con rol de administrador.

RF-3 Planes, suscripciones y contratación individual

La aplicación debe permitir consultar y contratar servicios que determinan el acceso a modelos y pipelines.

- RF-3.1 Consultar los planes disponibles.
- RF-3.2 Suscribirse a un plan.
- RF-3.3 Consultar los modelos de IA disponibles para contratación individual.
- RF-3.4 Alquilar un modelo de IA de forma individual.
- RF-3.5 Consultar compras, alquileres y suscripciones activas.
- RF-3.6 Activar de forma inmediata un servicio programado cuando proceda.
- RF-3.7 Modificar la renovación automática de planes o modelos contratados.

RF-4 Catálogo de modelos, modos y pipelines

La aplicación debe ofrecer al usuario un catálogo de pipelines ejecutables según los modelos disponibles para su cuenta.

- RF-4.1 Registrar modelos de IA disponibles en el sistema.
- RF-4.2 Asociar a cada modelo distintos modos de ejecución.
- RF-4.3 Definir pipelines formados por una o varias etapas ordenadas.
- RF-4.4 Mostrar al usuario únicamente los pipelines que puede ejecutar.
- RF-4.5 Informar al usuario cuando no tenga acceso a ningún pipeline.
- RF-4.6 Mostrar una guía de compra que ayude a identificar qué servicios son necesarios para ejecutar determinados pipelines.

RF-5 Configuración de pipelines

La aplicación debe permitir ejecutar pipelines con una configuración predefinida y, cuando sea necesario, modificar parámetros antes del procesamiento.

- RF-5.1 Consultar la configuración predeterminada de un pipeline.
- RF-5.2 Mostrar la configuración de forma editable en la interfaz.

- RF-5.3 Permitir al usuario modificar parámetros de ejecución en formato JSON.
- RF-5.4 Validar que la configuración personalizada tenga un formato correcto.
- RF-5.5 Registrar la configuración aplicada a la ejecución cuando sea distinta a la predeterminada.

RF-6 Procesamiento de imágenes mediante pipelines

La aplicación debe ejecutar el procesamiento de una imagen siguiendo las etapas del pipeline seleccionado.

- RF-6.1 Subir una imagen desde el panel principal.
- RF-6.2 Validar el archivo recibido antes de procesarlo.
- RF-6.3 Comprobar que el usuario tiene permiso para ejecutar el pipeline seleccionado.
- RF-6.4 Ejecutar las etapas del pipeline en el orden definido.
- RF-6.5 Encadenar la salida de una etapa como entrada de la siguiente cuando proceda.
- RF-6.6 Permitir pipelines con una única etapa o con varias etapas.
- RF-6.7 Generar resultados visuales y datos estructurados de cada etapa.

RF-7 Visualización y descarga de resultados

La aplicación debe mostrar los resultados de una ejecución de forma visual y permitir su descarga mientras estén disponibles.

- RF-7.1 Mostrar las imágenes generadas por cada etapa del pipeline.
- RF-7.2 Mostrar los datos JSON asociados a cada etapa.
- RF-7.3 Permitir la descarga de imágenes procesadas.
- RF-7.4 Permitir la descarga de resultados JSON.
- RF-7.5 Impedir la descarga de archivos inexistentes, caducados o no autorizados.

RF-8 Historial de ejecuciones

La aplicación debe permitir al usuario consultar las ejecuciones realizadas desde su cuenta.

- RF-8.1 Consultar el historial de ejecuciones del usuario autenticado.
- RF-8.2 Mostrar el pipeline ejecutado, fecha, estado, duración y posibles errores.
- RF-8.3 Indicar si una ejecución conserva archivos temporales disponibles.
- RF-8.4 Consultar el detalle de una ejecución concreta.
- RF-8.5 Impedir que un usuario acceda a ejecuciones de otros usuarios.

RF-9 Gestión de archivos temporales

La aplicación debe gestionar los archivos de entrada, salida e intermedios como recursos temporales asociados a una ejecución.

- RF-9.1 Registrar los archivos temporales asociados a una ejecución.
- RF-9.2 Asociar tokens de descarga a los archivos que deban ser accesibles desde la interfaz.
- RF-9.3 Definir una fecha de expiración para los archivos temporales.
- RF-9.4 Eliminar o invalidar archivos temporales caducados.
- RF-9.5 Evitar que la aplicación acumule indefinidamente imágenes procesadas.

RF-10 Administración de usuarios

La aplicación debe permitir al administrador consultar y gestionar el estado de las cuentas de usuario.

- RF-10.1 Acceder a un panel de administración.
- RF-10.2 Consultar el listado de usuarios.
- RF-10.3 Ver información básica de cada usuario.
- RF-10.4 Cambiar el estado de una cuenta cuando sea necesario.

- RF-10.5 Impedir que un administrador modifique accidentalmente el estado de su propia cuenta.

RF-11 Administración de pipelines

La aplicación debe permitir al administrador gestionar los pipelines disponibles en el sistema sin modificar directamente el código fuente.

- RF-11.1 Consultar los pipelines existentes.
- RF-11.2 Crear nuevos pipelines.
- RF-11.3 Definir las etapas de un pipeline.
- RF-11.4 Modificar la información y composición de un pipeline.
- RF-11.5 Deshabilitar o eliminar pipelines cuando sea posible.
- RF-11.6 Evitar la eliminación física de pipelines con ejecuciones asociadas, conservando la coherencia del historial.
- RF-11.7 Consultar el catálogo técnico de modelos y modos disponibles para definir etapas.
- RF-11.8 Validar que cada etapa utilice un modo perteneciente al modelo seleccionado y que ambos estén habilitados.

RF-12 Auditoría y mantenimiento del sistema

La aplicación debe incorporar funciones de revisión y mantenimiento que permitan conservar un estado coherente.

- RF-12.1 Consultar ejecuciones globales desde el panel de administración.
- RF-12.2 Renovar servicios activos cuando tengan renovación automática.
- RF-12.3 Desactivar servicios caducados cuando no deban renovarse.
- RF-12.4 Anonimizar cuentas dadas de baja tras el periodo previsto.
- RF-12.5 Registrar errores de ejecución de forma comprensible para el usuario.

RF-13 Interfaz web

La aplicación debe disponer de una interfaz web que permita utilizar las funciones principales sin acceder directamente a la API.

- RF-13.1 Disponer de página pública de inicio.
- RF-13.2 Disponer de pantallas de registro e inicio de sesión.
- RF-13.3 Disponer de panel principal de usuario.
- RF-13.4 Disponer de página de perfil.
- RF-13.5 Disponer de tienda y página de compras.
- RF-13.6 Disponer de guía de compra.
- RF-13.7 Disponer de historial y vista de resultados.
- RF-13.8 Disponer de panel de administración.

Requisitos no funcionales

Los requisitos no funcionales establecen las condiciones de calidad que debe cumplir el sistema, complementando a los requisitos funcionales descritos anteriormente.

- **RNF-1 Usabilidad.** La interfaz debe ser clara, comprensible y permitir ejecutar el flujo principal sin que el usuario conozca la estructura interna del sistema.
- **RNF-2 Seguridad.** Las operaciones privadas deben requerir autenticación y el servidor debe validar permisos antes de ejecutar acciones sensibles.
- **RNF-3 Privacidad.** Las imágenes y resultados deben tratarse como recursos temporales, evitando su conservación indefinida y controlando el acceso mediante tokens.
- **RNF-4 Rendimiento.** El sistema debe evitar procesamiento innecesarios cuando el archivo no sea válido o el usuario no tenga permisos.
- **RNF-5 Mantenibilidad.** El código debe organizarse de forma modular, separando controladores, modelos, servicios, plantillas y lógica específica de IA.
- **RNF-6 Escalabilidad funcional.** La arquitectura debe permitir añadir nuevos modelos, modos y pipelines con cambios reducidos sobre la estructura existente.

- **RNF-7 Trazabilidad.** El sistema debe conservar información básica de ejecuciones, estados y errores para poder revisar el uso principal de la aplicación.
- **RNF-8 Portabilidad.** La aplicación debe poder ejecutarse en un entorno documentado, con configuración centralizada y dependencias identificadas.
- **RNF-9 Robustez.** El sistema debe responder de forma controlada ante entradas no válidas, configuraciones incorrectas, archivos caducados o fallos internos.
- **RNF-10 Comprobabilidad.** Las partes principales deben poder verificarse mediante pruebas automatizadas, especialmente autenticación, permisos, tienda, pipelines y modelos de IA.

B.4. Especificación de requisitos

En esta sección se recogen los casos de uso principales del sistema. Antes de describirlos individualmente, la Figura B.1 muestra una visión general de los actores y funcionalidades más relevantes de la aplicación.

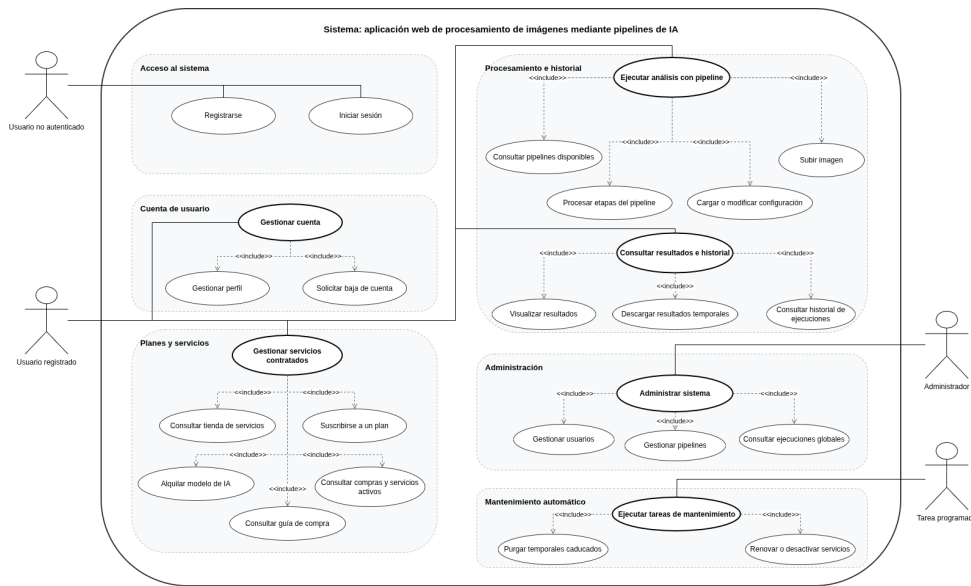


Figura B.1: Diagrama de casos de uso del sistema.

El diagrama diferencia cuatro actores: usuario no autenticado, usuario registrado, administrador y sistema. El usuario no autenticado puede registrarse e iniciar sesión. El usuario registrado puede gestionar su cuenta,

contratar servicios, consultar pipelines disponibles, ejecutar análisis sobre imágenes, visualizar resultados y consultar su historial. El administrador dispone de funciones de gestión de usuarios, pipelines y ejecuciones globales. Por último, el sistema representa las tareas automáticas de mantenimiento, como la limpieza de archivos temporales, renovación o desactivación de servicios y anonimización de cuentas dadas de baja.

A partir de esta visión general, se detallan a continuación los casos de uso identificados. Para cada uno de ellos se resume el actor principal, los requisitos asociados, la finalidad del caso de uso, sus precondiciones, el flujo principal, el resultado esperado y las posibles excepciones.

Actor	Usuario no autenticado.
RF	RF-1.1, RF-13.2.
Descripción	Creación de una cuenta para acceder a las funciones privadas de la aplicación.
Precondición	No existe una cuenta con el mismo correo o nombre de usuario.
Flujo principal	El usuario accede al registro, introduce sus datos, el sistema valida los campos, comprueba duplicados, crea la cuenta y asigna el rol de usuario.
Resultado	La cuenta queda creada y puede utilizarse para iniciar sesión.
Excepciones	Correo duplicado, nombre de usuario duplicado, contraseña no válida o datos incompletos.
Importancia	Alta.

Tabla B.1: CU-1 Registrarse.

Actor	Usuario.
RF	RF-1.2, RF-2.1, RF-2.2, RF-13.2.
Descripción	Acceso a la aplicación mediante credenciales válidas.
Precondición	El usuario dispone de una cuenta activa.
Flujo principal	El usuario introduce sus credenciales, el sistema las valida, comprueba el estado de la cuenta, genera el token de acceso y redirige al panel correspondiente.

Resultado	El usuario queda autenticado en el sistema.
Excepciones	Credenciales incorrectas, usuario inexistente o cuenta suspendida.
Importancia	Alta.

Tabla B.2: CU-2 Iniciar sesión.

Actor	Usuario.
RF	RF-1.4, RF-1.5, RF-1.6, RF-13.4.
Descripción	Consulta y modificación de los datos básicos de la cuenta.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede al perfil, consulta sus datos, modifica los campos permitidos y el sistema valida y guarda los cambios. Si cambia la contraseña, debe indicar la contraseña actual.
Resultado	Los datos de la cuenta quedan actualizados.
Excepciones	Contraseña actual incorrecta, nueva contraseña no válida o error de persistencia.
Importancia	Media.

Tabla B.3: CU-3 Gestionar perfil.

Actor	Usuario.
RF	RF-1.7, RF-12.4.
Descripción	Baja lógica de una cuenta y desactivación de sus servicios activos.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario solicita la baja, el sistema localiza la cuenta, la marca como borrada, desactiva sus servicios activos y registra la fecha de baja.
Resultado	La cuenta queda dada de baja y sus servicios dejan de estar activos.
Excepciones	Identidad no localizada o error al aplicar la baja.
Importancia	Media.

Tabla B.4: CU-4 Solicitar baja de cuenta.

Actor	Usuario.
RF	RF-3.1, RF-3.3, RF-13.5.
Descripción	Consulta de planes y modelos disponibles para contratación.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede a la tienda, el sistema recupera los planes y modelos habilitados y muestra el estado de cada servicio respecto al usuario.
Resultado	El usuario puede decidir qué plan o modelo contratar.
Excepciones	Error al recuperar el catálogo de servicios.
Importancia	Alta.

Tabla B.5: CU-5 Consultar tienda de servicios.

Actor	Usuario.
RF	RF-3.2, RF-3.6, RF-3.7.
Descripción	Contratación de un plan de servicio para ampliar el acceso a modelos y pipelines.
Precondición	El usuario ha iniciado sesión y el plan está disponible.
Flujo principal	El usuario selecciona un plan, el sistema comprueba su disponibilidad, registra la suscripción, define su vigencia y actualiza los permisos de acceso.
Resultado	El usuario dispone del plan contratado.
Excepciones	Plan inexistente, plan no disponible o error al registrar la suscripción.
Importancia	Alta.

Tabla B.6: CU-6 Suscribirse a un plan.

Actor	Usuario.
RF	RF-3.3, RF-3.4, RF-3.6, RF-3.7.
Descripción	Contratación individual de un modelo de IA para ejecutar pipelines que dependan de él.
Precondición	El usuario ha iniciado sesión y el modelo está habilitado.
Flujo principal	El usuario selecciona un modelo, el sistema comprueba que exista, que esté habilitado y que no exista ya un acceso activo, y registra el alquiler con su periodo de vigencia.
Resultado	El usuario puede acceder a pipelines que utilicen el modelo contratado.
Excepciones	Modelo inexistente, modelo ya contratado, fecha inválida o error de registro.
Importancia	Alta.

Tabla B.7: CU-7 Alquilar un modelo de IA.

Actor	Usuario.
RF	RF-3.5, RF-3.6, RF-3.7, RF-13.5.
Descripción	Consulta de planes, alquileres y servicios activos o programados.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede a sus compras, el sistema recupera sus suscripciones y alquileres, muestra fechas, estado y renovación automática, y permite modificar las opciones admitidas.
Resultado	El usuario conoce el estado de sus servicios.
Excepciones	Error al recuperar la información o intento de modificar un servicio inexistente.
Importancia	Media-alta.

Tabla B.8: CU-8 Consultar compras y servicios activos.

Actor	Usuario.
RF	RF-4.6, RF-13.6.
Descripción	Consulta orientativa de los modelos o planes necesarios para ejecutar cada pipeline.

Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede a la guía, el sistema obtiene los pipelines y sus modelos requeridos, indica qué requisitos cumple el usuario y qué servicios tendría que contratar.
Resultado	El usuario dispone de información para decidir qué servicio contratar.
Excepciones	No existen pipelines disponibles o se produce un error al calcular dependencias.
Importancia	Media.

Tabla B.9: CU-9 Consultar guía de compra.

Actor	Usuario.
RF	RF-4.3, RF-4.4, RF-4.5, RF-13.3.
Descripción	Obtención de los pipelines ejecutables según rol, plan y modelos contratados.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede al panel principal, el sistema identifica su cuenta, revisa sus permisos y muestra únicamente los pipelines que puede ejecutar.
Resultado	El usuario puede seleccionar un pipeline permitido.
Excepciones	Si no hay pipelines disponibles, se muestra un aviso y se deshabilita la ejecución.
Importancia	Muy alta.

Tabla B.10: CU-10 Consultar pipelines disponibles.

Actor	Usuario.
RF	RF-5.1, RF-5.2, RF-5.3, RF-5.4, RF-5.5.
Descripción	Consulta y edición de la configuración de ejecución de un pipeline.
Precondición	El usuario ha seleccionado un pipeline permitido.
Flujo principal	El usuario solicita la configuración, el sistema recupera la configuración predeterminada de las etapas, la interfaz la muestra en formato editable y el usuario puede modificarla antes de ejecutar.

Resultado	La configuración queda preparada para utilizarse en la ejecución.
Excepciones	Configuración no encontrada, JSON incorrecto o pipeline no permitido.
Importancia	Alta.

Tabla B.11: CU-11 Cargar y modificar configuración del pipeline.

Actor	Usuario.
RF	RF-6.1, RF-6.2, RF-6.3, RF-6.4, RF-6.5, RF-6.6, RF-6.7.
Descripción	Flujo principal de procesamiento de una imagen mediante un pipeline.
Precondición	El usuario ha iniciado sesión y tiene acceso al pipeline seleccionado.
Flujo principal	El usuario selecciona un pipeline, adjunta una imagen, ajusta opcionalmente la configuración y lanza el análisis. El sistema valida imagen, configuración y permisos, ejecuta las etapas, registra la ejecución y devuelve los resultados.
Resultado	La ejecución queda registrada y el usuario puede visualizar los resultados generados.
Excepciones	Archivo inválido, pipeline no permitido, configuración incorrecta, error de procesamiento o ausencia de resultados útiles.
Importancia	Muy alta.

Tabla B.12: CU-12 Ejecutar pipeline sobre una imagen.

Actor	Usuario.
RF	RF-7.1, RF-7.2, RF-7.3, RF-7.4, RF-7.5, RF-9.2.
Descripción	Visualización y descarga temporal de resultados generados por una ejecución.
Precondición	Existe una ejecución completada con archivos disponibles.
Flujo principal	El sistema muestra las etapas, imágenes y datos JSON generados. El usuario puede descargar los archivos disponibles y el sistema resuelve la descarga mediante tokens temporales.

Resultado	El usuario visualiza o descarga los resultados mientras estén disponibles.
Excepciones	Archivo caducado, token inválido, archivo eliminado físicamente o acceso no autorizado.
Importancia	Alta.

Tabla B.13: CU-13 Visualizar y descargar resultados.

Actor	Usuario.
RF	RF-8.1, RF-8.2, RF-8.3, RF-8.4, RF-8.5, RF-13.7.
Descripción	Consulta de ejecuciones realizadas por el usuario autenticado.
Precondición	El usuario ha iniciado sesión.
Flujo principal	El usuario accede al historial, el sistema recupera sus ejecuciones y muestra pipeline, fecha, estado, duración y errores. Si selecciona una ejecución, se muestran sus detalles y archivos disponibles.
Resultado	El usuario puede revisar sus ejecuciones y resultados temporales disponibles.
Excepciones	No existen ejecuciones, ejecución no encontrada o intento de acceder a una ejecución ajena.
Importancia	Alta.

Tabla B.14: CU-14 Consultar historial de ejecuciones.

Actor	Administrador.
RF	RF-2.3, RF-2.5, RF-10.1, RF-13.8.
Descripción	Acceso del administrador a la consola de gestión del sistema.
Precondición	El usuario ha iniciado sesión con rol de administrador.
Flujo principal	El administrador inicia sesión, el sistema detecta su rol, lo redirige al panel de administración y muestra las secciones de gestión disponibles.
Resultado	El administrador puede acceder a funciones de revisión y gestión.

Excepciones	Usuario no administrador o token no válido.
Importancia	Alta.

Tabla B.15: CU-15 Acceder al panel de administración.

Actor	Administrador.
RF	RF-10.2, RF-10.3, RF-10.4, RF-10.5.
Descripción	Consulta de usuarios y modificación controlada de su estado.
Precondición	El administrador ha accedido al panel de administración.
Flujo principal	El administrador consulta los usuarios, selecciona una cuenta, solicita un cambio de estado y el sistema valida y aplica la operación.
Resultado	El estado de la cuenta queda actualizado.
Excepciones	Usuario no encontrado, intento de auto-bloqueo o error al actualizar la base de datos.
Importancia	Alta.

Tabla B.16: CU-16 Gestionar usuarios desde administración.

Actor	Administrador.
RF	RF-11.1, RF-11.2, RF-11.3, RF-11.4, RF-11.5, RF-11.6, RF-11.7, RF-11.8.
Descripción	Creación, modificación, deshabilitación o eliminación de pipelines desde administración.
Precondición	El administrador ha iniciado sesión y existen modelos y modos registrados.
Flujo principal	El administrador consulta el catálogo, crea o selecciona un pipeline, define sus etapas y el sistema valida que los modelos y modos sean coherentes y estén habilitados antes de guardar los cambios.
Resultado	El catálogo de pipelines queda actualizado sin modificar el código fuente.
Excepciones	Pipeline inexistente, etapas inválidas, modelo o modo deshabilitado, falta de permisos o historial asociado que impida el borrado físico.

Importancia Muy alta.

Tabla B.17: CU-17 Gestionar pipelines desde administración.

Actor	Administrador.
RF	RF-12.1, RNF-7.
Descripción	Consulta administrativa del histórico general de ejecuciones del sistema.
Precondición	El administrador ha accedido al panel de administración.
Flujo principal	El administrador solicita la vista global, el sistema obtiene las ejecuciones registradas y muestra usuario, pipeline, fecha, estado, duración y errores.
Resultado	El administrador dispone de una visión global del uso del sistema.
Excepciones	Falta de permisos o error al obtener la información.
Importancia	Media-alta.

Tabla B.18: CU-18 Consultar ejecuciones globales.

Actor	Sistema.
RF	RF-9.4, RF-9.5, RF-12.2, RF-12.3, RF-12.4.
Descripción	Revisión de temporales, servicios caducados, renovaciones automáticas y cuentas dadas de baja.
Precondición	Existen ejecuciones, servicios o cuentas susceptibles de revisión.
Flujo principal	El sistema comprueba archivos temporales, elimina o invalida recursos caducados, revisa servicios vencidos, renueva o desactiva accesos y anonimiza cuentas dadas de baja cuando corresponda.
Resultado	El sistema conserva un estado coherente y evita acumulación de recursos innecesarios.
Excepciones	Si una tarea falla, no debe interrumpir el funcionamiento principal de la aplicación.
Importancia	Alta.

Tabla B.19: CU-19 Ejecutar tareas de mantenimiento.

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se recoge el diseño técnico del sistema, el objetivo es mostrar la estructura interna de la aplicación.

C.2. Diseño de datos

El modelo de datos se ha diseñado para representar de forma separada las entidades principales del sistema: usuarios, roles, planes, modelos de IA, pipelines, ejecuciones y archivos temporales. Esta división reduce redundancias, mantiene la trazabilidad de las operaciones y facilita futuras ampliaciones de la aplicación.

Diagrama entidad-relación

La Figura C.1 muestra el diagrama entidad-relación del sistema, donde se representan las entidades principales y sus asociaciones.

En el modelo, **USUARIO** actúa como entidad central, al relacionarse con roles, suscripciones, alquileres, pipelines y ejecuciones.

Diagrama del modelo relacional

La Figura C.2 recoge el modelo relacional final, con las tablas, claves y relaciones utilizadas para implementar la persistencia.

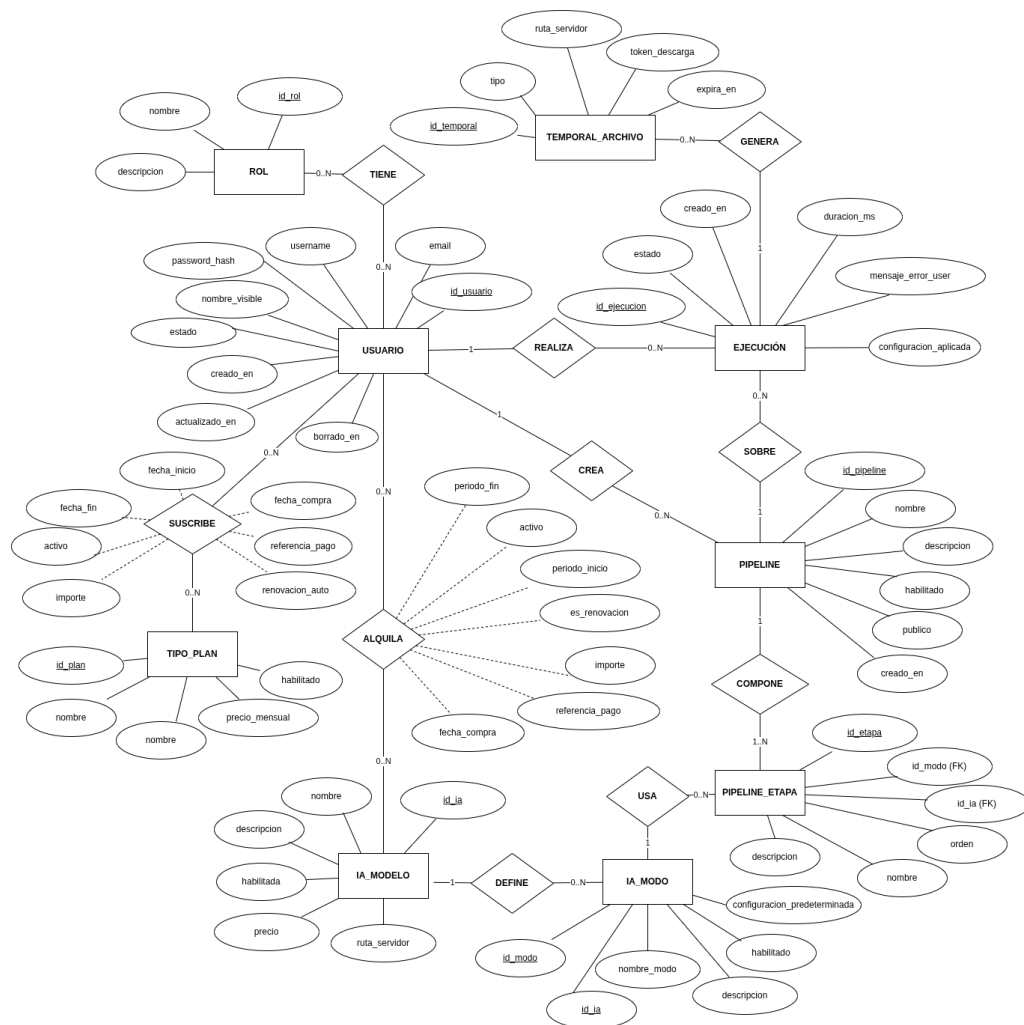


Figura C.1: Diagrama entidad-relación del sistema.

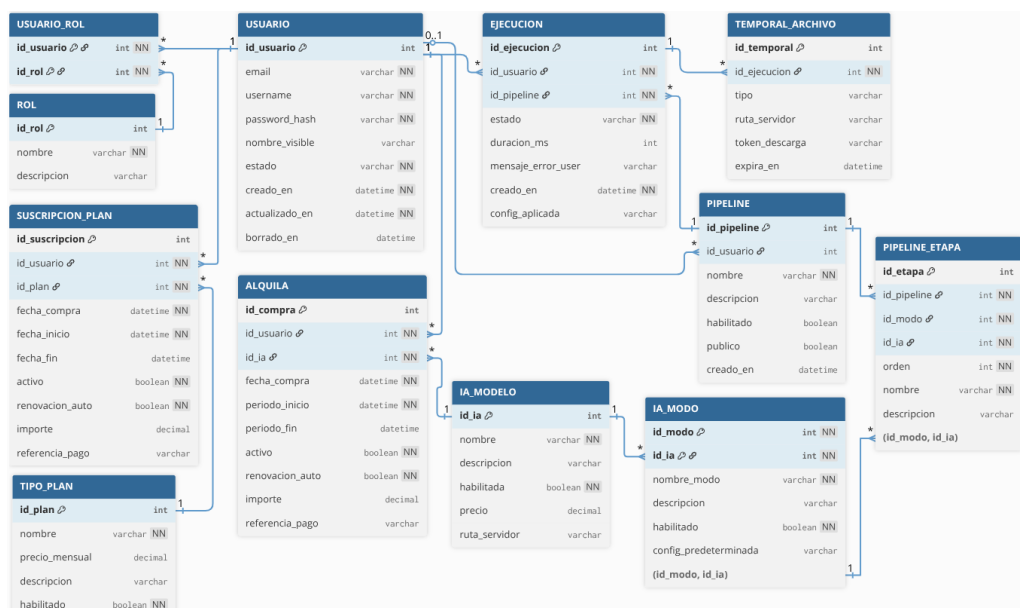


Figura C.2: Diagrama del modelo relacional del sistema.

Las relaciones muchos a muchos se resuelven mediante tablas intermedias, como `USUARIO_ROL`. El resto de asociaciones se implementan mediante claves foráneas directas.

Diccionario de datos

A continuación se resumen las tablas principales del modelo relacional. Para evitar una extensión excesiva, se recoge únicamente la finalidad de cada atributo y las anotaciones más relevantes de diseño.

Tabla `USUARIO`. Se describe en la Tabla [C.1](#).

Atributo	Descripción
<code>id_usuario</code>	Identificador interno y clave primaria.
<code>email</code>	Correo electrónico obligatorio y único.
<code>username</code>	Nombre de usuario obligatorio y único.
<code>password_hash</code>	Contraseña almacenada mediante hash.
<code>nombre_visible</code>	Nombre mostrado en la interfaz.
<code>estado</code>	Estado lógico de la cuenta.
<code>creado_en</code>	Fecha de creación.
<code>actualizado_en</code>	Fecha de última modificación.
<code>borrado_en</code>	Fecha de baja lógica, si procede.

Tabla C.1: Atributos de la tabla `USUARIO`.

Nota: `email` y `username` son únicos para evitar cuentas duplicadas. La baja se plantea de forma lógica para conservar la trazabilidad.

Tabla `ROL`. Se describe en la Tabla [C.2](#).

Atributo	Descripción
<code>id_rol</code>	Identificador interno y clave primaria.
<code>nombre</code>	Nombre del rol, obligatorio y único.
<code>descripcion</code>	Descripción opcional del rol.

Tabla C.2: Atributos de la tabla `ROL`.

Tabla `USUARIO_ROL`. Se describe en la Tabla [C.3](#).

Atributo	Descripción
<code>id_usuario</code>	Usuario al que se asigna el rol.

id_rol	Rol asignado al usuario.
--------	--------------------------

Tabla C.3: Atributos de la tabla USUARIO_ROL.

Nota: la clave primaria compuesta (id_usuario, id_rol) evita asignaciones duplicadas.

Tabla TIPO_PLAN. Se describe en la Tabla C.4.

Atributo	Descripción
id_plan	Identificador interno y clave primaria.
nombre	Nombre del plan, obligatorio y único.
precio_mensual	Precio mensual del plan.
descripcion	Descripción del plan.
habilitado	Indica si el plan puede contratarse.

Tabla C.4: Atributos de la tabla TIPO_PLAN.

Tabla SUSCRIPCION_PLAN. Se describe en la Tabla C.5.

Atributo	Descripción
id_suscripcion	Identificador interno y clave primaria.
id_usuario	Usuario titular de la suscripción.
id_plan	Plan contratado.
fecha_compra	Fecha de contratación.
fecha_inicio	Inicio de la vigencia.
fecha_fin	Fin de la vigencia, si existe.
activo	Indica si la suscripción está activa.
renovacion_auto	Indica si se renueva automáticamente.
importe	Importe aplicado en la contratación.
referencia_pago	Referencia opcional del pago.

Tabla C.5: Atributos de la tabla SUSCRIPCION_PLAN.

Nota: se conserva como histórico de contratación, por lo que sus relaciones principales usan RESTRICT.

Tabla IA_MODELO. Se describe en la Tabla C.6.

Atributo	Descripción
id_ia	Identificador interno y clave primaria.
nombre	Nombre del modelo, obligatorio y único.

descripcion	Descripción funcional del modelo.
habilitada	Indica si el modelo puede utilizarse.
precio	Precio del acceso individual.
ruta_servidor	Ruta o referencia interna del modelo.

Tabla C.6: Atributos de la tabla IA_MODELO.

Tabla IA_MODALO. Se describe en la Tabla C.7.

Atributo	Descripción
id_modo	Identificador interno del modo.
id_ia	Modelo al que pertenece.
nombre_modo	Nombre funcional del modo.
descripcion	Descripción del modo.
habilitado	Indica si el modo está disponible.
config_predeterminada	Configuración base del modo.

Tabla C.7: Atributos de la tabla IA_MODALO.

Nota: UNIQUE (id_ia, nombre_modo) evita duplicar modos dentro de un mismo modelo.

Tabla ALQUILA. Se describe en la Tabla C.8.

Atributo	Descripción
id_compra	Identificador interno y clave primaria.
id_usuario	Usuario que alquila el modelo.
id_ia	Modelo contratado.
fecha_compra	Fecha de la operación.
periodo_inicio	Inicio de la vigencia.
periodo_fin	Fin de la vigencia, si existe.
activo	Indica si el acceso está activo.
renovacion_auto	Indica si se renueva automáticamente.
importe	Importe aplicado.
referencia_pago	Referencia opcional del pago.

Tabla C.8: Atributos de la tabla ALQUILA.

Tabla PIPELINE. Se describe en la Tabla C.9.

Atributo	Descripción
id_pipeline	Identificador interno y clave primaria.
id_usuario	Usuario creador o propietario, si existe.

nombre	Nombre del pipeline.
descripcion	Descripción del flujo.
habilitado	Indica si puede ejecutarse.
creado_en	Fecha de creación.
publico	Indica si está disponible para usuarios.

Tabla C.9: Atributos de la tabla PIPELINE.

Nota: se usa ON DELETE SET NULL para conservar pipelines aunque desaparezca su creador.

Tabla EJECUCION. Se describe en la Tabla C.10.

Atributo	Descripción
id_ejecucion	Identificador interno y clave primaria.
id_usuario	Usuario que lanza la ejecución.
id_pipeline	Pipeline ejecutado.
estado	Estado de la ejecución.
duracion_ms	Duración en milisegundos.
mensaje_error_user	Mensaje de error mostrado al usuario.
creado_en	Fecha de creación.
config_aplicada	Configuración utilizada durante el procesamiento.

Tabla C.10: Atributos de la tabla EJECUCION.

Nota: las ejecuciones se conservan como histórico, por lo que sus relaciones principales usan RESTRICT.

Tabla TEMPORAL_ARCHIVO. Se describe en la Tabla C.11.

Atributo	Descripción
id_temporal	Identificador interno y clave primaria.
id_ejecucion	Ejecución asociada.
tipo	Tipo de archivo temporal.
ruta_servidor	Ruta interna del archivo.
token_descarga	Token de descarga controlada.
expira_en	Fecha de expiración.

Tabla C.11: Atributos de la tabla TEMPORAL_ARCHIVO.

Nota: el token evita exponer rutas internas. La relación con EJECUCION usa ON DELETE CASCADE.

Tabla PIPELINE_ETAPA. Se describe en la Tabla C.12.

Atributo	Descripción
<code>id_etapa</code>	Identificador interno y clave primaria.
<code>id_pipeline</code>	Pipeline al que pertenece.
<code>id_modo</code>	Modo ejecutado en la etapa.
<code>id_ia</code>	Modelo asociado al modo.
<code>orden</code>	Posición de la etapa dentro del flujo.
<code>nombre</code>	Nombre funcional de la etapa.
<code>descripcion</code>	Descripción de la etapa.

Tabla C.12: Atributos de la tabla PIPELINE_ETAPA.

Nota: las etapas dependen del pipeline y se eliminan en cascada. La referencia compuesta (`id_modo`, `id_ia`) garantiza la coherencia entre modelo y modo.

Relaciones principales y valoración del modelo

Las relaciones más relevantes del modelo son:

- **USUARIO – ROL:** relación muchos a muchos mediante **USUARIO_ROL**.
- **USUARIO – SUSCRIPCION_PLAN – TIPO_PLAN:** registra la contratación de planes.
- **USUARIO – ALQUILA – IA_MODELO:** registra el acceso individual a modelos.
- **IA_MODELO – IA_MODO:** permite que un modelo tenga varios modos de ejecución.
- **PIPELINE – PIPELINE_ETAPA:** define flujos compuestos por etapas ordenadas.
- **IA_MODO – PIPELINE_ETAPA:** conecta cada etapa con el modo de IA que debe ejecutar.
- **USUARIO – EJECUCION – PIPELINE:** registra qué usuario ejecuta qué pipeline.
- **EJECUCION – TEMPORAL_ARCHIVO:** asocia los archivos temporales a la ejecución que los genera.

El modelo cubre las funcionalidades principales de la aplicación: autenticación, roles, planes, alquileres de modelos, definición de pipelines, ejecución de análisis y gestión de archivos temporales. La separación entre catálogo y operaciones concretas permite conservar histórico aunque cambien precios, planes o modelos.

Además, la separación entre PIPELINE y PIPELINE_ETAPA permite definir flujos configurables sin codificarlos de forma fija. Para mantener la integridad, se

usa **RESTRICT** en información histórica, **CASCADE** en dependencias fuertes y **SET NULL** cuando interesa conservar un registro aunque desaparezca su propietario.

C.3. Diseño arquitectónico

El diseño arquitectónico describe cómo se organiza internamente la aplicación y cómo se reparten las responsabilidades entre interfaz, controladores, servicios, modelos y base de datos.

La aplicación se ha desarrollado con **Flask**. Por ello, no se aplica un patrón MVC clásico de forma estricta, sino una adaptación en la que los **Blueprints** actúan como controladores, las plantillas HTML y respuestas JSON como vistas, y las clases **SQLAlchemy** como modelo persistente. Además, se incorpora una capa de servicios para aislar la lógica de negocio y el procesamiento de imágenes.

De forma resumida, la arquitectura se organiza en los siguientes bloques:

- **Cliente:** navegador desde el que el usuario interactúa con la aplicación.
- **Vistas:** plantillas HTML y respuestas JSON generadas por la aplicación.
- **Controladores:** módulos Flask encargados de recibir peticiones, validar datos, comprobar permisos y devolver respuestas.
- **Servicios:** lógica interna de ejecución de pipelines, selección de modelos y procesamiento de imágenes.
- **Modelo:** entidades SQLAlchemy que representan usuarios, roles, planes, modelos, pipelines, ejecuciones y archivos temporales.
- **Base de datos:** persistencia principal del sistema.

A continuación se presentan tres vistas complementarias: la arquitectura general, la organización real de módulos y la estructura de la capa de servicios.

Arquitectura MVC adaptada a Flask con capa de servicios

La Figura **C.3** muestra la arquitectura general de la aplicación. En ella se observa cómo el cliente se comunica con Flask mediante peticiones HTTP,

que son atendidas por controladores y derivadas, cuando es necesario, hacia servicios internos o hacia la capa de persistencia.

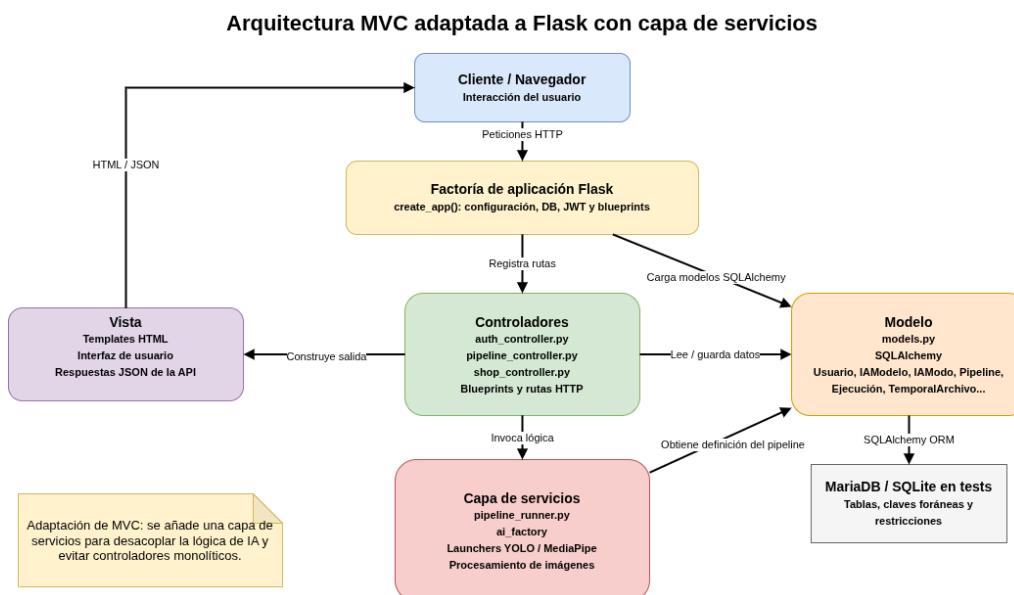


Figura C.3: Arquitectura MVC adaptada a Flask con capa de servicios.

Los controladores son `auth_controller.py`, `shop_controller.py` y `pipeline_controller.py`. El primero gestiona autenticación, registro, perfil y usuarios; el segundo agrupa las operaciones de tienda, planes y alquileres; y el tercero concentra la consulta, configuración, ejecución e historial de pipelines.

La capa de servicios evita que los controladores acumulen toda la lógica. En especial, permite separar la ejecución de pipelines y la resolución de modelos de IA respecto de las rutas HTTP. Esta decisión mejora la mantenibilidad y facilita añadir nuevos modelos, modos o flujos de procesamiento.

El modelo persistente se implementa mediante SQLAlchemy, manteniendo una correspondencia directa con las tablas descritas en el diseño de datos. Esta separación también favorece las pruebas, ya que permite probar controladores, servicios y persistencia de forma más aislada.

Organización de módulos y dependencias principales

La Figura C.4 muestra la organización real del proyecto a nivel de archivos, paquetes y dependencias principales.

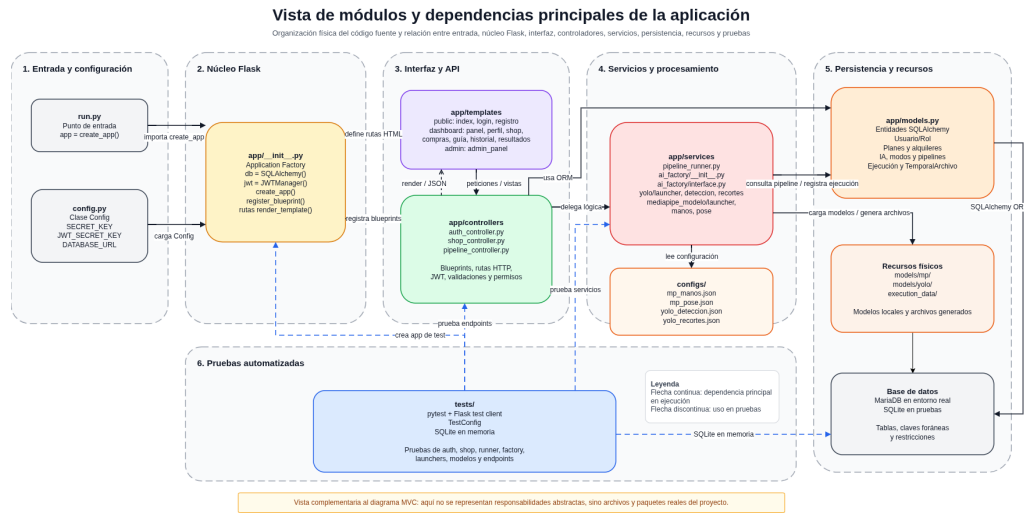


Figura C.4: Vista de módulos y dependencias principales de la aplicación.

El punto de entrada se encuentra en `run.py`, que crea la aplicación mediante `create_app()`. La configuración se centraliza en `config.py`, mientras que `app/__init__.py` inicializa extensiones como `SQLAlchemy` y `JWT`, carga la configuración y registra los `Blueprints`.

La carpeta `app/controllers` contiene las rutas HTTP y las validaciones iniciales. La carpeta `app/templates` agrupa las plantillas HTML de la interfaz. La lógica de aplicación se sitúa en `app/services`, donde se encuentran el orquestador de pipelines y la factoría de modelos de IA. Por su parte, `app/models.py` concentra las entidades `SQLAlchemy`.

También existen recursos auxiliares, como las configuraciones de ejecución en `configs`, los modelos locales en carpetas de `models` y los archivos temporales generados durante las ejecuciones. Finalmente, el paquete `tests` contiene las pruebas automatizadas, ejecutadas en un entorno aislado.

Esta organización refleja la arquitectura definida: entrada, configuración, controladores, servicios, modelo, recursos y pruebas quedan separados en componentes diferenciados.

Diagrama de clases y módulos principales de la capa de servicios

La Figura C.5 muestra los elementos principales que intervienen en la ejecución de pipelines y en la resolución dinámica de modelos de inteligencia artificial.

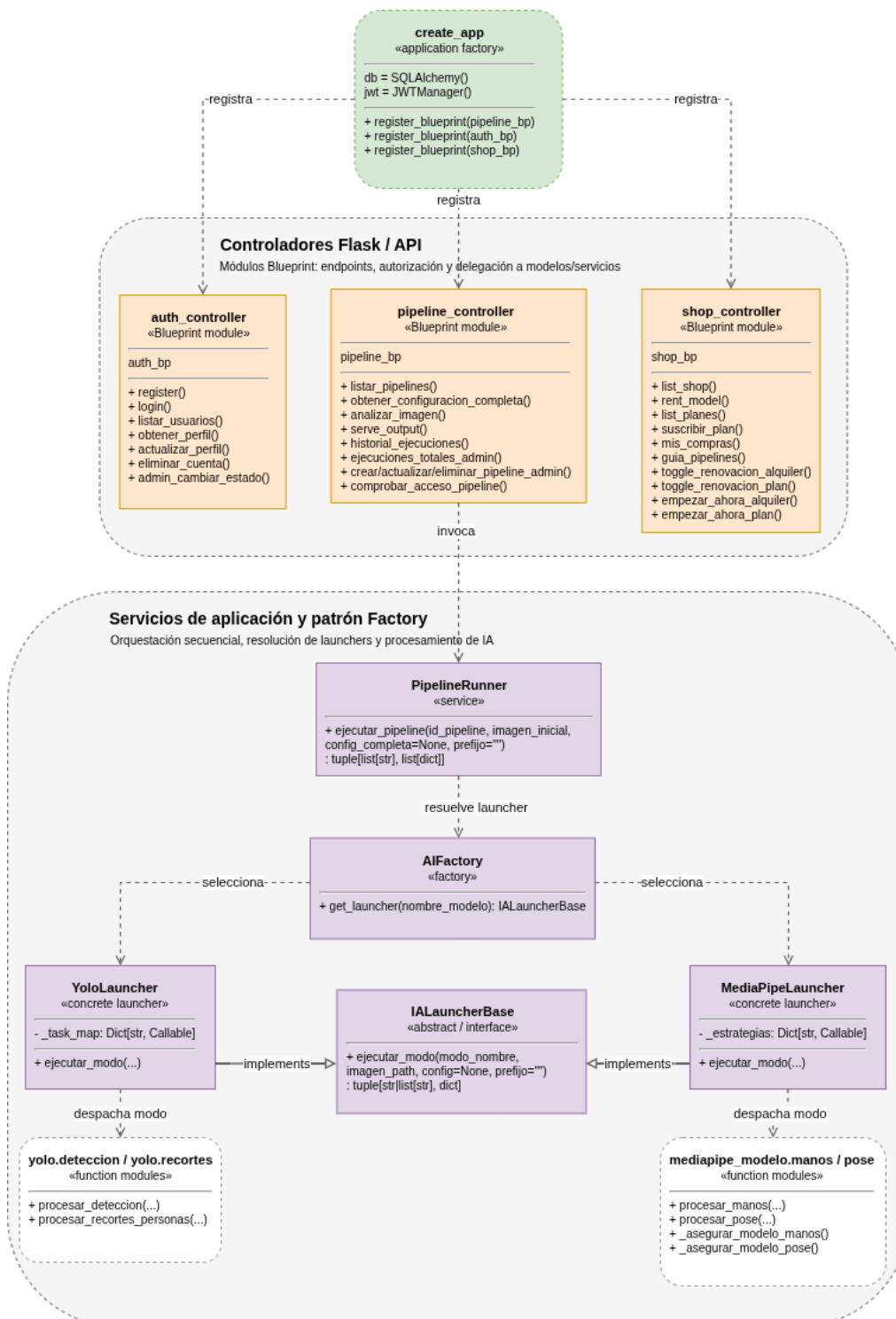


Figura C.5: Diagrama de clases y módulos principales de la capa de servicios.

El flujo parte de `pipeline_controller`, que recibe la solicitud de ejecución, valida la imagen, comprueba permisos y delega el procesamiento en `PipelineRunner`. Este servicio recorre las etapas del pipeline, obtiene la configuración aplicable y solicita a la factoría el lanzador correspondiente.

La factoría de IA desacopla el orquestador de las implementaciones concretas. Si una etapa requiere YOLO, se selecciona el lanzador correspondiente; si requiere MediaPipe, se utiliza el lanzador asociado a ese modelo. Cada lanzador encapsula la lógica específica y delega finalmente en los módulos funcionales de procesamiento.

Gracias a esta estructura, los pipelines no quedan codificados como funciones fijas, sino como secuencias de etapas almacenadas y configurables. Esto permite que el administrador modifique la composición de los pipelines desde la aplicación y facilita la incorporación futura de nuevos modelos o modos de ejecución.

C.4. Diseño procedimental

El diseño procedimental describe los principales flujos de ejecución del sistema a partir de los casos de uso definidos.

Los diagramas incluyen los flujos principales y, cuando procede, alternativas de error relacionadas con credenciales incorrectas, falta de permisos, validaciones no superadas, recursos inexistentes o fallos de persistencia.

Flujos de autenticación y gestión de cuenta

Este bloque recoge los procedimientos relacionados con el registro, inicio de sesión, gestión del perfil y baja lógica de una cuenta.

La Figura C.6 muestra el registro de usuario: validación de datos, comprobación de duplicados, creación de la cuenta y asignación del rol correspondiente.

La Figura C.7 representa el inicio de sesión, incluyendo la validación de credenciales, comprobación del estado de la cuenta y generación del token de acceso.

La Figura C.8 recoge la consulta y modificación del perfil del usuario autenticado.

La Figura C.9 muestra la baja lógica de la cuenta y la desactivación de los servicios asociados.

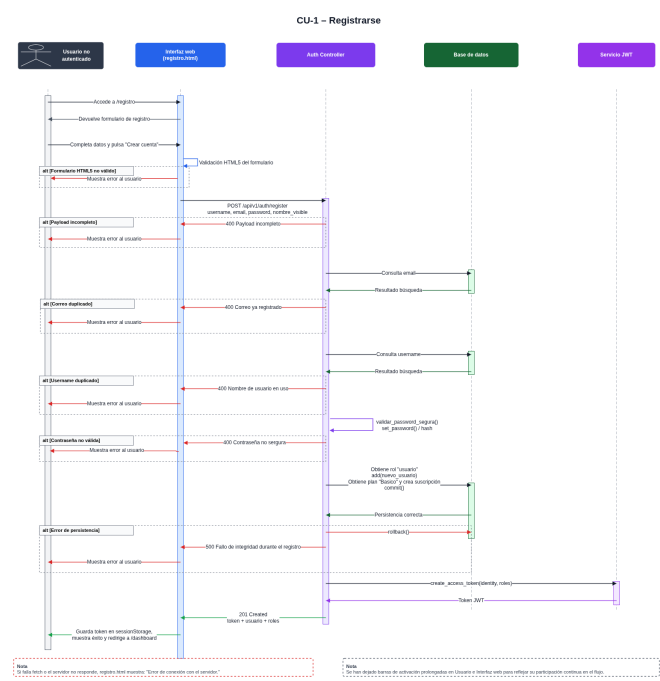


Figura C.6: Diagrama de secuencia del CU-1 Registrarse.

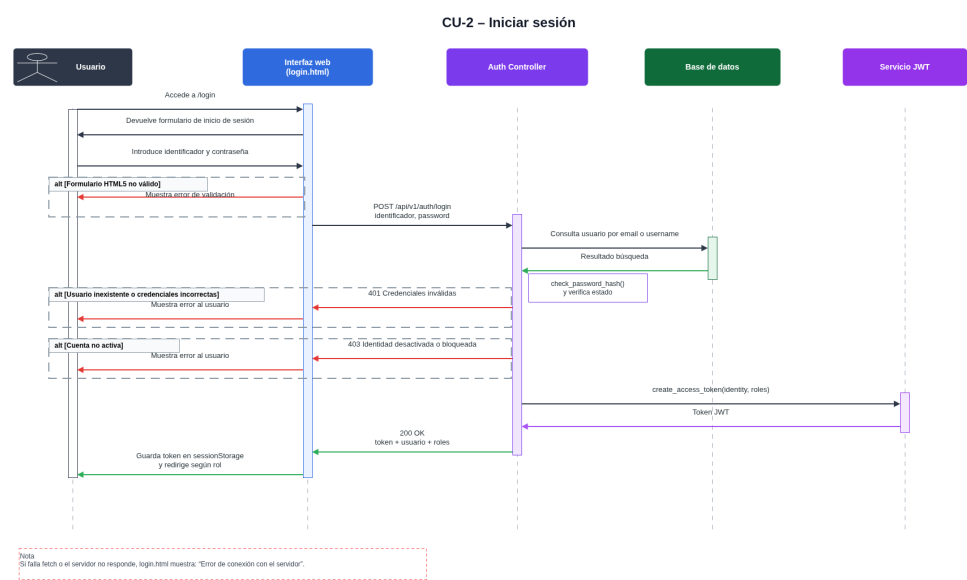


Figura C.7: Diagrama de secuencia del CU-2 Iniciar sesión.

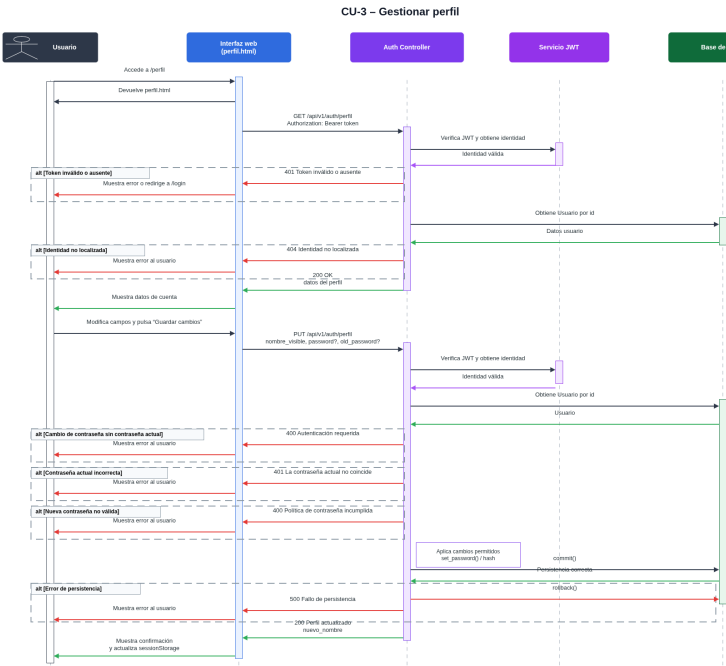
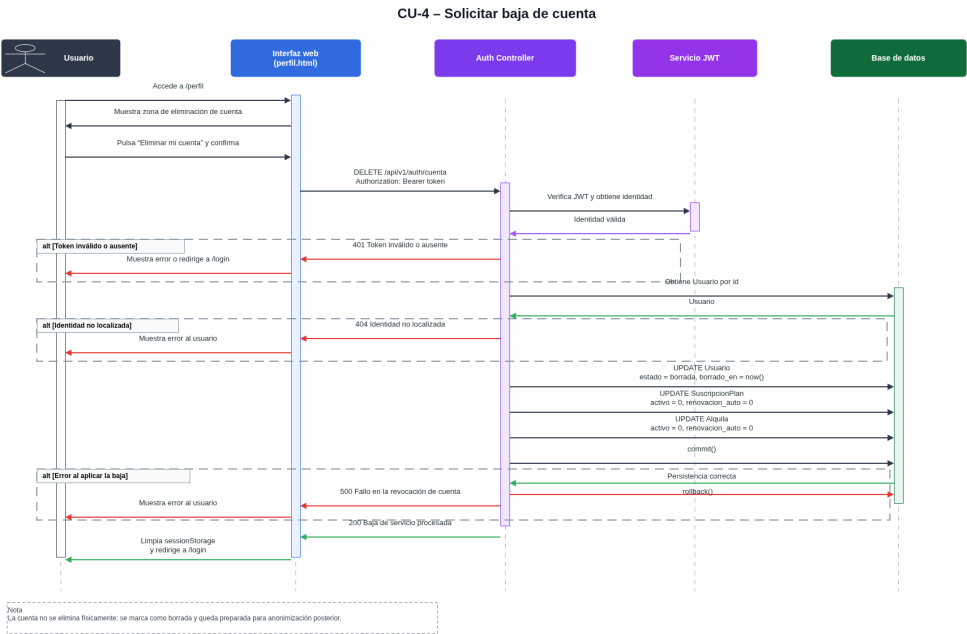


Figura C.8: Diagrama de secuencia del CU-3 Gestionar perfil.



Nota
La cuenta no se elimina físicamente: se marca como borrada y queda preparada para anonimización posterior.

Figura C.9: Diagrama de secuencia del CU-4 Solicitar baja de cuenta.

Flujos de tienda, planes y servicios

Este bloque agrupa los procedimientos relacionados con la consulta de servicios, contratación de planes, alquiler de modelos de IA y revisión de compras activas.

La Figura C.10 representa la consulta de la tienda, donde se recuperan planes, modelos disponibles y estado de contratación del usuario.

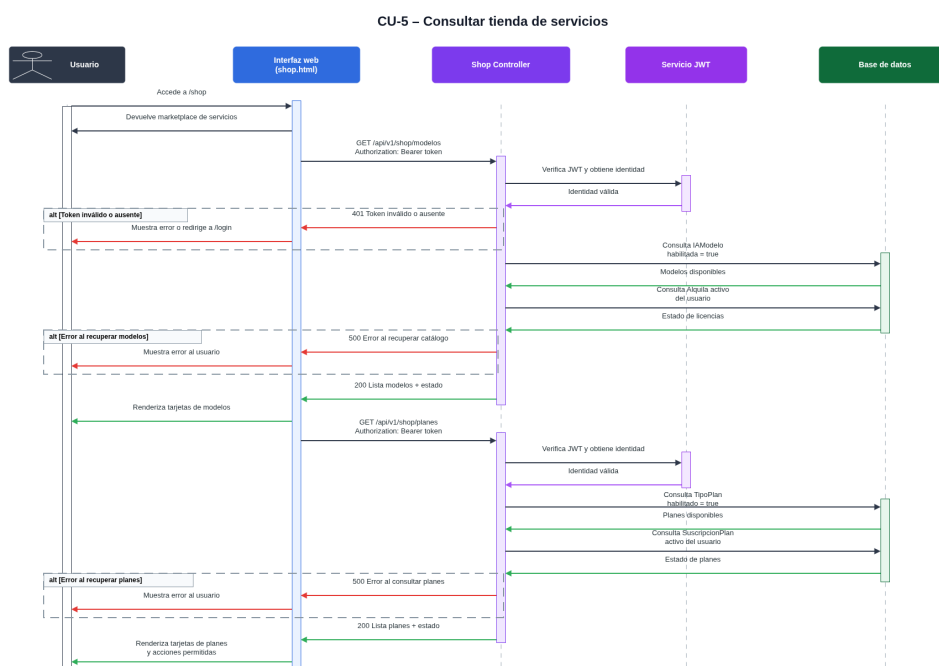


Figura C.10: Diagrama de secuencia del CU-5 Consultar tienda de servicios.

La Figura C.11 muestra el proceso de suscripción a un plan.

La Figura C.12 representa el alquiler individual de un modelo de inteligencia artificial.

La Figura C.13 recoge la consulta de compras, suscripciones y alquileres activos del usuario.

La Figura C.14 muestra la guía de compra, que cruza los pipelines disponibles con los modelos requeridos y los servicios activos del usuario.

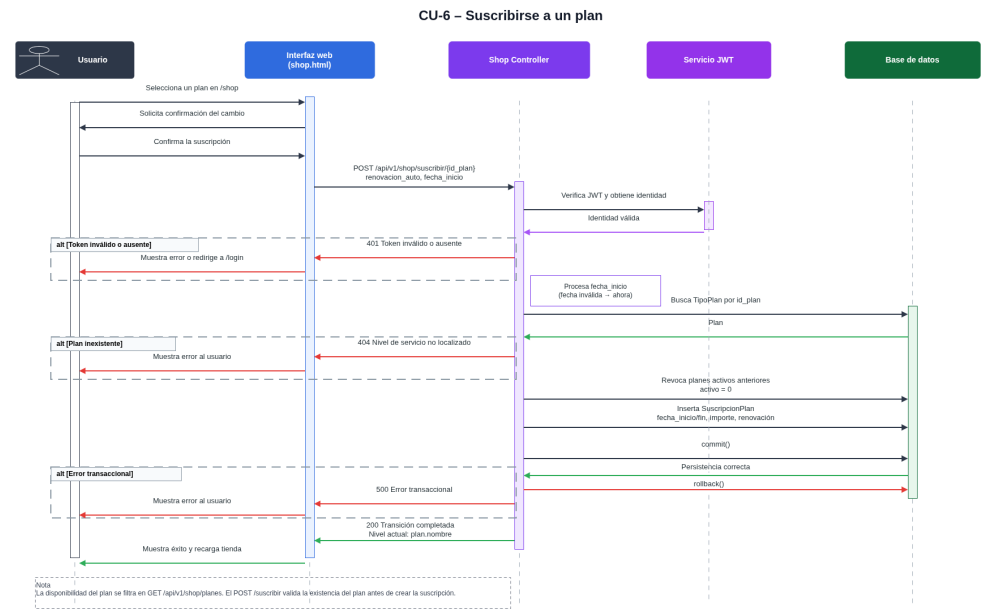


Figura C.11: Diagrama de secuencia del CU-6 Suscribirse a un plan.

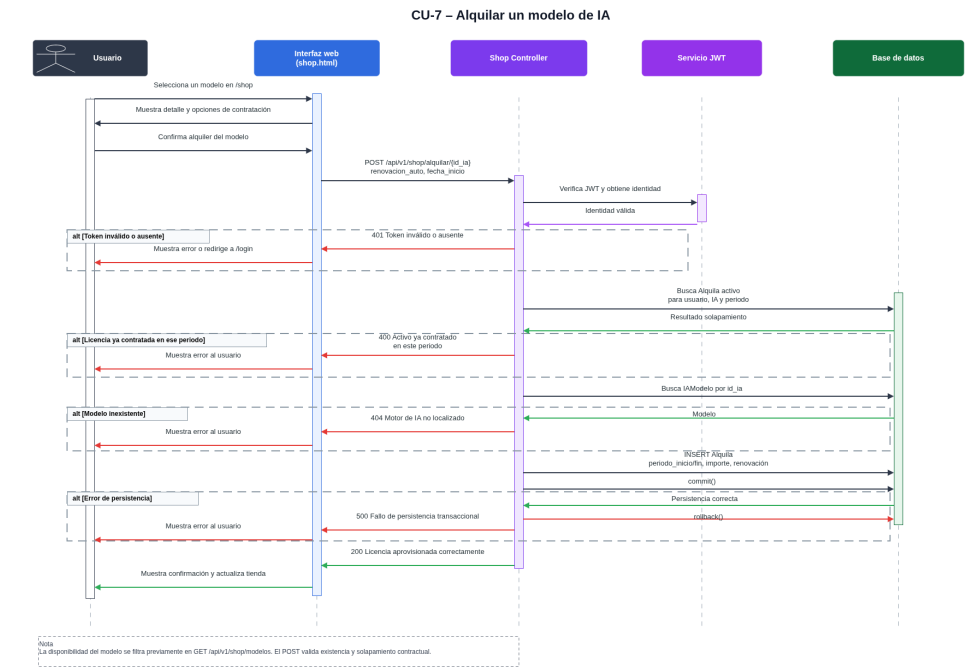


Figura C.12: Diagrama de secuencia del CU-7 Alquilar un modelo de IA.

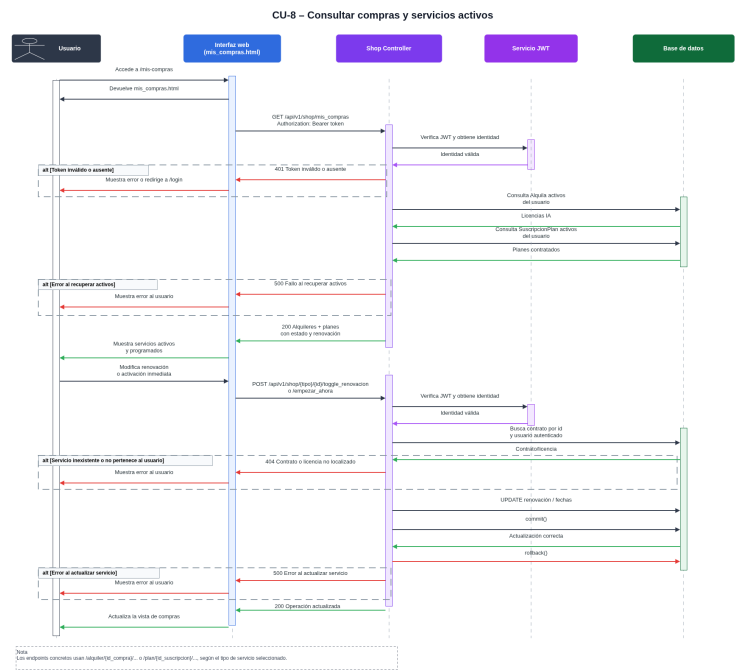


Figura C.13: Diagrama de secuencia del CU-8 Consultar compras y servicios activos.

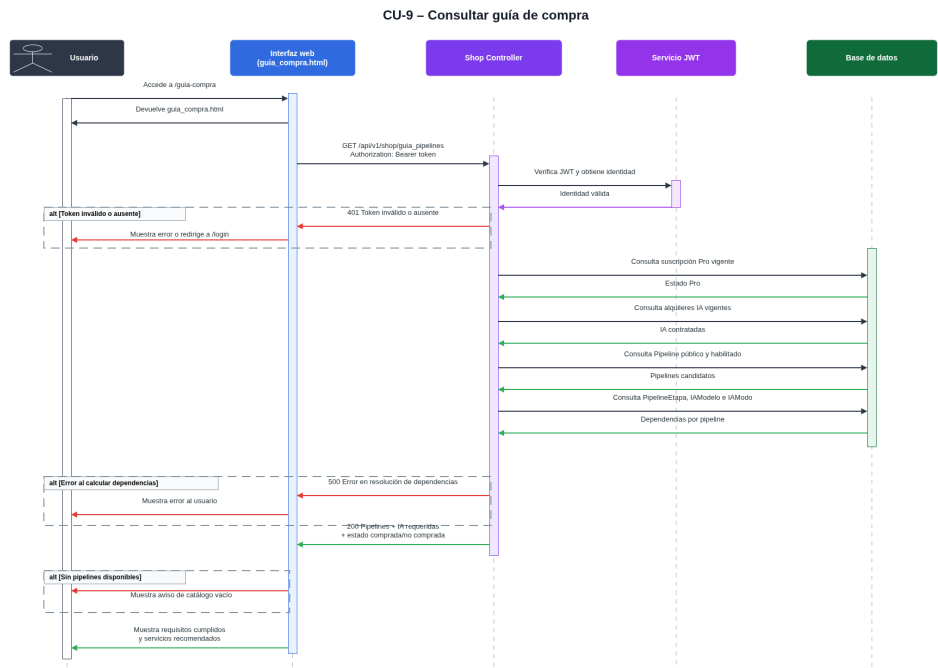


Figura C.14: Diagrama de secuencia del CU-9 Consultar guía de compra.

Flujos de consulta, configuración y ejecución de pipelines

Este bloque recoge la funcionalidad principal de la aplicación: consulta de pipelines disponibles, carga de configuración, ejecución del análisis, visualización de resultados e historial.

La Figura C.15 representa la consulta de pipelines ejecutables según los permisos, plan y modelos activos del usuario.

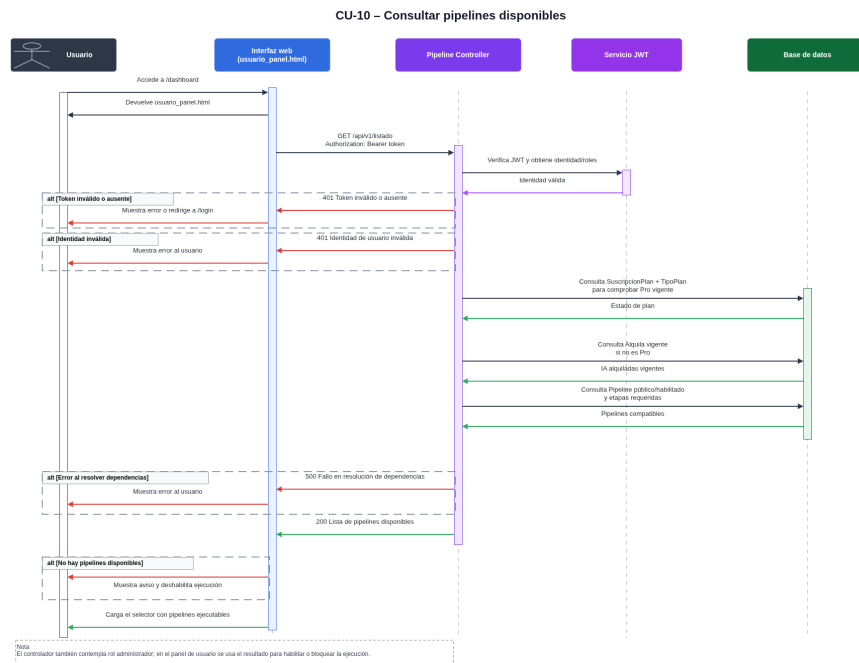


Figura C.15: Diagrama de secuencia del CU-10 Consultar pipelines disponibles.

La Figura C.16 muestra la carga y posible modificación de la configuración del pipeline antes de ejecutarlo.

La Figura C.17 representa el flujo central del sistema: subida de imagen, validación, comprobación de permisos, ejecución de etapas, registro de resultados y devolución de la respuesta.

La Figura C.18 muestra la visualización y descarga temporal de resultados mediante tokens de acceso.

La Figura C.19 recoge la consulta del historial de ejecuciones del usuario autenticado.

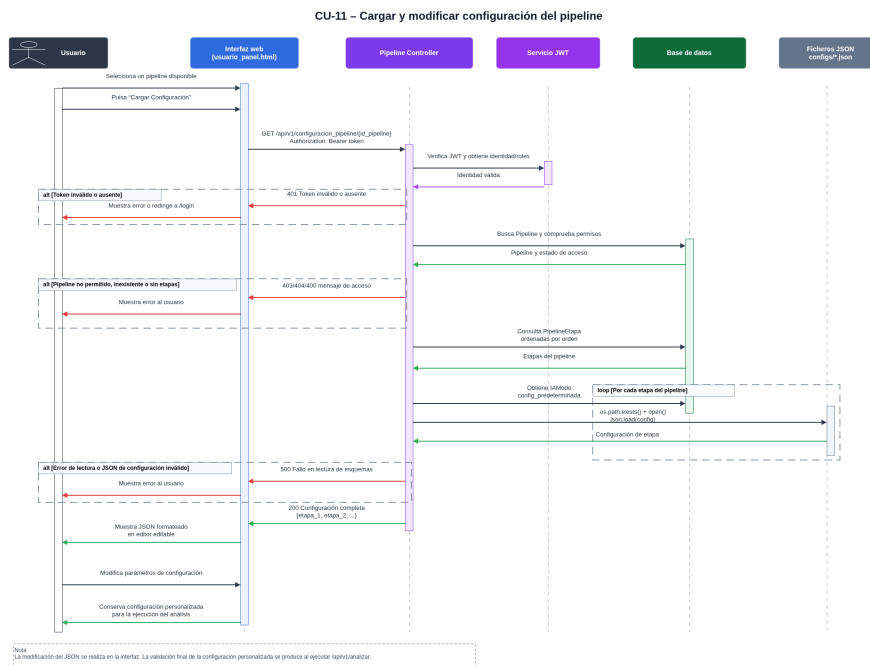


Figura C.16: Diagrama de secuencia del CU-11 Cargar y modificar configuración del pipeline.

Flujos de administración

Este bloque recoge los procedimientos restringidos al rol administrador: acceso al panel, gestión de usuarios, gestión de pipelines y consulta global de ejecuciones.

La Figura C.20 muestra la validación del rol administrador y el acceso al panel de administración.

La Figura C.21 representa la consulta de usuarios y modificación controlada de su estado.

La Figura C.22 muestra la gestión de pipelines desde administración, incluyendo creación, modificación, deshabilitación o eliminación cuando procede.

La Figura C.23 representa la consulta global de ejecuciones realizadas en el sistema.

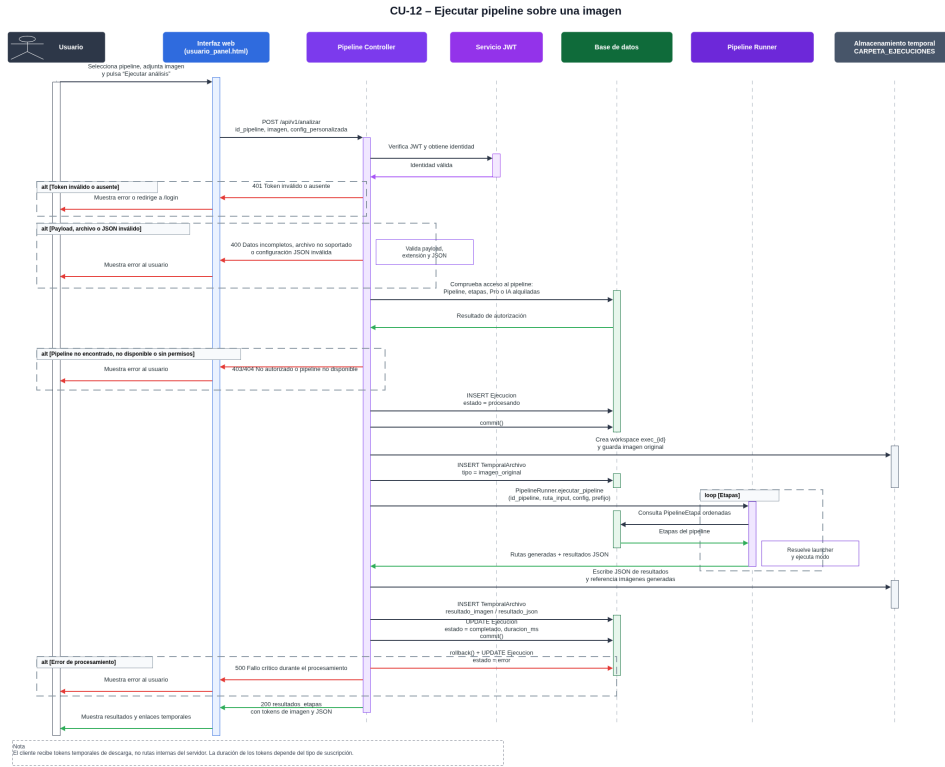


Figura C.17: Diagrama de secuencia del CU-12 Ejecutar pipeline sobre una imagen.

Flujo de mantenimiento automático

El mantenimiento automático agrupa las operaciones internas encargadas de conservar la coherencia del sistema, como la limpieza de archivos temporales y la revisión de servicios caducados.

La Figura C.24 muestra el flujo de mantenimiento, donde el sistema revisa temporales expirados, actualiza registros asociados y comprueba suscripciones o alquileres vencidos.

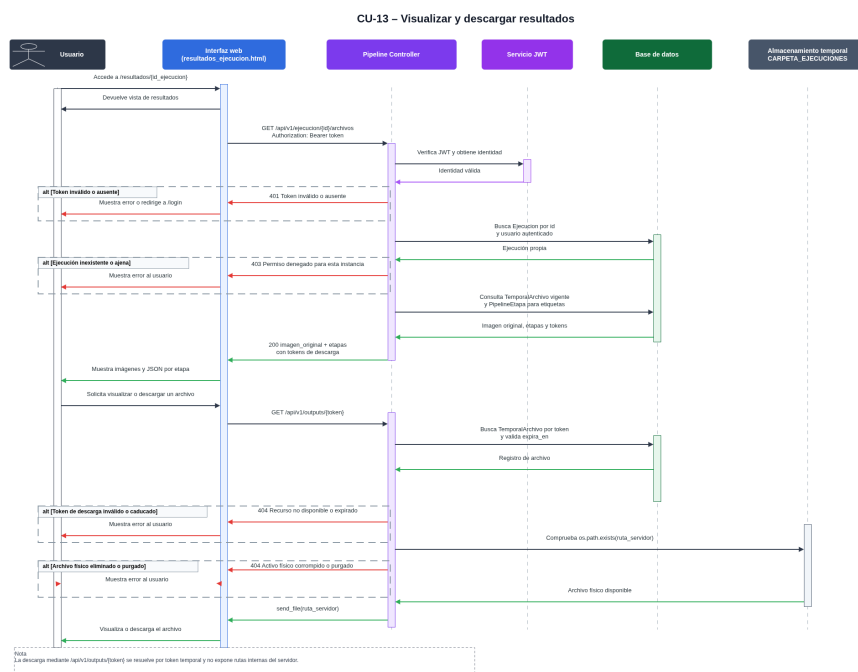


Figura C.18: Diagrama de secuencia del CU-13 Visualizar y descargar resultados.

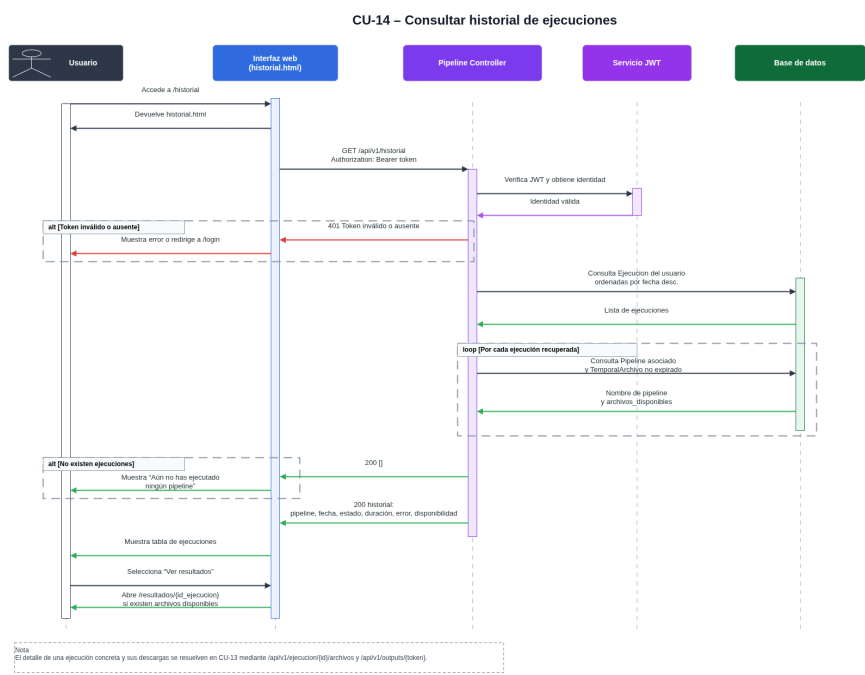


Figura C.19: Diagrama de secuencia del CU-14 Consultar historial de ejecuciones.

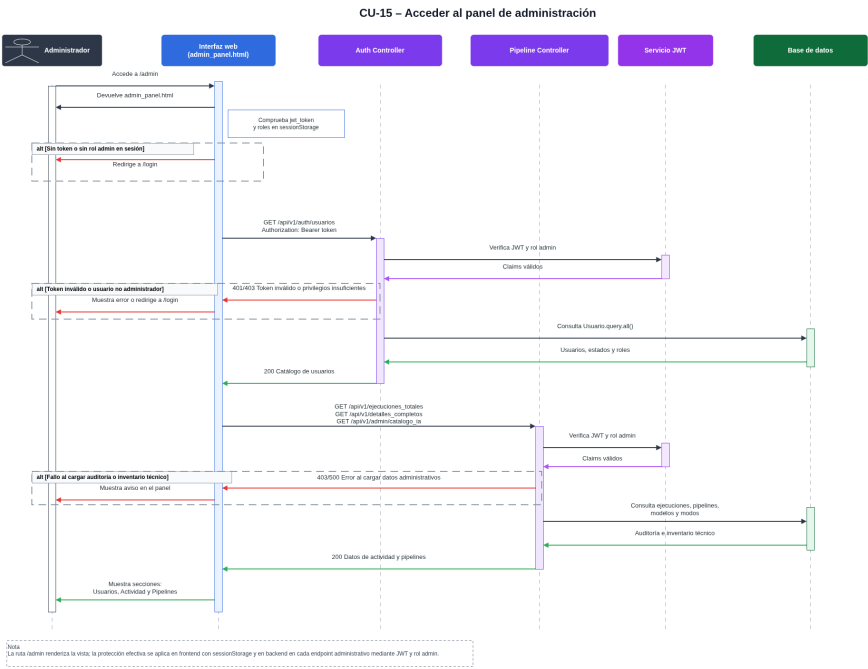


Figura C.20: Diagrama de secuencia del CU-15 Acceder al panel de administración.

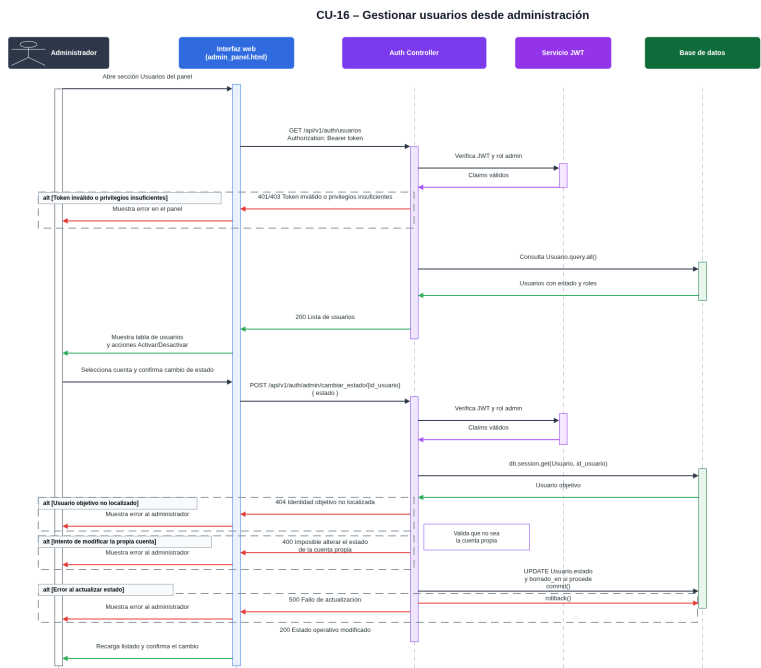


Figura C.21: Diagrama de secuencia del CU-16 Gestionar usuarios desde administración.

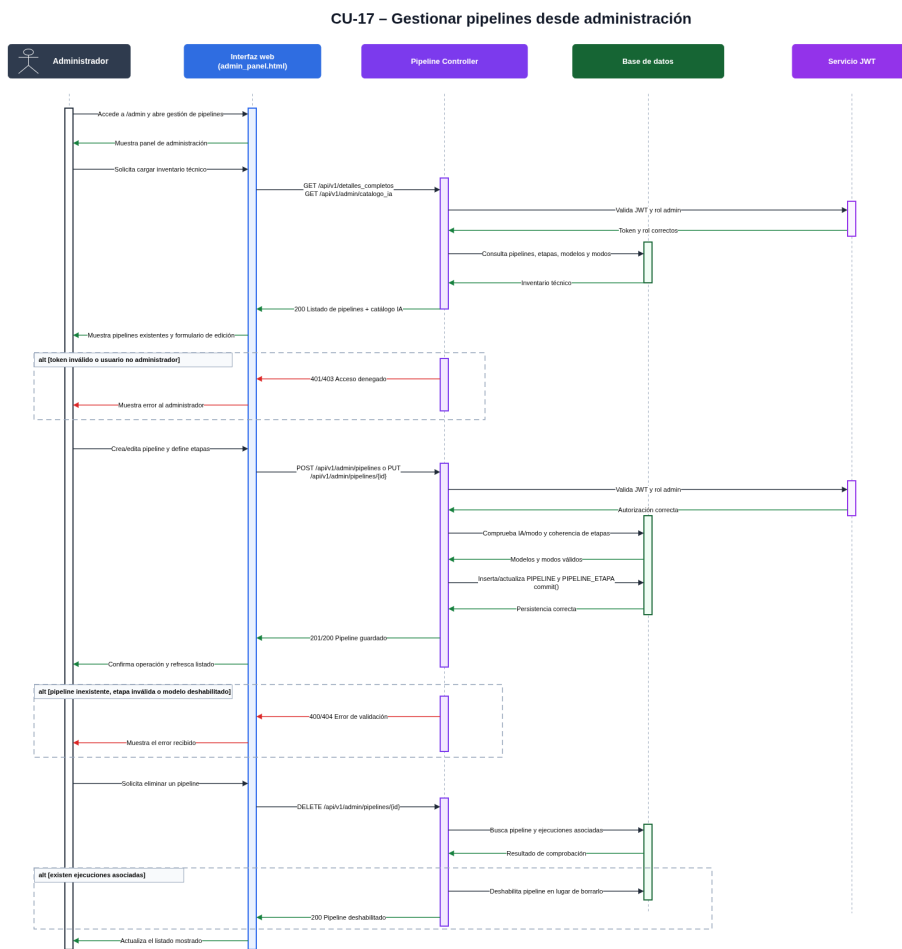


Figura C.22: Diagrama de secuencia del CU-17 Gestionar pipelines desde administración.

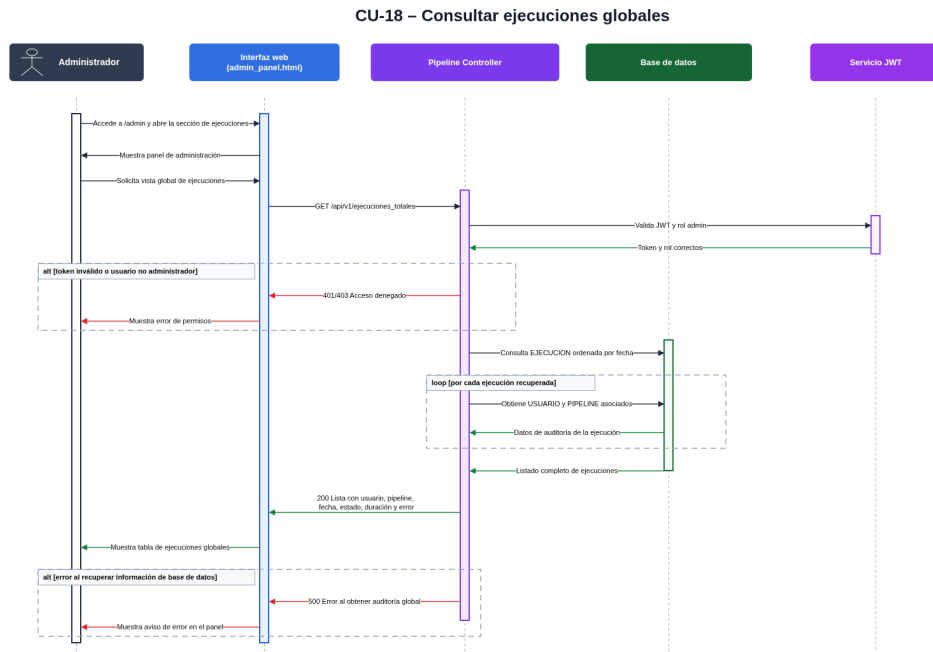


Figura C.23: Diagrama de secuencia del CU-18 Consultar ejecuciones globales.

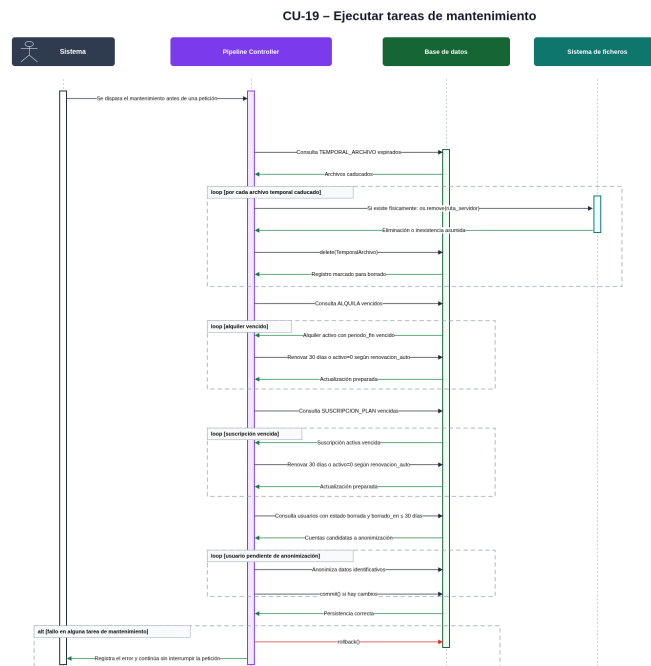


Figura C.24: Diagrama de secuencia del CU-19 Ejecutar tareas de mantenimiento.

Síntesis del diseño procedimental

Los diagramas anteriores muestran que los procedimientos principales siguen una estructura común: la interfaz recoge la acción del usuario, el controlador valida la petición y los permisos, la base de datos mantiene el estado persistente y, cuando se requiere procesamiento, los servicios internos ejecutan la lógica correspondiente.

Esta organización permite separar autenticación, gestión de servicios, ejecución de pipelines, administración y mantenimiento. Como resultado, el sistema mantiene una estructura procedimental coherente, trazable y preparada para ampliaciones futuras.

C.5. Diseño de interfaces

El diseño de interfaces recoge la organización visual y de navegación de la aplicación web. La interfaz se ha construido mediante plantillas HTML renderizadas desde Flask y lógica JavaScript para consumir los endpoints del backend.

Desde el punto de vista funcional, las pantallas se agrupan en tres zonas: zona pública, zona de usuario autenticado y zona de administración. Esta separación permite diferenciar claramente las acciones disponibles para cada tipo de usuario.

Criterios de diseño

El diseño de la interfaz se ha realizado siguiendo los siguientes criterios:

- **Claridad:** cada pantalla muestra únicamente la información necesaria para su función.
- **Separación por perfiles:** las operaciones de usuario y administración aparecen en zonas distintas.
- **Coherencia de navegación:** las páginas principales mantienen enlaces hacia las secciones relacionadas.
- **Retroalimentación:** los formularios muestran mensajes de error o confirmación.
- **Ocultación de complejidad:** el usuario no necesita conocer la estructura interna de modelos, controladores o archivos temporales.

Relación de interfaces principales

La Tabla C.13 resume las principales interfaces de la aplicación y su finalidad.

Interfaz	Ruta	Finalidad
Página de inicio	/	Presenta la aplicación y permite acceder al registro o inicio de sesión.
Registro	/registro	Permite crear una cuenta de usuario.
Inicio de sesión	/login	Permite autenticar al usuario y redirigirlo según su rol.
Panel de usuario	/dashboard	Permite consultar pipelines, cargar configuración y ejecutar análisis de imágenes.
Perfil	/perfil	Permite consultar y modificar datos de la cuenta.
Tienda	/shop	Muestra planes y modelos disponibles para contratación.
Mis compras	/mis-compras	Presenta suscripciones y alquileres activos o programados.
Guía de compra	/guia-compra	Indica qué modelos o servicios necesita el usuario para utilizar cada pipeline.
Historial	/historial	Muestra las ejecuciones realizadas por el usuario.
Resultados	/resultados/{id}	Permite consultar los resultados de una ejecución concreta.
Panel de administración	/admin	Permite gestionar usuarios, actividad global y pipelines.

Tabla C.13: Interfaces principales de la aplicación.

Estas interfaces cubren el flujo completo de uso: registro, autenticación, contratación de servicios, ejecución de pipelines, consulta de resultados e interacción administrativa.

Navegabilidad e interacción con el backend

La Figura C.25 muestra la navegación entre las zonas públicas, privadas y administrativas de la aplicación.

El acceso inicial se realiza desde la página pública. Tras iniciar sesión, el sistema redirige al usuario al panel correspondiente según su rol. Desde el

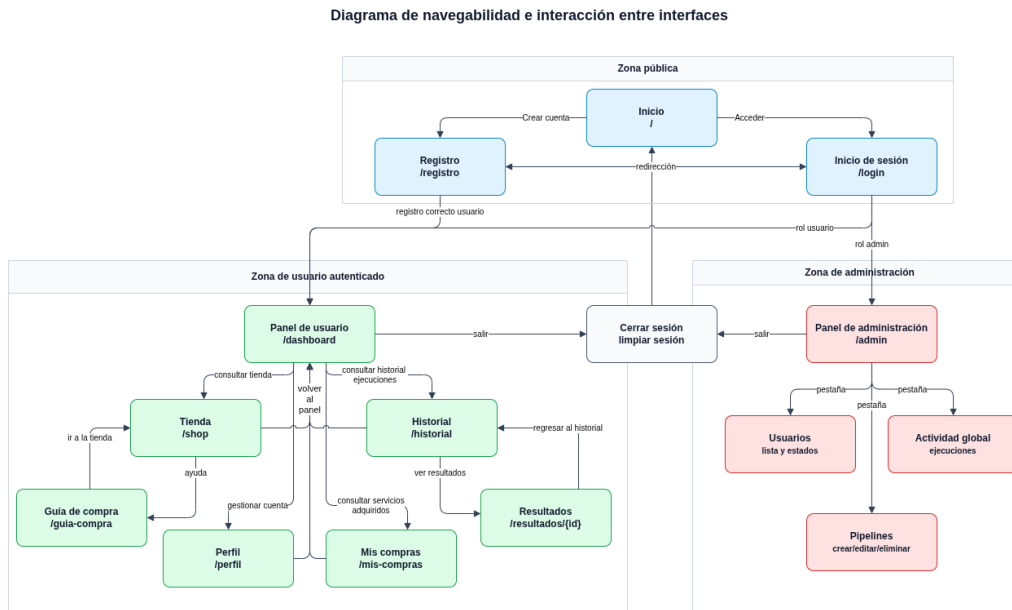


Figura C.25: Diagrama de navegabilidad e interacción entre interfaces.

panel de usuario se accede a perfil, tienda, compras, historial y ejecución de pipelines. El panel de administración concentra las funciones de supervisión y configuración.

La Figura C.26 representa la relación entre las principales interfaces, los endpoints consumidos mediante JavaScript y los controladores del backend.

Las pantallas públicas se comunican con el controlador de autenticación. Las páginas privadas envían peticiones con el token de acceso para que el backend pueda identificar al usuario y aplicar reglas de autorización. El panel principal consume endpoints de pipelines, mientras que tienda, compras y guía se relacionan con el controlador de servicios. El panel de administración utiliza endpoints restringidos.

Flujo de interfaz para la ejecución de pipelines

La Figura C.27 muestra el flujo de interacción seguido para ejecutar un pipeline desde el panel principal.

El usuario selecciona un pipeline, carga su configuración, puede modificarla, adjunta una imagen y lanza el análisis. La interfaz envía la solicitud al backend, que valida permisos, archivo y configuración. Finalmente, se

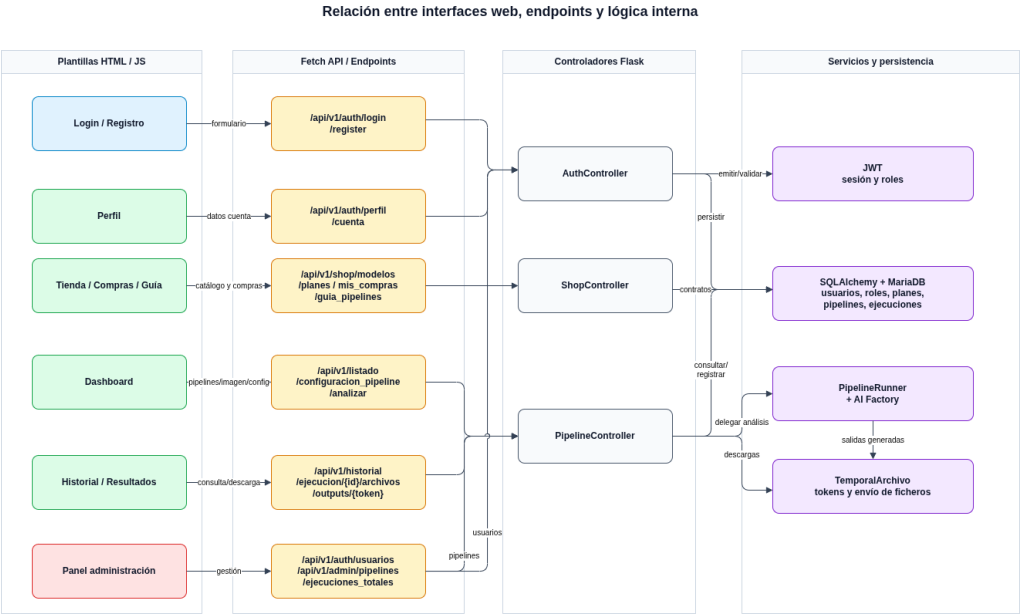


Figura C.26: Relación entre interfaces web, endpoints y lógica interna.

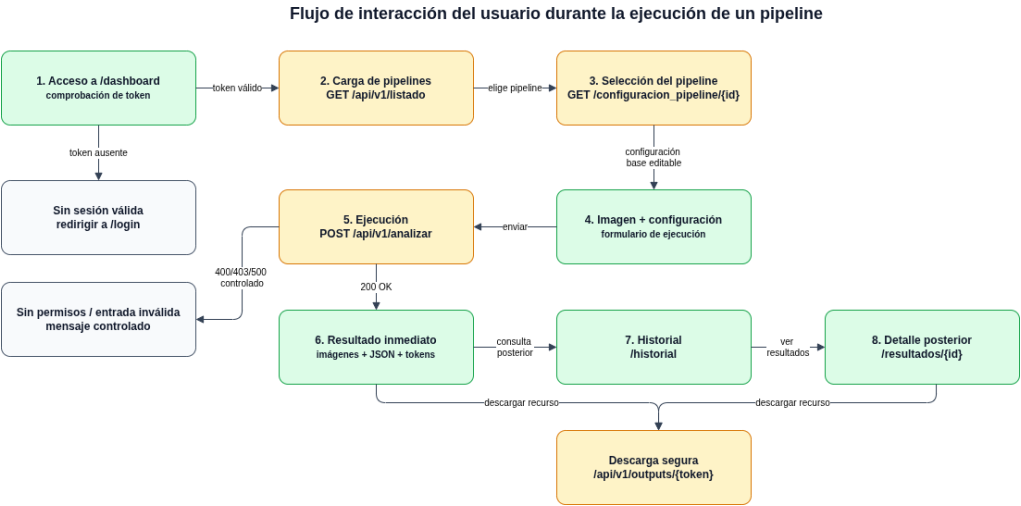


Figura C.27: Flujo de interacción de la interfaz durante la ejecución de un pipeline.

muestran las imágenes generadas, los datos JSON y los enlaces temporales de descarga.

Capturas de las interfaces funcionales iniciales

A continuación se incluyen capturas de la primera versión funcional de las interfaces. Estas pantallas no representan el diseño visual final de la aplicación, sino el estado inicial de la interfaz antes de aplicar mejoras de estilos visuales asistidas por inteligencia artificial.

El objetivo de estas capturas es dejar constancia de que la aplicación ya disponía de una interfaz operativa para validar los principales casos de uso: registro, inicio de sesión, consulta de servicios, ejecución de pipelines, visualización de resultados, historial y administración. Las interfaces finales, junto con la explicación detallada de su uso, se recogen posteriormente en el manual de usuario.

Interfaces públicas iniciales

La Figura C.28 muestra la página inicial de la primera versión funcional de la aplicación.



Figura C.28: Interfaz pública inicial de la aplicación antes del refinamiento visual.

Las Figuras C.29 y C.30 muestran las pantallas iniciales de inicio de sesión y registro.

Figura C.29: Interfaz inicial de inicio de sesión.

Figura C.30: Interfaz inicial de registro de usuario.

Interfaces iniciales del usuario autenticado

La Figura C.31 muestra el panel principal inicial del usuario autenticado, desde el que ya era posible seleccionar un pipeline, cargar configuración y enviar una imagen a procesar.

Figura C.31: Panel principal inicial del usuario para la ejecución de pipelines.

La Figura C.32 muestra la primera versión de la tienda de servicios.

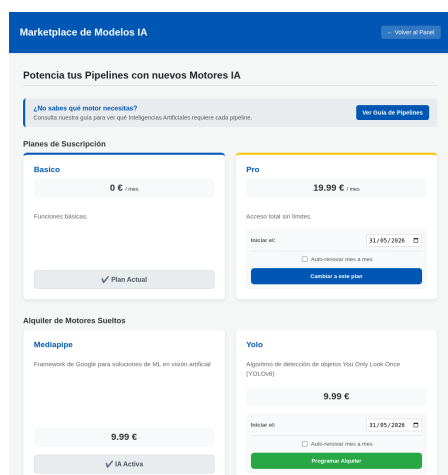


Figura C.32: Interfaz inicial de tienda de planes y modelos de IA.

La Figura C.33 muestra la guía inicial de compra, utilizada para relacionar pipelines con los modelos necesarios.

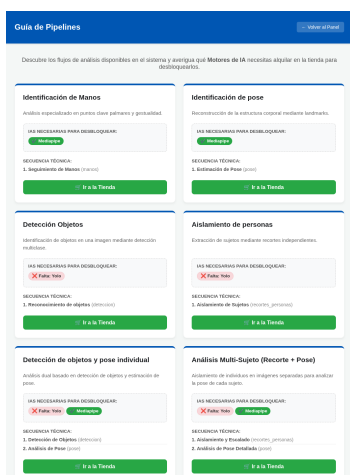


Figura C.33: Interfaz inicial de guía de pipelines y modelos necesarios.

La Figura C.34 muestra la pantalla inicial de suscripciones y alquileres del usuario.

La Figura C.35 muestra la primera versión funcional de la pantalla de perfil.

La Figura C.36 muestra el historial inicial de ejecuciones.

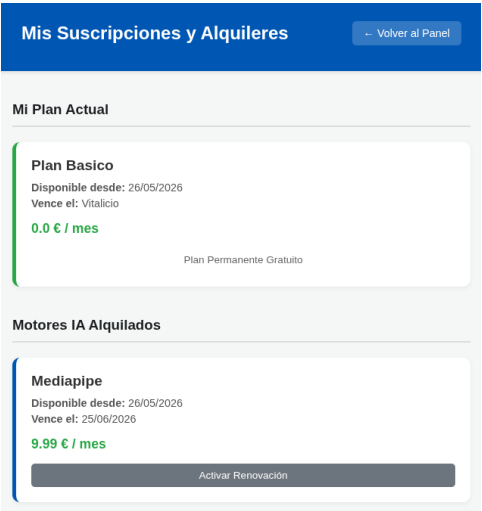


Figura C.34: Interfaz inicial de inicio de sesión.

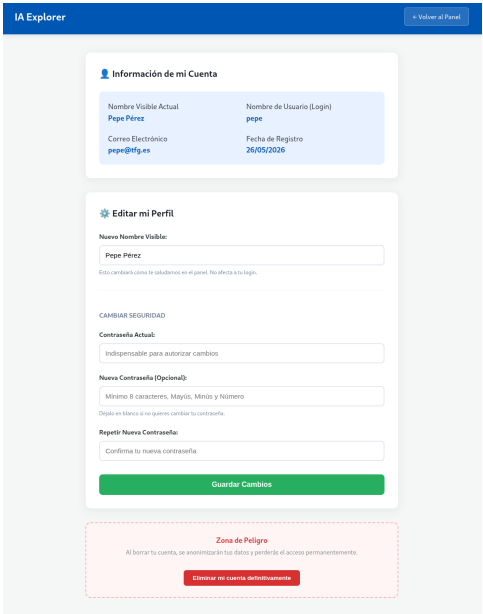


Figura C.35: Interfaz inicial de consulta y edición del perfil de usuario

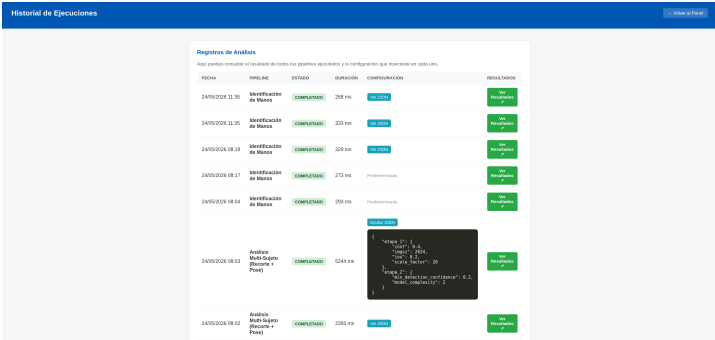


Figura C.36: Interfaz inicial de historial de ejecuciones del usuario.

La Figura C.37 muestra la primera versión de la vista de resultados de una ejecución.

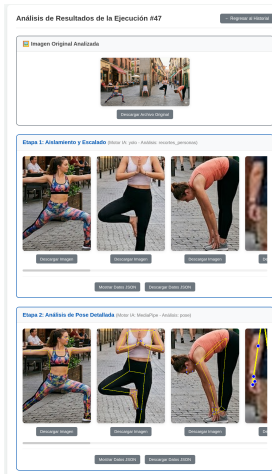


Figura C.37: Interfaz inicial de visualización de resultados de una ejecución.

Interfaces iniciales de administración

Las interfaces de administración iniciales permitían validar las operaciones restringidas al rol administrador antes del refinamiento visual.

La Figura C.38 muestra la gestión inicial de usuarios.



Figura C.38: Interfaz administrativa inicial de gestión de usuarios.

La Figura C.39 muestra la primera versión de la actividad global del sistema.

La Figura C.40 muestra la administración inicial de pipelines. Esta pantalla ya permitía validar la creación, edición, deshabilitación o eliminación de pipelines desde la aplicación.

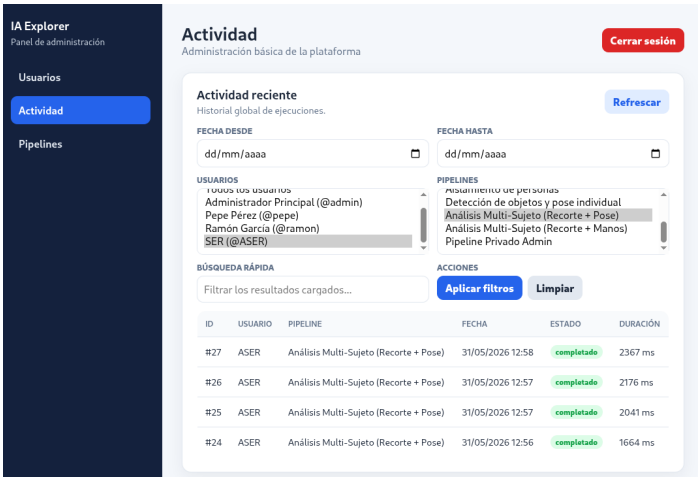


Figura C.39: Interfaz administrativa inicial de actividad global.

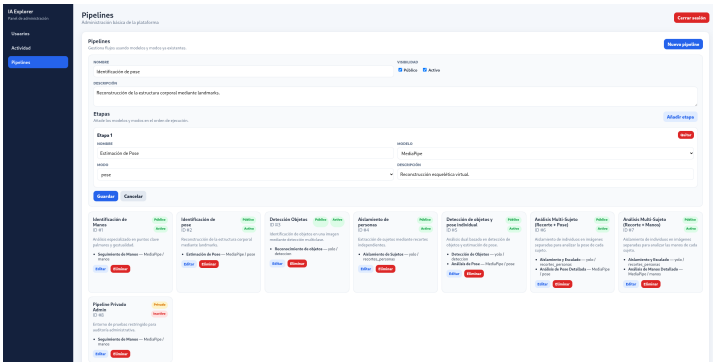


Figura C.40: Interfaz administrativa inicial de gestión de pipelines.

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este apéndice recoge la información técnica necesaria para comprender, instalar, ejecutar, probar y mantener el proyecto desde el punto de vista de un programador. A diferencia de la documentación de usuario, centrada en el uso funcional de la aplicación, aquí se describen la estructura interna del sistema, los módulos principales y las formas de preparar un entorno de ejecución o desarrollo.

La aplicación se ha desarrollado en Python utilizando Flask como framework web. La persistencia se realiza mediante SQLAlchemy y Flask-SQLAlchemy sobre MariaDB. El procesamiento de imágenes se apoya en OpenCV, MediaPipe y Ultralytics/YOLO.

El proyecto puede ejecutarse de dos formas:

- **Docker Compose:** opción recomendada para levantar el sistema completo de forma reproducible, incluyendo aplicación web y base de datos.
- **Entorno virtual de Python:** opción orientada al desarrollo local, depuración y ejecución directa de pruebas.

Desde el punto de vista del usuario final no es necesario conocer estos pasos técnicos. Una vez desplegado el sistema, el usuario accede simplemente desde un navegador web a la URL de la aplicación.

D.2. Estructura de directorios

El código fuente ejecutable se encuentra dentro del directorio `src`. A continuación se muestra la estructura principal de directorios del proyecto, sin incluir ficheros concretos:

Estructura principal de directorios

```
src/
|-- app/
|   |-- controllers/
|   |-- services/
|       |-- ai_factory/
|           |-- mediapipe_modelo/
|           |-- yolo/
|   |-- templates/
|       |-- admin/
|       |-- dashboard/
|       |-- public/
|   |-- utils/
|
|-- configs/
|-- models/
|   |-- mp/
|   |-- yolo/
|-- execution_data/
|-- tests/
|-- .github/
|   |-- workflows/
```

La finalidad de cada directorio se resume en la Tabla [D.1](#).

Tabla D.1: Directorios principales del proyecto.

Directorio	Finalidad
<code>app/</code>	Código principal de la aplicación Flask.
<code>app/controllers/</code>	Controladores HTTP: autenticación, tienda, usuario, administración y ejecución de pipelines.
<code>app/services/</code>	Lógica interna de la aplicación, especialmente ejecución de pipelines y procesamiento de IA.
<code>app/services/ai_factory/</code>	Factoría de modelos de IA e interfaz común para los lanzadores.
<code>app/services/ai_factory/mediapipe_modelo/</code>	Implementación de los modos asociados a MediaPipe.

Directorio	Finalidad
<code>app/services/ai_factory/yolo/</code>	Implementación de los modos asociados a YOLO.
<code>app/templates/</code>	Plantillas HTML de la interfaz web.
<code>app/templates/admin/</code>	Vistas del panel de administración.
<code>app/templates/dashboard/</code>	Vistas del usuario autenticado.
<code>app/templates/public/</code>	Vistas públicas: inicio, registro e inicio de sesión.
<code>app/utils/</code>	Utilidades auxiliares reutilizables.
<code>configs/</code>	Configuraciones predeterminadas de los modos de ejecución.
<code>models/</code>	Recursos locales de modelos de inteligencia artificial.
<code>models/mp/</code>	Modelos locales utilizados por MediaPipe.
<code>models/yolo/</code>	Recursos asociados a YOLO.
<code>execution_data/</code>	Archivos generados durante las ejecuciones: entradas, salidas y resultados temporales.
<code>tests/</code>	Pruebas automatizadas del proyecto.
<code>.github/workflows/</code>	Flujos de integración continua y análisis automático.

Además de estos directorios, la carpeta `src` contiene los ficheros de arranque, configuración, dependencias, despliegue, análisis estático y cobertura, descritos en los apartados correspondientes.

D.3. Manual del programador

Esta sección describe el funcionamiento interno de la aplicación desde el punto de vista de un desarrollador que necesite mantener o ampliar el sistema.

La arquitectura se basa en Flask, SQLAlchemy y una capa propia de servicios. Las plantillas HTML forman la interfaz, los controladores gestionan las peticiones HTTP, los modelos representan la persistencia y los servicios concentran la lógica de ejecución. La escalabilidad del proyecto se apoya especialmente en tres elementos: la base de datos, el orquestador de pipelines y la factoría de modelos de inteligencia artificial.

Flujo interno de ejecución

El flujo principal comienza cuando un usuario selecciona un pipeline, adjunta una imagen y solicita el análisis. El controlador recibe la petición, valida los datos, comprueba permisos y delega la ejecución en `PipelineRunner`.

El controlador principal se encuentra en:

Controlador principal

```
app/controllers/pipeline_controller.py
```

La ruta principal de análisis es:

Endpoint principal de análisis

```
/api/v1/analizar
```

El proceso seguido por esta ruta puede resumirse así:

1. Recibe el identificador del pipeline, la imagen y la configuración opcional.
2. Valida el archivo recibido y el JSON de configuración.
3. Comprueba que el usuario tiene acceso al pipeline.
4. Crea un registro en `EJECUCION`.
5. Guarda la imagen original en `execution_data/`.
6. Ejecuta el pipeline mediante `PipelineRunner`.
7. Registra los archivos generados en `TEMPORAL_ARCHIVO`.
8. Devuelve tokens de descarga, evitando exponer rutas internas del servidor.

El orquestador se encuentra en:

Orquestador de pipelines

```
app/services/pipeline_runner.py
```

Su método principal es:

Método principal del orquestador

```
PipelineRunner.ejecutar_pipeline(  
    id_pipeline,  
    imagen_inicial,  
    config_completa=None,  
    prefijo=""  
) -> tuple[list[str], list[dict]]:
```

Este método consulta las etapas del pipeline, las ordena por el campo **orden**, obtiene el modelo y modo de cada etapa, resuelve el lanzador correspondiente mediante **AIFactory** y ejecuta cada modo sobre la imagen o imágenes de entrada.

El resultado devuelto tiene la forma:

Salida del orquestador

```
(rutas_finales, resultados)
```

donde **rutas_finales** contiene las imágenes generadas y **resultados** contiene los datos estructurados de cada etapa. El sistema también permite salidas múltiples, por ejemplo cuando una etapa genera varios recortes y una etapa posterior procesa cada uno de ellos.

Factoría de modelos de inteligencia artificial

La factoría se encuentra en:

Factoría de modelos de IA

```
app/services/ai_factory/__init__.py
```

Su responsabilidad es devolver el lanzador adecuado según el nombre del modelo:

Resolución de lanzador

```
AIFactory.get_launcher(nombre_modelo)
```

Actualmente resuelve dos familias principales:

- **yolo**: mediante **YoloLauncher**.
- **mediapipe**: mediante **MediaPipeLauncher**.

Todos los lanzadores cumplen la interfaz común definida en:

Interfaz común de lanzadores

`app/services/ai_factory/interface.py`

El método obligatorio es:

Contrato común de ejecución

```
def ejecutar_modos(
    self,
    modo_nombre: str,
    imagen_path: str,
    config: Dict[str, Any] = None,
    prefijo: str = ""
) -> Tuple[Union[str, List[str]], Dict[str, Any]]:
```

Su salida esperada es:

Salida esperada de un launcher

`(rutas_generadas, datos_json)`

El primer elemento puede ser una ruta o una lista de rutas. El segundo debe ser un diccionario con los resultados estructurados del análisis.

Launchers existentes

Las implementaciones actuales se resumen en la Tabla [D.2](#).

Modelo	Launcher	Modos principales
YOLO	<code>app/services/ai_factory/yolo/launcher.py</code>	deteccion, recortes_personas.
MediaPipe	<code>app/services/ai_factory/mediapipe_modelo/launcher.py</code>	manos, pose.

Tabla D.2: Launchers existentes para los modelos de inteligencia artificial.

En YOLO, los modos se resuelven mediante el diccionario interno `_task_map`. En MediaPipe se utiliza el diccionario `_estrategias`. En ambos casos, la clave del diccionario debe coincidir con el nombre técnico del modo registrado en la base de datos.

Configuración por etapas

Cada modo puede tener una configuración predeterminada almacenada en formato JSON dentro del directorio `configs/`. La ruta de dicha configuración se guarda en:

Campo de configuración predeterminada

```
IA_MOD0.config_predeterminada
```

Cuando el usuario solicita la configuración de un pipeline, el endpoint:

Endpoint de configuración de pipeline

```
/api/v1/configuracion_pipeline/<id_pipeline>
```

devuelve la configuración agrupada por etapas:

Formato de configuración por etapas

```
{
  "etapa_1": { ... },
  "etapa_2": { ... }
}
```

Durante la ejecución, `PipelineRunner` obtiene la configuración concreta de cada etapa mediante:

Obtención de configuración por etapa

```
config_completa.get(f"etapa_{etapa.orden}", {})
```

Por tanto, si se añade un nuevo modo, sus parámetros deben estar definidos en un fichero JSON y ser interpretados por la función Python correspondiente.

Relación con la base de datos

La ampliación del sistema no depende solo del código. Para que un modelo, modo o pipeline pueda utilizarse desde la aplicación, también debe estar registrado en la base de datos.

Las entidades principales implicadas son:

- `IA_MODELO`: familias de modelos disponibles.

- IA_MODO: modos asociados a cada modelo.
- PIPELINE: flujos de procesamiento.
- PIPELINE_ETAPA: etapas concretas de cada pipeline.

La tabla PIPELINE_ETAPA incluye `id_ia` e `id_modo`. La clave foránea compuesta con `IA_MODO` evita que una etapa asocie un modo a un modelo al que no pertenece.

Para que una etapa sea válida deben coincidir el modelo, el modo, la relación `id_ia-id_modo`, el nombre esperado por el lanzador y el fichero de configuración asociado.

Extensión del sistema con nuevos modelos y modos

La extensibilidad del sistema se apoya en la separación entre el orquestador, la factoría, los lanzadores y la información persistida en base de datos. La Figura D.1 resume los puntos principales de extensión.

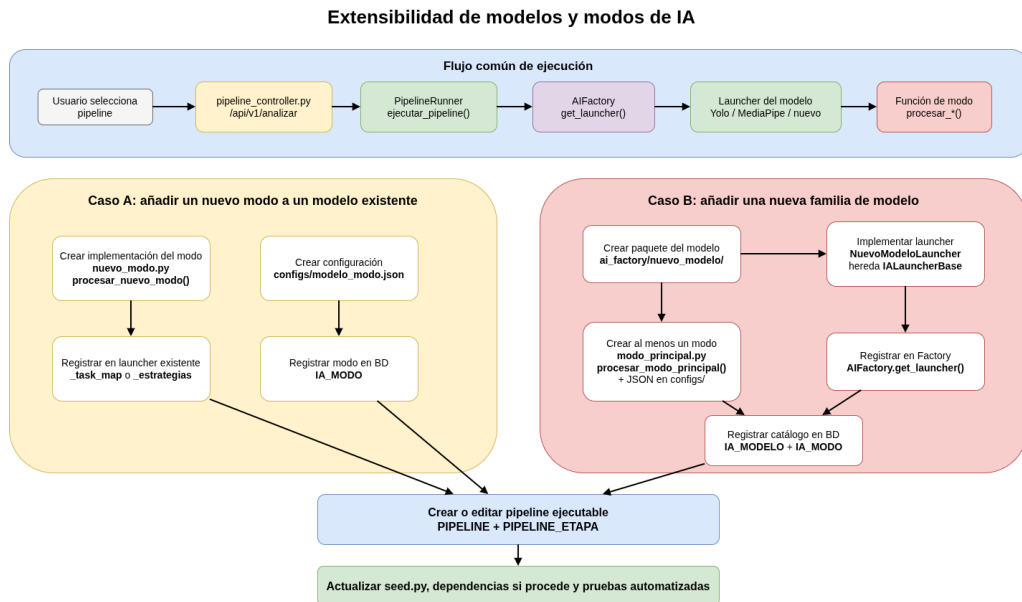


Figura D.1: Flujo arquitectónico para extender el sistema con nuevos modos y modelos de inteligencia artificial.

Añadir un nuevo modo a un modelo existente

Añadir un nuevo modo implica incorporar una operación nueva a una familia ya soportada, como YOLO o MediaPipe. No requiere modificar `AIFactory`; el cambio se concentra en el lanzador del modelo, la función de procesamiento, la configuración y la base de datos.

El procedimiento mínimo es:

1. Crear un fichero JSON de configuración en `configs/`.
2. Crear el archivo Python del nuevo modo dentro de la carpeta del modelo.
3. Implementar una función con el contrato común:

Contrato de un nuevo modo

```
def ejecutar_modoo(
    self,
    modo_nombre: str,
    imagen_path: str,
    config: Dict[str, Any] = None,
    prefijo: str = ""
) -> Tuple[Union[str, List[str]], Dict[str, Any]]:
```

A continuación se completa el registro del modo:

4. Registrar la función en el lanzador correspondiente.
5. Añadir el modo en `IA_MOD0`.
6. Añadirlo en `seed.py` si debe formar parte del entorno inicial.
7. Crear o actualizar pipelines desde administración o desde `seed.py`.
8. Añadir pruebas automatizadas.

El nombre usado como clave en el lanzador debe coincidir con `IA_MOD0.nombre_modoo`. Se recomienda utilizar nombres en minúsculas, sin espacios y sin tildes.

Añadir un nuevo modelo de inteligencia artificial

Añadir una nueva familia de modelos requiere crear una nueva carpeta dentro de la factoría, implementar un lanzador propio, registrarlo en `AIFactory` y añadir el modelo y sus modos a la base de datos.

El procedimiento mínimo es:

1. Revisar dependencias Python y librerías del sistema.
2. Actualizar `requirements.txt`, `requirements.lock` y, si procede, `Dockerfile`.
3. Crear una carpeta en `app/services/ai_factory/`.
4. Implementar un `launcher.py` que herede de `IALauncherBase`.
5. Implementar los modos de procesamiento.
6. Registrar el nuevo lanzador en `AIFactory`.
7. Crear los ficheros JSON de configuración.
8. Añadir el modelo en `IA_MODELO`.
9. Añadir sus modos en `IA_MODALO`.
10. Crear pipelines asociados desde administración o desde `seed.py`.
11. Añadir pruebas de factoría, lanzador, modo y pipeline.

En condiciones normales no debería ser necesario modificar nada en `pipeline_controller.py`, `PipelineRunner` ni las tablas principales del modelo relacional. La ampliación debe concentrarse en los nuevos módulos de procesamiento, el lanzador, la factoría, la configuración, los registros de base de datos y las pruebas.

Gestión de archivos generados

Los modos de procesamiento guardan sus salidas como archivos físicos. El usuario no recibe rutas internas, sino tokens asociados a registros en `TEMPORAL_ARCHIVO`.

Las funciones de procesamiento deben guardar los resultados en el directorio de trabajo recibido en `imagen_path`. El nombre del archivo debe

incorporar el `prefijo` cuando sea necesario mantener trazabilidad entre etapas.

Ejemplos de nombres utilizados en el proyecto son:

- `deteccion_<nombre_archivo>` para detecciones YOLO.
- `recorte_<indice>_<nombre_archivo>` para recortes de personas.
- `mp_<nombre_archivo>` para detección de manos.
- `mp_pose_<nombre_archivo>` para estimación de pose.

Cuando se utilice OpenCV, debe comprobarse explícitamente el resultado de `cv2.imwrite()`, ya que puede devolver `False` si no consigue guardar el archivo.

Reglas de sincronización

Para que una ampliación funcione correctamente deben coincidir los elementos indicados en la Tabla D.3.

Elemento	Debe coincidir con
Nuevo modo	<code>IA_MOD0.nombre_mod0</code> , clave del lanzador, función Python, fichero JSON y etapas que lo usan.
Nuevo modelo	<code>IA_MODELO.nombre</code> , condición en <code>AIFactory</code> , carpeta del modelo, lanzador, modos, configuraciones y dependencias.

Tabla D.3: Reglas de sincronización para nuevos modelos y modos.

Si alguno de estos puntos no está sincronizado, pueden aparecer modelos visibles pero no ejecutables, modos implementados pero no disponibles en la interfaz o pipelines con etapas incoherentes.

Resumen operativo para extender el sistema

La Tabla D.4 resume las acciones mínimas necesarias para ampliar el sistema.

Ampliación	Acciones principales
Nuevo modo	Crear JSON, implementar función, registrarla en el lanzador, añadir el modo en base de datos, actualizar <code>seed.py</code> si procede, crear pipelines y añadir pruebas.
Nuevo modelo	Revisar dependencias, crear carpeta, implementar lanzador, registrar en <code>AIFactory</code> , crear modos y JSON, añadir registros en base de datos, crear pipelines y añadir pruebas.

Tabla D.4: Resumen operativo para extender el sistema.

Con esta organización, la aplicación puede crecer sin reescribir el flujo general de ejecución. Los pipelines se definen como datos persistentes y el código mantiene una lógica común basada en factoría, lanzadores y modos especializados.

D.4. Compilación, instalación y ejecución del proyecto

Al tratarse de una aplicación Python, el proyecto no requiere una fase de compilación tradicional. No obstante, sí es necesario preparar el entorno, configurar variables, inicializar la base de datos y arrancar la aplicación.

En esta sección se documentan las dos formas de ejecución del proyecto: Docker Compose y entorno virtual de Python.

Requisitos previos

Los requisitos dependen del modo de ejecución elegido. La Tabla [D.5](#) resume las herramientas necesarias.

Modo	Requisitos
Docker Compose	Git, Docker, Docker Compose y conexión a Internet durante la primera construcción.
Entorno virtual	Linux, Git, Python 3.11, <code>venv</code> , <code>pip</code> , MariaDB y librerías del sistema necesarias para visión por computador.

Tabla D.5: Requisitos según el modo de ejecución.

En sistemas Debian o Ubuntu pueden instalarse las herramientas principales para desarrollo local con:

Instalación de herramientas en Debian/Ubuntu

```
sudo apt update
sudo apt install -y git python3.11 python3.11-venv python3-pip
sudo apt install -y mariadb-server
sudo apt install -y build-essential ffmpeg libgl1 libglib2.0-0
libgomp1 libsm6 libxext6 libxrender1
```

MariaDB puede activarse con:

Activación de MariaDB

```
sudo systemctl enable mariadb
sudo systemctl start mariadb
sudo systemctl status mariadb
```

Obtención del proyecto

El proyecto se obtiene desde GitHub:

Clonado del repositorio

```
git clone https://github.com/alvaroooper/TFG.git
cd TFG/src
```

Todos los comandos deben ejecutarse desde **src**, ya que en esta carpeta se encuentran la aplicación, la configuración, los ficheros Docker, el script de inicialización y las pruebas.

Ejecución mediante Docker Compose

La ejecución mediante Docker Compose es la opción recomendada para probar el sistema completo de forma reproducible, ya que levanta conjuntamente la aplicación web y la base de datos.

Esta forma de ejecución es válida tanto en sistemas Linux como en Windows, siempre que Docker y Docker Compose estén disponibles. En Windows se recomienda utilizar Docker Desktop y ejecutar los comandos desde PowerShell, Git Bash o una terminal equivalente. No es necesario instalar Python, MariaDB ni las dependencias internas del proyecto en el equipo anfitrión, ya que estos componentes se ejecutan dentro de contenedores.

Configuración del archivo .env

Si existe `.env.example`, se copia para crear el fichero de configuración local. En Linux, Git Bash o PowerShell puede utilizarse:

Creación del fichero de entorno

```
cp .env.example .env
```

En caso de utilizar la terminal clásica de Windows, el comando equivalente sería:

Creación del fichero de entorno en cmd

```
copy .env.example .env
```

Una configuración de ejemplo para Docker Compose es:

Ejemplo de configuración .env para Docker Compose

```
SECRET_KEY=CAMBIAR_CLAVE_FLASK
JWT_SECRET_KEY=CAMBIAR_CLAVE_JWT

MARIADB_ROOT_PASSWORD=CAMBIAR_PASSWORD_ROOT
MARIADB_DATABASE=ia_orquestada_db
MARIADB_USER=usuarioIAOrquestadaDB
MARIADB_PASSWORD=CAMBIAR_PASSWORD_USUARIO

DATABASE_URL=mysql+pymysql://usuarioIAOrquestadaDB:
    CAMBIAR_PASSWORD_USUARIO@db:3306/ia_orquestada_db?charset=utf8mb4
```

Los valores anteriores son de ejemplo. En un entorno real deben sustituirse por claves y contraseñas propias. Dentro de Docker, el host de la base de datos es `db`, no `localhost`.

Arranque inicial

Antes de arrancar los servicios puede validarse la configuración:

Validación de Docker Compose

```
docker compose config
```

El primer arranque puede realizarse en tres pasos:

Primer arranque del sistema

```
# 1. Construir y arrancar la base de datos
docker compose up -d --build db

# 2. Inicializar esquema y datos de demostración
docker compose run --rm web python seed.py

# 3. Arrancar la aplicación web
docker compose up -d web
```

Acceso desde el equipo y otros equipos de la red local

Por defecto, cuando la aplicación se ejecuta en Docker y el puerto está publicado correctamente, puede accederse desde el propio equipo anfitrión mediante:

Acceso desde el equipo anfitrión

```
http://localhost:5000
```

Sin embargo, si se desea acceder desde otro equipo conectado a la misma red local, no debe utilizarse `localhost`, ya que ese nombre siempre hace referencia al propio equipo desde el que se abre el navegador. En su lugar, debe utilizarse la dirección IP del equipo donde se está ejecutando Docker.

Para que el acceso desde otros equipos funcione, deben cumplirse tres condiciones:

- La aplicación debe escuchar en `0.0.0.0:5000`, permitiendo conexiones externas al contenedor.
- El servicio web debe publicar el puerto mediante Docker Compose.
- El firewall del equipo anfitrión debe permitir conexiones entrantes al puerto 5000.

En el fichero `docker-compose.yml`, el servicio web debe incluir una asignación de puertos como la siguiente:

Publicación del puerto en Docker Compose

```
ports:
- "5000:5000"
```

Esta configuración indica que el puerto 5000 del equipo anfitrión se redirige al puerto 5000 del contenedor. Si se utilizase una configuración limitada a 127.0.0.1, el acceso quedaría restringido al propio equipo.

Para conocer la dirección IP del equipo anfitrión en Linux puede ejecutarse:

Consulta de IP en Linux

```
hostname -I
```

También puede consultarse con:

Consulta detallada de interfaces en Linux

```
ip addr
```

En Windows, la dirección IP puede obtenerse desde PowerShell o símbolo del sistema mediante:

Consulta de IP en Windows

```
ipconfig
```

Una vez conocida la dirección IP del equipo donde se ejecuta Docker, desde otro equipo de la misma red se accede sustituyendo `localhost` por dicha dirección. Por ejemplo:

Acceso desde otro equipo de la red local

```
http://192.168.1.27:5000
```

Si el sistema operativo bloquea las conexiones entrantes, será necesario permitir el puerto 5000 en el firewall. En sistemas Linux con `ufw`, puede hacerse con:

Apertura del puerto 5000 en Linux

```
sudo ufw allow 5000/tcp
```

En Windows, debe permitirse el acceso a Docker Desktop o crear una regla de entrada para el puerto 5000 en el Firewall de Windows.

Esta configuración está pensada para pruebas dentro de una red local. Para exponer la aplicación a Internet sería necesario un despliegue más completo, con servidor público, configuración de red, dominio, HTTPS y, preferiblemente, un proxy inverso. Por ese motivo, en este proyecto el acceso

externo se contempla únicamente para entornos de demostración o pruebas dentro de la misma red.

Uso habitual

Una vez inicializada la base de datos, los comandos más frecuentes se resumen en el siguiente bloque:

Comandos habituales de Docker Compose

```
# Arrancar todos los servicios
docker compose up -d

# Detener la aplicación y la base de datos
docker compose down

# Consultar logs de la aplicación
docker compose logs -f web

# Consultar logs de la base de datos
docker compose logs -f db

# Reiniciar únicamente la aplicación web
docker compose restart web
```

Reinicialización

Si se desea volver a cargar los datos iniciales de demostración, puede ejecutarse:

Recarga de datos iniciales

```
docker compose run --rm web python seed.py
```

Este comando debe usarse con precaución, ya que reinicializa la base de datos.

Para eliminar el volumen de MariaDB y partir desde cero:

Reinicio completo de la base de datos

```
docker compose down -v
docker compose up -d --build db
docker compose run --rm web python seed.py
docker compose up -d web
```

Consulta de la base de datos

Si se desea consultar la base de datos desde DBeaver u otro cliente compatible con MariaDB, se puede utilizar una conexión como la siguiente:

- **Tipo:** MariaDB.
- **Host:** localhost.
- **Puerto:** 3307.
- **Base de datos:** ia_orquestada_db.
- **Usuario:** usuarioIAOrquestadaDB.
- **Contraseña:** la indicada en MARIADB_PASSWORD.

Desde la aplicación se usa db:3306; desde el equipo anfitrión se utiliza el puerto expuesto por Docker Compose.

Ejecución mediante entorno virtual de Python

La ejecución mediante entorno virtual está orientada a desarrollo, depuración y ejecución directa de pruebas para evitar conflictos se recomienda utilizarlo en Debian.

Creación del entorno virtual

Desde src:

Creación del entorno virtual

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Configuración de MariaDB local

Se accede a MariaDB:

Acceso a MariaDB

```
sudo mariadb
```

Se crea la base de datos y el usuario:

Creación de base de datos y usuario

```
CREATE DATABASE ia_orquestada_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

CREATE USER 'usuarioIAOrquestadaDB'@'localhost'
IDENTIFIED BY 'CAMBIAR_PASSWORD_USUARIO';

GRANT ALL PRIVILEGES ON ia_orquestada_db.*
TO 'usuarioIAOrquestadaDB'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

Configuración del archivo .env

Para ejecución local, DATABASE_URL debe apuntar a localhost:

Ejemplo de configuración .env para entorno local

```
SECRET_KEY=CAMBIAR_CLAVE_FLASK
JWT_SECRET_KEY=CAMBIAR_CLAVE_JWT

DATABASE_URL=mysql+pymysql://usuarioIAOrquestadaDB:
CAMBIAR_PASSWORD_USUARIO@localhost:3306/ia_orquestada_db?charset=
utf8mb4
```

Inicialización y ejecución

Con el entorno virtual activado:

Inicialización y ejecución local

```
python seed.py
python run.py
```

La aplicación queda disponible en:

URL local de acceso

```
http://localhost:5000
```

Uso de la base de datos Docker desde entorno local

También puede ejecutarse la aplicación desde el entorno virtual y utilizar únicamente la base de datos levantada con Docker Compose:

Arranque solo de MariaDB mediante Docker

```
docker compose up -d db
```

En este caso, la conexión local debe apuntar al puerto expuesto por Docker:

DATABASE_URL usando MariaDB en Docker

```
DATABASE_URL=mysql+pymysql://usuarioIAOrquestadaDB:  
CAMBIAR_PASSWORD_USUARIO@localhost:3307/ia_orquestada_db?charset=  
utf8mb4
```

Después pueden ejecutarse los comandos habituales:

Ejecución local usando base de datos Docker

```
python seed.py  
python run.py
```

Gestión del fichero requirements.lock

El fichero `requirements.lock` se utiliza para construir la imagen Docker de forma más controlada, fijando versiones y hashes. No debe editarse manualmente.

Cuando se modifique `requirements.txt`, se regenera con:

Regeneración de requirements.lock

```
python -m pip install pip-tools  
python -m piptools compile --generate-hashes --output-file=  
requirements.lock requirements.txt
```

Después se recomienda comprobar la construcción de la imagen:

Comprobación de construcción Docker

```
docker compose build --no-cache web
```


No se recomienda ejecutar `pip freeze > requirements.txt` tras instalar herramientas auxiliares, ya que podría añadir dependencias que no forman parte de la ejecución real de la aplicación.

Problemas frecuentes en entorno local

Los problemas más habituales se resumen en la Tabla D.6.

Problema	Comprobación o solución
Versión incorrecta de Python	Ejecutar <code>python3.11 -version</code> antes de crear el entorno virtual.
Error de conexión con MariaDB	Revisar <code>DATABASE_URL</code> y comprobar el servicio con <code>sudo systemctl status mariadb</code> .
Puerto 5000 ocupado	Comprobar el proceso con <code>sudo lsof -i :5000</code> .
Puerto de MariaDB ocupado	Revisar la configuración local o usar la base de datos Docker expuesta en otro puerto.
Errores con OpenCV, MediaPipe o Ultralytics	Instalar las librerías del sistema indicadas en los requisitos previos.
Conflictos de dependencias	Recrear el entorno virtual e instalar de nuevo <code>requirements.txt</code> .

Tabla D.6: Problemas frecuentes en entorno local de desarrollo.

Para recrear el entorno virtual:

Recreación del entorno virtual

```
deactivate
rm -rf .venv
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

D.5. Pruebas del sistema

Pruebas unitarias del backend

Durante el desarrollo se han realizado pruebas unitarias centradas principalmente en el backend, ya que ahí se concentra la parte crítica del sistema:

autenticación, control de acceso, gestión de usuarios, contratación de servicios, filtrado de pipelines, ejecución de modelos de IA, persistencia de resultados y tratamiento de errores.

Las pruebas se encuentran en el directorio `tests/`.

Para aislar las pruebas del entorno real se utiliza una configuración específica con SQLite en memoria, evitando modificar la base de datos MariaDB de desarrollo o despliegue.

Ejecución de las pruebas

Con el entorno virtual activado, las pruebas pueden ejecutarse con:

Ejecución de todas las pruebas

```
pytest
```

Para obtener una salida más detallada:

Ejecución detallada

```
pytest -v
```

También puede ejecutarse un archivo concreto:

Ejecución de un archivo de pruebas

```
pytest tests/test_auth_controller.py
```

O una prueba específica:

Ejecución de una prueba concreta

```
pytest tests/test_auth_controller.py::test_login_exito_y_fallos
```

Ejecución de pruebas con cobertura

La cobertura sobre el paquete principal se obtiene con:

Cobertura en terminal

```
pytest --cov=app --cov-report=term-missing
```

Para generar un informe HTML:

Cobertura en HTML

```
pytest --cov=app --cov-report=html
```

El informe se genera en:

Directorio del informe HTML

```
htmlcov/
```

Informe de cobertura del backend

Durante el desarrollo se generó un informe completo con:

Comando usado para el informe completo

```
pytest --cov=app --cov-report=term-missing --cov-report=html tests
```

El resultado obtenido fue una cobertura global del **93%** sobre el paquete principal `app`, .

La Figura D.2 muestra el informe de cobertura generado.

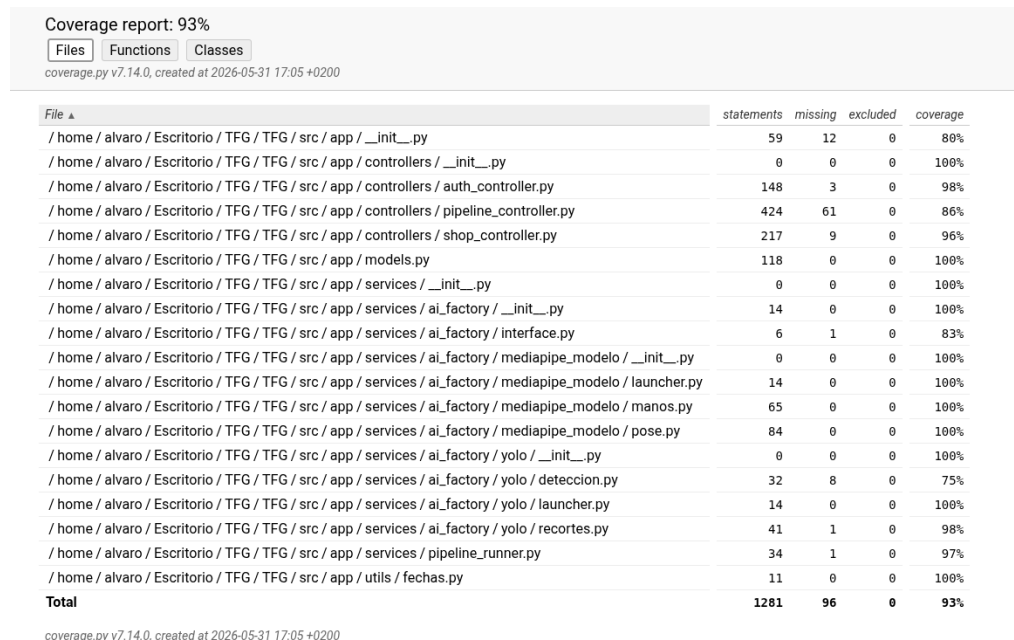


Figura D.2: Informe de cobertura del backend generado mediante `pytest-cov`.

La cobertura no garantiza por sí sola la ausencia de errores, pero se ha utilizado como métrica de apoyo para detectar zonas menos probadas y reforzar los componentes principales del backend.

Integración continua y análisis estático con SonarCloud

Además de las pruebas ejecutadas en local, el proyecto se ha integrado con GitHub Actions y SonarCloud. Esta integración permite ejecutar análisis automáticos orientados a detectar problemas de mantenibilidad, seguridad, duplicación, complejidad o prácticas no recomendadas.

El análisis se ejecuta automáticamente cuando se realizan cambios sobre las ramas principales del proyecto, concretamente en operaciones de *push* o *pull request* hacia `main` o `develop`. De esta forma, la calidad del código puede revisarse durante el desarrollo y antes de integrar cambios en ramas estables.

El flujo de integración continua prepara un entorno Linux, configura Python 3.11, instala las dependencias del sistema necesarias para las librerías de visión por computador, instala las dependencias Python del proyecto y ejecuta las pruebas con cobertura. Posteriormente, SonarCloud toma como entrada el informe de cobertura generado y realiza el análisis estático del código.

La Tabla D.7 resume los elementos principales de la configuración utilizada para el análisis.

Elemento	Configuración aplicada
Ramas analizadas	<code>main</code> y <code>develop</code> , tanto en <i>push</i> como en <i>pull request</i> .
Entorno de ejecución	Ubuntu con Python 3.11.
Dependencias del sistema	Librerías necesarias para ejecutar componentes relacionados con visión por computador.
Dependencias Python	Instalación de <code>requirements.txt</code> , <code>pytest</code> y <code>pytest-cov</code> .
Pruebas ejecutadas	Pruebas ubicadas en <code>src/tests</code> .
Informe de cobertura	Generación de <code>coverage.xml</code> para su consumo por SonarCloud.
Código fuente analizado	Directorio <code>src</code> , aplicando exclusiones justificadas.

Tabla D.7: Configuración principal del análisis automático con GitHub Actions y SonarCloud.

La configuración de SonarCloud distingue entre código fuente, pruebas, cobertura y recursos auxiliares. Esta separación es importante para que las métricas representen la calidad del código desarrollado y no se vean distorsionadas por archivos que no forman parte de la lógica productiva de la aplicación.

En concreto, el análisis toma como fuente principal el directorio `src`, pero excluye determinados elementos. Estas exclusiones no tienen como objetivo ocultar código relevante, sino evitar que SonarCloud evalúe archivos generados, datos temporales, entornos virtuales, recursos pesados o scripts auxiliares que no representan lógica de negocio. La Tabla D.8 resume las exclusiones más importantes y su justificación.

Elemento excluido	Motivo
<code>src/tests/**</code>	Las pruebas se declaran como código de test mediante <code>sonar.tests</code> ; no deben computar como código productivo.
<code>src/seed.py</code>	Es un script auxiliar para inicializar y reinicializar datos de demostración; no forma parte del flujo normal de ejecución de la aplicación.
<code>src/.env</code>	Fichero de configuración de entorno, con valores dependientes del despliegue y potencialmente sensibles.
<code>src/models/**</code>	Carpeta destinada a recursos locales de modelos de IA. No debe confundirse con <code>src/app/models.py</code> , que sí pertenece al código fuente de la aplicación.
<code>src/ejecucion_data/**</code> , <code>outputs/**</code> , <code>uploads/**</code> e <code>instance/**</code>	Directorios de ejecución, almacenamiento temporal o datos generados por la aplicación.
<code>__pycache__</code> , <code>.venv</code> , <code>venv</code> , <code>env</code> y <code>migrations</code>	Archivos generados automáticamente, entornos locales o contenido no mantenido manualmente como parte del código principal.

Tabla D.8: Exclusiones aplicadas en SonarCloud y justificación de cada una.

También se han definido exclusiones específicas para la cobertura. En este caso se excluyen las pruebas, el script de inicialización, el punto de entrada, la configuración y los archivos `__init__.py`. Estos elementos tienen poca lógica funcional propia o actúan como código de arranque, configuración o ensamblado. Por ello, incluirlos en la cobertura podría alterar la métrica

sin aportar información útil sobre la validación de la lógica principal del sistema.

El código más relevante de la aplicación, como controladores, servicios, factoría de modelos, ejecución de pipelines, entidades persistentes de la aplicación y funciones de procesamiento, permanece dentro del análisis. Por tanto, la configuración busca que las métricas se centren en el código mantenido y ejecutable del proyecto, evitando ruido provocado por recursos externos o generados.

Durante la integración se revisaron incidencias señaladas por la herramienta y se aplicaron correcciones relacionadas con la construcción Docker, la gestión de dependencias y determinados patrones sensibles desde el punto de vista de seguridad. De esta forma, SonarCloud se ha utilizado como apoyo a la revisión técnica del proyecto.

La estrategia de validación combina tres niveles:

- **pytest**: valida el comportamiento funcional del backend.
- **pytest-cov**: mide la cobertura de código ejercitada por las pruebas.
- **SonarCloud**: revisa calidad, seguridad y mantenibilidad mediante análisis estático.

La Figura D.3 muestra el informe de análisis estático realizado.

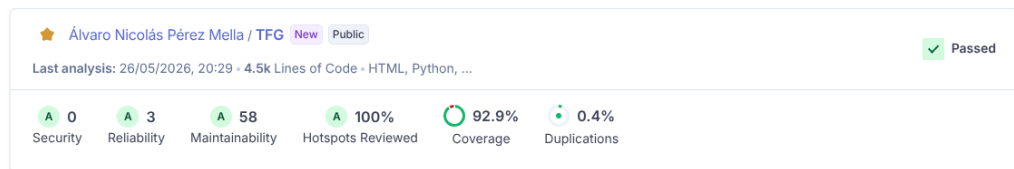


Figura D.3: Informe de análisis estático del proyecto generado mediante SonarCloud.

Apéndice E

Documentación de usuario

E.1. Introducción

Este apéndice recoge la documentación orientada al usuario final de la aplicación desarrollada en este Trabajo de Fin de Grado. A diferencia de la documentación técnica de programación, esta sección no describe la instalación interna del sistema, la configuración de Docker, la base de datos ni las dependencias del proyecto.

Desde el punto de vista del usuario, la aplicación se utiliza como una plataforma web. Por tanto, una vez desplegada por el personal técnico o por el administrador del sistema, el usuario solo necesita acceder a la dirección proporcionada mediante un navegador web.

El objetivo de este apéndice es explicar los requisitos básicos de acceso y dejar preparada la sección posterior del manual de usuario, donde se describirá el uso funcional de la aplicación: registro, inicio de sesión, ejecución de pipelines, consulta de resultados, gestión de compras y funciones de administración.

E.2. Requisitos de usuarios

Para utilizar la aplicación no es necesario instalar Python, MariaDB, Docker, Docker Compose ni ninguna dependencia interna del proyecto. Todos esos elementos forman parte del despliegue técnico y se documentan en el apéndice de documentación técnica de programación.

El usuario únicamente necesita disponer de:

- **Navegador web actualizado:** necesario para acceder a la interfaz de la aplicación.
- **Conexión a Internet:** o acceso a la red en la que esté desplegada la plataforma.
- **Dirección de acceso:** URL proporcionada por el administrador o responsable técnico.

En un entorno local de demostración, la aplicación puede estar disponible en:

url de acceso

`http://localhost:5000`

En caso de acceder desde otro equipo de la misma red, deberá utilizarse la dirección IP o URL proporcionada por el administrador del sistema.

En un despliegue real, el acceso se realizaría mediante la URL pública o privada configurada para la plataforma.

E.3. Instalación

Desde el punto de vista del usuario final, no es necesario instalar la aplicación ni preparar ningún componente técnico en su equipo.

La aplicación se utiliza directamente desde un navegador web, accediendo a la dirección proporcionada por el administrador o responsable del sistema. La instalación, configuración y despliegue de la plataforma corresponden al personal técnico y se describen en la documentación técnica de programación.

Por tanto, el usuario solo necesita disponer de un navegador web actualizado, conexión a Internet o acceso a la red donde esté desplegada la aplicación, y una cuenta válida para iniciar sesión.

La aplicación se ejecuta escuchando en `0.0.0.0:5000` para permitir conexiones desde fuera del contenedor. Sin embargo, cuando se accede desde el propio equipo anfitrión, la URL habitual de acceso es `http://localhost:5000`.

E.4. Manual del usuario

Problemas frecuentes de acceso

Los problemas más habituales desde el punto de vista del usuario se resumen en la Tabla E.1.

Situación	Posible causa o solución
La página no carga	Comprobar la conexión a Internet y que la URL introducida sea correcta.
La URL no es válida	Solicitar al administrador la dirección actual de la aplicación.
No se puede iniciar sesión	Revisar usuario y contraseña. Si el problema continúa, contactar con el administrador.
La cuenta aparece como no activa	La cuenta puede estar suspendida, dada de baja o pendiente de revisión.
No aparecen pipelines disponibles	Puede que el usuario no tenga contratado el plan o modelo necesario.
No se pueden descargar resultados	El archivo puede haber caducado o no estar disponible.

Tabla E.1: Problemas frecuentes de acceso para el usuario.

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo recoge una reflexión personal sobre la relación entre el Trabajo de Fin de Grado desarrollado y los principios de sostenibilidad trabajados durante la formación universitaria. Aunque el proyecto tiene un carácter principalmente técnico, centrado en el desarrollo de una aplicación web para la orquestación de modelos de inteligencia artificial aplicados al procesamiento de imágenes, su diseño también incorpora decisiones vinculadas con la sostenibilidad ambiental, social, económica y ética.

La sostenibilidad no se ha entendido únicamente como una cuestión medioambiental, sino como una forma de diseñar tecnología responsable, mantenible, segura y útil para las personas. En este sentido, el proyecto se relaciona especialmente con los Objetivos de Desarrollo Sostenible 8 y 9 de la Agenda 2030: trabajo decente y crecimiento económico, e industria, innovación e infraestructura.

F.2. Innovación, infraestructura y mantenibilidad

Uno de los aspectos más relevantes del proyecto es la construcción de una plataforma modular capaz de integrar distintos modelos de inteligencia artificial y diferentes modos de ejecución. La aplicación no se limita a ejecutar un único algoritmo, sino que permite organizar modelos en pipelines

configurables. Esto favorece una infraestructura software flexible, alineada con el ODS 9.

Desde el punto de vista de sostenibilidad tecnológica, esta decisión evita desarrollar una aplicación distinta para cada modelo o caso de uso. En lugar de duplicar soluciones, se ha diseñado una arquitectura basada en separación de responsabilidades, servicios y patrones de creación, de modo que la incorporación de nuevos modelos pueda realizarse con menor impacto sobre el código existente. Esta forma de trabajar contribuye a reducir deuda técnica y facilita la evolución futura del sistema.

También se ha incorporado Docker como mecanismo de despliegue reproducible. Esta decisión mejora la portabilidad del proyecto, reduce problemas derivados de diferencias entre entornos y facilita que otros usuarios puedan ejecutar la aplicación sin instalar manualmente todas las dependencias.

F.3. Uso responsable de recursos y datos

El proyecto incorpora decisiones orientadas a un uso más responsable de los recursos computacionales y del almacenamiento. Las imágenes y resultados generados durante las ejecuciones no se almacenan indefinidamente, sino que se gestionan como archivos temporales con políticas de expiración. Esta medida evita la acumulación innecesaria de datos, reduce el uso de almacenamiento y limita la permanencia de información sensible.

Además, se ha trabajado con modelos que pueden quedar disponibles de forma local una vez descargados, reduciendo la dependencia de llamadas externas continuas. Desde el punto de vista ético, se ha tenido en cuenta que el procesamiento de imágenes puede implicar información sensible. Por ello, el sistema evita exponer rutas internas de archivos y utiliza tokens para acceder a los resultados.

F.4. Relación con el trabajo decente y la formación profesional

El proyecto también se relaciona con el ODS 8, ya que desarrolla competencias técnicas aplicables al ámbito profesional: diseño de arquitecturas web, persistencia de datos, pruebas automatizadas, despliegue mediante contenedores, control de acceso y uso de modelos de inteligencia artificial. Estas competencias son relevantes para el desarrollo de software de calidad y para la incorporación responsable de tecnologías emergentes.

Durante el desarrollo he aprendido que la sostenibilidad de un sistema no depende solo de su funcionalidad inicial, sino también de su mantenibilidad, seguridad y capacidad de evolución. Un proyecto difícil de desplegar, probar o ampliar acaba consumiendo más recursos técnicos y humanos. Por ello, decisiones como la modularidad, la documentación, los tests unitarios y la cobertura del backend forman parte de una visión sostenible del desarrollo software.

F.5. Conclusión

La realización de este Trabajo de Fin de Grado me ha permitido aplicar la sostenibilidad desde una perspectiva informática práctica. El proyecto no resuelve por sí solo un problema medioambiental, pero sí incorpora principios de diseño responsable: reutilización, modularidad, control de recursos, protección de datos, despliegue reproducible y facilidad de mantenimiento.

Como aprendizaje personal, considero que la sostenibilidad curricular en ingeniería informática implica tomar decisiones técnicas pensando no solo en que el sistema funcione, sino en que pueda mantenerse, auditarse, ampliarse y utilizarse de forma segura. Esta visión resulta fundamental para desarrollar tecnología útil, responsable y alineada con las necesidades sociales y profesionales actuales.

Bibliografía

- [1] Python Cryptographic Authority. cryptography. <https://pypi.org/project/cryptography/>, 2026. [Online; accedido 22-mayo-2026].
- [2] SQLAlchemy Authors. Ssqlalchemy. <https://pypi.org/project/SQLAlchemy/>, 2026. [Online; accedido 22-mayo-2026].
- [3] The MediaPipe Authors. Mediapipe. <https://pypi.org/project/mediapipe/>, 2026. [Online; accedido 22-mayo-2026].
- [4] Creative Commons. Attribution 4.0 international cc by 4.0. <https://creativecommons.org/licenses/by/4.0/>, 2026. [Online; accedido 22-mayo-2026].
- [5] Flask-JWT-Extended contributors. Flask-jwt-extended. <https://pypi.org/project/Flask-JWT-Extended/>, 2026. [Online; accedido 22-mayo-2026].
- [6] Gunicorn contributors. Gunicorn. <https://pypi.org/project/gunicorn/>, 2026. [Online; accedido 22-mayo-2026].
- [7] Pillow contributors. Pillow. <https://pypi.org/project/Pillow/>, 2026. [Online; accedido 22-mayo-2026].
- [8] PyMySQL contributors. Pymysql. <https://pypi.org/project/PyMySQL/>, 2026. [Online; accedido 22-mayo-2026].
- [9] Tesoreria General de la Seguridad Social. Bases y tipos de cotizacion 2026, 2026. [Online; accedido 22-mayo-2026].

- [10] Ministerio de Trabajo y Economía Social. Actualizacion de las tablas salariales del xix convenio colectivo estatal de empresas de consultoria, tecnologias de la informacion y estudios de mercado y de la opinion publica, 2026. [Online; accedido 22-mayo-2026].
- [11] Boletin Oficial del Estado. Ley organica 3/2018, de 5 de diciembre, de proteccion de datos personales y garantia de los derechos digitales. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>, 2018. [Online; accedido 22-mayo-2026].
- [12] Boletin Oficial del Estado. Real decreto-ley 3/2026, de 3 de febrero, tarifa de primas para la cotizacion por accidentes de trabajo y enfermedades profesionales, 2026. [Online; accedido 22-mayo-2026].
- [13] NumPy Developers. Numpy. <https://pypi.org/project/numpy/>, 2026. [Online; accedido 22-mayo-2026].
- [14] Google AI Edge. Hand landmarks detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2026. [Online; accedido 22-mayo-2026].
- [15] Google AI Edge. Pose landmark detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker, 2026. [Online; accedido 22-mayo-2026].
- [16] Union Europea. Reglamento ue 2016/679, articulo 17, derecho de supresion. <https://www.boe.es/doue/2016/119/L00001-00088.pdf>, 2016. [Online; accedido 22-mayo-2026].
- [17] Free Software Foundation. Gnu affero general public license. <https://www.gnu.org/licenses/agpl-3.0.en.html>, 2026. [Online; accedido 22-mayo-2026].
- [18] GitHub. Gnu affero general public license v3.0. <https://choosealicense.com/licenses/agpl-3.0/>, 2026. [Online; accedido 22-mayo-2026].
- [19] Google. Model card hand tracking with fairness. https://storage.googleapis.com/mediapipe-assets/Model%20Card%20Hand%20Tracking%20%28Lite_Full%29%20with%20Fairness%20Oct%202021.pdf, 2021. [Online; accedido 22-mayo-2026].
- [20] Saurabh Kumar. python-dotenv. <https://pypi.org/project/python-dotenv/>, 2026. [Online; accedido 22-mayo-2026].

- [21] OpenCV on Wheels contributors. opencv-python-headless. <https://pypi.org/project/opencv-python-headless/>, 2026. [Online; accedido 22-mayo-2026].
- [22] Pallets Projects. Flask. <https://pypi.org/project/Flask/>, 2026. [Online; accedido 22-mayo-2026].
- [23] Pallets Projects. Flask-sqlalchemy. <https://pypi.org/project/Flask-SQLAlchemy/>, 2026. [Online; accedido 22-mayo-2026].
- [24] Pallets Projects. Werkzeug. <https://pypi.org/project/Werkzeug/>, 2026. [Online; accedido 22-mayo-2026].
- [25] pytest-dev team. pytest. <https://pypi.org/project/pytest/>, 2026. [Online; accedido 22-mayo-2026].
- [26] PyTorch Team. torch. <https://pypi.org/project/torch/>, 2026. [Online; accedido 22-mayo-2026].
- [27] PyTorch Team. torchvision. <https://pypi.org/project/torchvision/>, 2026. [Online; accedido 22-mayo-2026].
- [28] Ultralytics. Ultralytics. <https://pypi.org/project/ultralytics/>, 2026. [Online; accedido 22-mayo-2026].
- [29] Ultralytics. Ultralytics licensing. <https://www.ultralytics.com/license>, 2026. [Online; accedido 22-mayo-2026].